

AI를 내 편으로 만들고 10년을 따라잡는 법

당신이 ChatGPT만 쓰는 동안
누군가는 에이전트와 함께 일하고 있다

서문: 이 책의 유효기간은 1년이다

솔직하게 말씀드리겠습니다. 이 책의 유효기간은 1년입니다.

지금 이 순간에도 새로운 모델이 출시되고, 어제까지 최선이었던 워크플로우가 오늘 구식이 되고 있습니다. 제가 이 책을 쓰는 동안에도 도구의 버전이 바뀌었고, 제가 “이게 최선입니다”라고 적은 문장 위에 물음표를 붙여야 할 순간이 두 번은 있었습니다. 초안을 쓸 때 참고한 기능이 탈고 시점에 이미 deprecated 된 적도 있었습니다.

그런데도 이 책을 씁니다.

유효기간이 1년이라는 걸 알면서도 쓰는 이유가 있습니다. 지금 이 1년이, 누군가에게는 커리어 전체의 방향이 결정되는 1년이기 때문입니다. 지금 이 순간 어떤 감각을 장착하느냐에 따라, 3년 후의 시장에서 당신이 어디에 서 있을지가 정해집니다.

멘토링에서 본 것

저는 현업 개발자로 일하면서 주니어 개발자들을 멘토링하고 있습니다. 대기업, 스타트업, 부트캠프 출신까지 다양한 분들을 만납니다. 그리고 최근 1년간, 하나의 패턴을 너무 선명하게 목격하고 있습니다.

AI를 실무에 녹여 쓰는 개발자와 그렇지 않은 개발자 사이의 격차가, 제가 경험한 어떤 기술 격차보다 빠르게 벌어지고 있다는 것입니다.

이건 코딩 실력의 문제가 아닙니다. 머리가 나빠서가 아닙니다. 환경과 노출의 문제입니다.

대기업이나 빠르게 움직이는 스타트업에 있는 개발자들은 이미 AI 코딩 도구를 일상적으로 씁니다. 회사가 라이선스를 제공하고, 팀 내에서 프롬프트를 공유하고, “이거 AI한테 시켜봤어?”가 코드 리뷰에서 자연스럽게 나옵니다. 그 환경에 있으면 의식하지 않아도 감각이 쌓입니다.

하지만 그런 환경이 아닌 곳에 있는 분들은 완전히 다른 세상에 살고 있습니다. 소규모 회사에서 레거시 코드를 유지보수하는 분들, 취업 준비 중인 분들, 비전공자로 전향한 분들. “AI 코딩 도구가 좋다더라” 정도는 들었지만, 그걸 어떻게 실무에 쓰는지, 왜 써야 하는지, 안 쓰면 어떻게 되는지에 대한 감각 자체가 없습니다.

이건 그분들의 잘못이 아닙니다. 그냥 정보가 안 도달하는 곳에 있는 겁니다.

멘토링을 하다 보면 이런 대화가 반복됩니다.

“요즘 AI 도구 쓰세요?”

“아, 가끔 ChatGPT로 에러 메시지 검색은 해요.”

“개발할 때 AI 코딩 어시스턴트는요?”

“아직 안 써봤는데, 써봐야 하나요?”

이 질문을 들을 때마다 가슴이 답답해집니다. 써봐야 하느냐고요. 지금 업계가 어떻게 돌아가는지 아니면 그런 질문이 나올 수가 없습니다.

더 마음이 아픈 건, 이분들이 게으른 게 아니라는 겁니다. 오히려 열심히 합니다. 알고리즘 문제를 풀고, 사이드 프로젝트를 만들고, 기술 블로그를 읽습니다. 하지만 그 노력의 방향이 2023년에 머물러 있습니다. 세상은 이미 다른 게임을 하고 있는데, 이전 게임의 규칙으로 열심히 뛰고 있는 겁니다.

어느 멘토링 세션에서 경력 3년차 백엔드 개발자를 만났습니다. 실력 있는 분이었습니다. Spring Boot로 깔끔하게 서비스를 만들 줄 알았고, 테스트 코드도 꼼꼼히 작성했습니다. 그런데 AI 코딩 도구는 전혀 안 쓰고 있었습니다.

“회사에서 안 쓰니까요. 저도 필요성을 못 느꼈어요.”

한 시간 동안 제가 실제로 AI 도구를 써서 개발하는 과정을 보여드렸습니다. 그분의 반응이 아직도 기억납니다.

“이걸... 다들 이렇게 하고 있었어요?”

네. 다들 이렇게 하고 있었습니다. 다만 당신이 있는 곳에서는 그게 안 보였을 뿐입니다.

그 세션 이후 그분은 2주 만에 AI 코딩 도구를 자기 워크플로우에 완전히 통합했습니다. 생산성이 체감으로 2배 이상 올랐다고 했습니다. 능력이 없었던 게 아닙니다. 노출이 없었던 겁니다. 한 번 보고 나니 나머지는 스스로 해냈습니다.

이런 경험이 쌓이면서 생각했습니다. 이 이야기를 멘토링에서만 할 게 아니라, 글로 써야겠다고. 제가 직접 만나지 못하는 수많은 “3년차 백엔드 개발자”들에게도 이 감각을 전달해야겠다고.

숫자가 말해주는 현실

감정적으로만 말씀드리는 게 아닙니다. 채용 시장에서 지각변동이 일어나고 있습니다. 주니어 개발자가 하던 일의 상당 부분을 AI가 해내기 시작하면서, 기업들이 채용 전략을 바꾸고 있습니다. 구체적인 숫자는 2장에서 다루겠습니다. 지금은 하나만 기억하십시오. 변화의 방향은 이미 정해졌고, 속도는 당신이 생각하는 것보다 빠릅니다.

“한국은 아직 아니지 않나요?”라고 생각하실 수 있습니다. 맞습니다. 한국은 아직 미국보다 체감 속도가 느립니다. 하지만 그게 더 위험합니다. 변화가 느리게 와서 준비할 시간이 있는 게 아니라, 느리게 오다가 한꺼번에 몰아칠 겁니다. 한국 IT 산업의 트렌드가 미국을 1~2년 시차로 따라가는 건 이 업계에 계신 분이라면 다들 아실 겁니다.

클라우드, 컨테이너, 마이크로서비스가 다 그랬습니다. 미국에서 2~3년 먼저 터지고, 한국에서는 “아직 우리는 아니야”라고 하다가, 어느 날 갑자기 채용 공고마다 “Docker/Kubernetes 경험 필수”가 붙었습니다. AI 코딩 도구도 같은 경로를 밟고 있습니다. 다만 이번에는 그 시차가 훨씬 짧을 겁니다.

지금 미국에서 일어나고 있는 일이, 곧 여기서 일어납니다.

이 책이 도구 튜토리얼이 아닌 이유

시중에 AI 코딩 도구 사용법을 알려주는 콘텐츠는 이미 넘칩니다. “Cursor 이렇게 쓰세요”, “Copilot 단축키 모음”, “ChatGPT 프롬프트 템플릿 100선” 같은 것들이요. 저도 그런 글을 읽고, 영상을 보고, 도움을 받았습니다.

하지만 그런 콘텐츠만으로는 부족합니다.

도구 사용법을 알려주는 건 “자동차 페달 밟는 법”을 알려주는 것과 같습니다. 필요하지만 충분하지 않습니다. 페달을 밟을 줄 안다고 운전을 잘하는 게 아니듯, AI 도구의 버튼을 누를 줄 안다고 AI 네이티브 개발자가 되는 게 아닙니다.

더 정확하게 말씀드리면 이렇습니다. AI 코딩 도구를 “쓰는” 것과 AI 코딩 도구를 “부리는” 것은 완전히 다릅니다. 쓰는 건 도구가 제안하는 걸 수동적으로 받아들이는 겁니다. 부리는 건 내가 원하는 결과물을 얻기 위해 도구를 전략적으로 조작하는 겁니다. 이 차이가 결과물의 품질을 결정합니다.

이 책이 다루려는 건 그 깊은 층위의 이야기입니다.

AI와 함께 일한다는 것이 개발자의 사고방식을 어떻게 바꾸는가. 코드를 설계하고, 문제를 분해하고, 품질을 판단하는 방식이 어떻게 달라지는가. “코드를 직접 치는 사람”에서 “AI가 만든 결과물을 판단하고 교정하는 사람”으로 역할이 이동할 때, 어떤 역량이 더 중요해지고 어떤 역량이 덜 중요해지는가.

이건 특정 도구에 종속되지 않는 이야기입니다.

Cursor가 사라져도, Copilot이 다른 것으로 대체되어도, Claude가 GPT-7에 밀려나도, 이 책에서 이야기하는 사고방식과 접근법은 유효할 겁니다. 도구는 바뀝니다. 하지만 AI와 협업하는 개발자의 사고 체계는 쉽게 바뀌지 않습니다.

1년 후에 이 책에 나온 특정 기능 설명이 맞지 않게 될 수는 있습니다. 하지만 이 책이 전달하려는 패러다임 자체가 무효화되지는 않을 겁니다. 저는 그 패러다임을 전달하기 위해 이 책을 씁니다.

누구를 위한 책인가

이 책은 주로 이런 분들을 위해 썼습니다.

경력 1~5년차 개발자로, AI 코딩 도구를 써보긴 했는데 “제대로” 쓰고 있는 건지 확신이 없는 분. 소규모 회사나 SI 환경에서 일하면서 AI 도구 도입에 대한 논의 자체가 없는 환경에 있는 분. 부트캠프를 수료하고 취업을 준비하면서, 면접에서 “AI 도구 활용 경험”을 물어보는데 뭘 어떻게 답해야 할지 막막한 분. 시니어인데 팀에 AI 도구를 도입하려고 하니 어디서부터 시작해야 할지 모르겠는 분.

한 줄로 요약하면, “AI가 개발을 바꾸고 있다는 건 아는데, 나는 구체적으로 뭘 어떻게 해야 하지?”라는 질문을 가진 분들을 위한 책입니다.

특히 한 가지 강조하고 싶은 게 있습니다. 이 책은 “뒤쳐진 사람들을 위한 구제책”이 아닙니다. 뒤쳐진 게 아닙니다. 시작 안 한 겁니다. 시작하면 됩니다.

반대로 이런 분들에게는 이 책이 맞지 않을 수 있습니다. 이미 AI 코딩 도구를 능숙하게 쓰면서 자기만의 워크플로우를 구축한 분. AI/ML 모델 자체를 연구하거나 개발하는 분. 이 책은 AI를 만드는 사람이 아니라, AI를 써서 소프트웨어를 만드는 사람을 위한 책입니다.

이 책의 구성

이 책은 크게 세 부분으로 나뉩니다.

첫째, 왜 지금인가 (1~3장). AI가 개발자의 일을 실제로 어떻게 바꾸고 있는지, 바뀐 환경에서 살아남으려면 무엇이 달라져야 하는지를 이야기합니다. 패러다임이 어떻게 전환되고 있고, 그 전환이 개발자 간의 양극화를 어떻게 가속하고 있는지를 데이터와 실제 사례를 통해 보여드리고, 같은 AI를 쓴다고 말하면서도 하늘과 땅 차이가 나는 **도구 갭**의 실체를 파헤칩니다.

둘째, 실전 워크플로우. AI 코딩 도구를 활용한 개발 프로세스를 처음부터 끝까지 다룹니다. 프로젝트 설정, 코드 작성, 디버깅, 리팩터링, 테스트, 코드 리뷰까지. 이 부분이 1년 후에 가장 많이 낡을 부분이기도 합니다. 하지만 “어떤 도구의 어떤 기능”보다 “어떤 상황에서 어떤 판단을 하는가”에 초점을 맞추었기 때문에, 도구가 바뀌어도 판단의 프레임워크는 남을 겁니다.

셋째, 생존 전략. AI 시대에 개발자로서 커리어를 어떻게 설계할 것인가. 어떤 역량에 투자해야 하고, 어떤 역량은 AI에게 위임해도 되는가. 주니어에서 시니어로 성장하는 경로가 어떻게 달라지는가. 이 부분은 가장 주관적이지만, 동시에 가장 오래 유효할 부분이기도 합니다.

각 챕터 끝에는 스스로 점검할 수 있는 셀프 체크, 바로 실천할 수 있는 미션, AI와 함께 활용하는 프롬프트 예시를 넣었습니다. 각 챕터는 독립적으로 읽을 수 있도록 구성했습니다. 처음부터 순서대로 읽으셔도 좋고, 자신에게 급한 챕터부터 펼쳐보셔도 좋습니다.

솔직한 고백 하나

이 책을 쓰면서 가장 많이 한 고민은 “이걸 내가 쓸 자격이 있나”였습니다.

저도 AI 네이티브 개발의 전문가가 아닙니다. 모든 답을 가지고 있지 않습니다. 하루가 다르게 바뀌는 이 분야에서 “전문가”라는 타이틀 자체가 좀 우습습니다. 6개월 전의 전문가적 판단이 지금은 틀린 것으로 판명나기도 하는 분야입니다.

그러면 앞서 가는 사람이나? 그것도 아닙니다. 솔직히 ‘앞’이라는 게 뭘까요. 이 분야에서 3개월 먼저 시작한 것이 우위가 되는 시대는 이미 지났습니다. 어제의 선두가 오늘의 후발 주자가 되고, 방향을 잘못 잡으면 아무리 빨리 달려도 멀어지기만 하는 세계입니다. 저도 수없이 방향을 틀었고, 잘못된 확신에 시간을 낭비한 적이 있습니다.

다만 한 가지는 확실합니다. 저는 이 변화의 한가운데에서 일하고 있고, 동시에 그 변화에서 소외된 사람들을 매주 만나고 있습니다. 최전선에 있는 사람의 시점과, 그 최전선이 어디인지조차 모르는 사람의 시점. 이 둘 사이를 매주 오가는 사람이 보는 풍경이 있습니다. 그 풍경을 가감 없이 전달하는 것. 그것이 이 책을 쓰는 이유입니다.

멘토링에서 매주 몇 명에게만 할 수 있는 이야기를, 이 책을 통해 더 많은 분들에게 하는 겁니다.

1년 후에

1년 후, 이 책의 구체적인 내용 중 일부는 분명히 낡아 있을 겁니다. “그때는 그게 최선이었지”라고 할 부분이 생길 겁니다. 제가 추천한 도구가 서비스를 종료했을 수도 있습니다. 제가 “아직 먼 미래”라고 한 것이 벌써 현실이 되어 있을 수도 있습니다.

하지만 1년 후에도 변하지 않을 것이 있습니다.

AI를 도구로 제대로 부릴 줄 아는 개발자와 그렇지 못한 개발자 사이의 격차는 더 벌어져 있을 거라는 것. 그리고 그 격차를 만드는 핵심 요인은 특정 도구의 숙련도가 아니라, AI와 협업하는 방식에 대한 이해라는 것. 그 이해를 가진 사람은 도구가 바뀌어도 금방 적응합니다. 그 이해 없이 도구만 익힌 사람은, 도구가 바뀔 때마다 처음부터 다시 시작해야 합니다.

이 책이 1년 후에 낡더라도, 이 책을 읽고 바뀐 당신의 사고방식은 낡지 않습니다.

그러니까 유효기간이 1년인 이 책을, 지금 읽으셔야 합니다.

시작하겠습니다. # 1장. 게임의 규칙이 바뀌다 – AI, 도구의 진화가 아닌 패러다임 쉬프트

옆자리 동료와 당신과 다른 게임을 하고 있습니다

오늘도 IntelliJ를 열었습니다. JIRA 티켓을 확인하고, Spring Boot 프로젝트에서 컨트롤러를 작성하기 시작합니다. import문을 정리하고, DTO 클래스를 만들고, 서비스 레이어를 구성합니다.

옆자리 동료는 터미널에 자연어로 뭔가를 입력하고 있습니다. 5분 뒤, 동료의 화면에는 컨트롤러, 서비스, 리포지토리, 테스트 코드가 이미 생성되어 있습니다. 동료는 생성된 코드를 리뷰하고, 비즈니스 로직 몇 줄을 수정한 뒤 PR을 올립니다.

당신은 아직 DTO를 만들고 있습니다.

이것은 능력의 차이가 아닙니다. 게임 자체가 다른 겁니다.

지금 벌어지고 있는 일은 “도구가 좋아졌다”는 수준이 아닙니다.

왜 지금이 개발의 역사가 뒤집히는 순간인가?

이 질문에 답하기 위해, 우리는 먼저 과학사에서 가장 영향력 있는 개념 하나를 빌려와야 합니다.

토머스 쿤의 패러다임 쉬프트

1962년, 미국의 과학철학자 토머스 쿤(Thomas Kuhn)은 『과학혁명의 구조(The Structure of Scientific Revolutions)』라는 책을 출간합니다. 이 책에서 쿤은 과학의 발전에 대한 통념을 완전히 뒤엎는 주장을 펼칩니다.

“과학은 점진적으로 발전하지 않는다.”

쿤에 따르면, 과학은 지식이 차곡차곡 쌓여 올라가는 계단이 아닙니다. 기존 체계로 설명할 수 없는 현상들이 하나둘 누적되다가, 임계점을 넘는 순간 기존 체계가 무너지고 새로운 체계가 들어섭니다. 쿤은 이것을 **불연속적 혁명(Revolution)**이라 불렀습니다.

천동설에서 지동설로의 전환을 떠올려 보십시오. 천동설 체계 안에서 행성의 역행 운동은 점점 더 복잡한 주전원(epicycle)을 추가해야만 설명할 수 있었습니다. 하지만 코페르니쿠스가 태양을 중심에 놓는 순간, 그 모든 복잡성이 단순한 타원 궤도로 해소되었습니다. 이것이 패러다임 쉬프트입니다.

쿤이 특히 강조한 개념이 하나 더 있습니다. **공약 불가능성(Incommensurability)**입니다. 새로운 패러다임은 이전 패러다임과 비교 자체가 불가능합니다. 관점이 다르고, 사용하는 언어가 다르며, 문제를 해결하는 방식 자체가 다릅니다. 뉴턴 역학의 관점에서 양자역학을 평가하는 것이 무의미하듯, 이전 패러다임의 잣대로 새로운 패러다임의 가치를 측정하려는 시도는 본질적으로 불가능합니다.

지금 소프트웨어 개발에서 벌어지고 있는 일이 정확히 이것입니다. AI가 몇 초 만에 수백 줄의 코드를 생성하고, 자연어 대화만으로 애플리케이션의 골격이 만들어지는 현실. 이것은 기존 개발 방식을 ‘좀 더 빠르게’ 하는 것이 아닙니다. 개발이라는 행위 자체의 정의를 바꾸고 있습니다.

첫 번째 유비: 마차에서 자동차로 - “더 빠른 말”의 오류

19세기 말, 운송 산업의 핵심 과제는 명확했습니다. 더 빠르고, 더 강하고, 더 오래 달릴 수 있는 말을 만드는 것이었습니다. 품종을 개량하고, 사료를 최적화하고, 채찍질 기술을 연마했습니다. 마차의 바퀴를 더 가볍게 만들고, 차체의 공기역학을 개선했습니다. 이 모든 노력은 하나의 전제 위에 서 있었습니다. 운송의 미래는 '말'이라는 전제.

그러다 칼 벤츠와 헨리 포드가 등장합니다. 자동차 - 당시에는 '말이 없는 마차(horseless carriage)'라고 불렸습니다 - 는 단순히 빠른 말이 아니었습니다. 말이라는 변수 자체를 방정식에서 제거해 버린 것입니다.

헨리 포드는 이렇게 말했습니다.

“사람들에게 무엇을 원냐고 물었다면, 그들은 '더 빠른 말'이라고 대답했을 것이다.”

이 문장이 중요한 이유는, 기존 패러다임 안에 있는 사람들은 새로운 패러다임을 상상할 수조차 없다는 쿤의 통찰과 정확히 일치하기 때문입니다. 마차 시대의 사람들에게 “미래에는 말이 필요 없어집니다”라고 말하면, 그들은 비웃었을 것입니다. 말 없이 어떻게 짐을 나르고, 사람을 이동시키느냐고 반문했을 것입니다.

자동차의 등장은 단순히 운송 수단 하나를 바꾼 것이 아니었습니다. 도시의 구조가 변했습니다. 도로가 포장되었고, 주유소가 세워졌으며, 신호등이 만들어졌습니다. 마부라는 직업은 사라졌지만, 운전기사, 정비사, 도시 설계사, 교통 엔지니어라는 새로운 직업군이 탄생했습니다. 판이 완전히 바뀐 겁니다.

이제 이 비유를 현재로 가져오겠습니다.

지금 코딩 속도를 10% 올리려고 타이핑 연습을 하는 것은, 자동차 시대에 말의 근육을 키우는 것과 다를 바 없습니다.

기존 패러다임의 효율을 극대화하려는 노력 자체가 방향을 잘못 잡은 것입니다.

물론 자동차가 등장한 직후에도 말은 당분간 도로 위를 달렸습니다. 하지만 방향은 이미 정해져 있었습니다. 중요한 것은 변화의 속도가 아니라, 변화의 방향이거든요. 그리고 지금, 그 방향은 이미 정해졌습니다.

두 번째 유비: 사실주의 회화에서 인상주의로 - 기술이 예술을 죽였는가

19세기 이전, 서양 회화에서 화가의 최고 덕목은 명확했습니다. 눈에 보이는 그대로를 화폭에 똑같이 재현하는 기술(Skill)이었습니다. 인물화의 피부결, 정물화의 과일 표면에 맺힌 이슬, 풍경화의 빛과 그림자. 이 모든 것을 얼마나 정밀하게 재현할 수 있느냐가 화가의 실력을 가르는 잣대였습니다. 수년간의 도제 수업과 아카데미 훈련은 바로 이 '재현 기술'을 연마하기 위한 과정이었습니다.

그런데 1839년, 사진술이 발명됩니다. 루이 다게르의 다게레오타입은 현실을 화폭 위에 재현하는 데 수주가 걸리던 작업을 단 몇 분으로 단축시켰습니다. 화가들은 경악했습니다. 프랑스 화가 폴 들라로슈(Paul Delaroche)는 이렇게 선언했습니다.

“오늘로서 회화는 죽었다.”

당시의 시각에서 이 선언은 합리적이었습니다. 회화의 핵심 가치가 '사실적 재현'이라면, 그것을 완벽하게 해내는 기계가 등장한 이상, 회화의 존재 이유는 사라진 것입니다.

하지만 회화는 죽지 않았습니다.

오히려 사진술의 등장 이후 회화는 역사상 가장 폭발적인 창조의 시대를 맞이합니다. 인상주의가 탄생한 것입니다. 모네는 빛의 순간적 인상을 포착했고, 고흐는 감정과 내면의 격동을 캔버스에 쏟아냈으며, 세잔은 사물의 본질적 구조를 해체하고 재구성했습니다. 이들은 '똑같이 그리는 노동'을 기계에게 넘기고, 대신 자신만의 해석과 느낌, 즉 **'무엇을(What)'이 아닌 '왜(Why)'**를 그리기 시작했습니다.

여기서 핵심적인 전환이 일어났습니다. 회화의 가치 기준 자체가 바뀐 것입니다. '얼마나 정확하게 재현하느냐'에서 '어떤 시각으로 세계를 해석하느냐'로. 기술적 숙련도가 아닌, 예술적 비전이 화가를 정의하는 기준이 된 것입니다.

소프트웨어 개발에서도 똑같은 가치 기준의 재정의가 일어나고 있습니다. **개발의 가치 기준이 바뀌고 있습니다.** '얼마나 빨리 코드를 짜느냐'에서 **'어떤 시스템을 설계하느냐'**로. 코드를 빠르게 타이핑하는 능력은 더 이상 개발자를 정의하는 기준이 아닙니다.

지금 AI가 소프트웨어 개발에서 하고 있는 일이 정확히 이것입니다. AI가 코드를 '짜는(재현하는)' 기술을 가져갔습니다. 함수를 작성하고, 알고리즘을 구현하고, 보일러플레이트를 생성하는 일 - 한때 개발자의 핵심 역량이라 여겨졌던 이 기술적 재현 능력을 AI가 빠르고 정확하게 수행합니다.

들라로슈의 선언을 빌려 말하자면, “오늘로서 코딩은 죽었다”고 할 수 있을지 모릅니다. 하지만 회화가 죽지 않았듯, 개발도 죽지 않을 것입니다. 대신 개발자의 역할이 재정의됩니다. 코드를 타이핑하는 사람에서, 시스템을 '해석'하고 '의도'를 불어넣는 아키텍트로. 인상주의 화가들이 카메라가 대체할 수 없는 인간 고유의 영역 - 해석, 감정, 비전 - 으로 이동했듯, 개발자는 AI가 대체할 수 없는 영역 - 의도의 정의, 시스템의 설계, 가치의 판단 - 으로 이동해야 합니다.

세 번째 유비: 필사본에서 인쇄술로 - 지식 독점의 붕괴

멘토링을 하다 보면, 이런 질문을 받을 때가 있습니다. “그래도 기본기는 직접 쳐봐야 느는 거 아니가요? 알고리즘을 안 보고 짤 수 있어야 진짜 아니가요?”

그럴 때 저는 이 이야기를 합니다.

중세 유럽에서 책 한 권은 경이로운 물건이었습니다. 수도원의 필사실(scriptorium)에서 수도사들이 양피지 위에 한 글자 한 글자 손으로 베껴 써야 했기 때문입니다. 한 권의 성경을 완성하는 데 수개월, 때로는 수년이 걸렸습니다. 그 결과, 지식은 극소수의 권력자만이 소유할 수 있는 희소 자원이었습니다. 이 시대의 ‘현자’란 지식을 많이 ‘외우고 있는’ 사람을 의미했습니다. 기억력이 곧 지적 권위였습니다.

1440년대, 요하네스 구텐베르크가 금속 활자 인쇄술을 발명합니다. 이 발명은 지식의 복제 비용을 사실상 0에 수렴시켰습니다. 책은 더 이상 성스러운 유일본이 아니라, 무한히 복제할 수 있는 상품이 되었습니다. 그리고 이 기술적 변화는 문명 전체를 뒤흔들었습니다. 종교 개혁이 가능해졌고, 대학이 확산되었으며, 과학 혁명의 토대가 마련되었습니다.

필사생(Scribe)이라는 직업은 소멸했습니다. 하지만 그 자리에 편집자, 출판인, 서평가, 번역가, 저널리스트라는 새로운 직업군이 탄생했습니다. 지식을 ‘복제하는 기술’이 가치를 잃은 대신, 지식을 ‘골라내고, 해석하고, 전파하는 능력’이 새로운 가치가 된 것입니다.

이 유비는 현재의 소프트웨어 개발에 직접적으로 대응합니다. 문법(Syntax) 지식의 희소성은 이제 0입니다. `public static void main` 을 안 보고 칠 수 있는 것, 특정 프레임워크의 어노테이션을 줄줄이 외우는 것, 알고리즘 문제를 백지 위에서 풀어내는 것 - 한때 개발자의 역량을 증명하는 리트머스 시험이었던 이런 능력들은, AI가 0초 만에 생성할 수 있는 재화가 되었습니다. 코드를 얼마나 외우느냐는 더 이상 권력이 아닙니다.

그렇다면 새로운 권력은 무엇일까요? 구텐베르크 이후의 세계가 알려주듯, 그것은 **지식을 골라내고 검증해서 가치를 만드는 능력**입니다. AI가 생성한 코드가 올바른지 판단하는 능력, 수십 가지 가능한 구현 중 이 맥락에서 최선인 것을 선택하는 능력, 단편적 코드 조각들을 일관된 시스템으로 엮어내는 능력. 이것이 인쇄술 이후의 세계에서 편집자와 출판인이 차지했던 자리와 같은, AI 시대 개발자의 새로운 영역입니다.

코드 생성 능력이 민주화된 세상에서는, 누구나 코드를 만들 수 있습니다. 차별화는 더 이상 코드를 ‘짤 수 있느냐’가 아니라, ‘무엇을 짤 것인가를 판단할 수 있느냐’에 달려 있습니다.

프로그래밍 세 시대: 하드웨어에서 의도로

이 패턴은 소프트웨어 개발의 역사에서도 정확히 반복되었습니다.

제1 패러다임: 기계어와 어셈블리어 시대 (The Era of Hardware)

프로그래밍의 시작점에서, 인간은 기계의 언어에 맞춰야 했습니다. 0과 1의 이진수, 레지스터 주소, 메모리 오프셋. 컴퓨터가 이해하는 방식으로 인간이 사고해야 했던 시기입니다. 가장 단순한 연산조차도 기계의 물리적 구조를 이해해야만 가능했거든요.

```
; Assembly: "1 + 2 = 3"을 위한 코드
MOV AX, 0001h
MOV BX, 0002h
ADD AX, BX
MOV [result], AX
```

1 더하기 2라는 인간에게는 자명한 연산을 위해, 개발자는 레지스터를 지정하고, 값을 이동시키고, 연산 결과를 메모리에 저장하는 과정을 일일이 기술해야 했습니다. 물리적 하드웨어의 제약이 곧 소프트웨어의 제약이었고, 기계를 이해하는 극소수만이 프로그래밍에 접근할 수 있었습니다.

이 시대의 핵심 질문은 이것이었습니다. “인간의 언어로 코딩할 수는 없을까?”

제2 패러다임: 고수준 언어와 추상화 시대 (The Era of Logic & Syntax)

그 질문에 대한 답이 컴파일러였습니다. 인간의 논리(Logic)를 코드로 작성하면, 기계가 알아서 기계어로 번역해 주는 프로그램. FORTRAN, C, Java, Python – 이 언어들은 개발자를 하드웨어의 족쇄에서 해방 시켰습니다.

```
// Java: "1 + 2 = 3"을 위한 코드
public class Main {
    public static void main(String[] args) {
        int a = 1;
        int b = 2;
        System.out.println(a + b);
    }
}
```

어셈블리와 비교하면 혁명적인 발전입니다. 하지만 이 패러다임에도 분명한 특성이 있었습니다. **문법 (Syntax)을 얼마나 잘 외우느냐, 알고리즘을 얼마나 효율적으로 짜느냐(How)** – 이것이 실력의 척도였습니다. 이 코드를 안 보고 치는 것이 “실력”이었습니다.

그리고 우리는 이 패러다임 안에서 **지난 50년**을 보냈습니다. 반세기 동안 프로그래밍의 본질은 바뀌지 않았습니다. 언어는 달라졌어도, 프레임워크는 진화했어도, 핵심은 같았습니다. 인간이 문법에 맞춰 로직을 직접 작성한다. IDE가 좋아지고, 자동완성이 등장하고, 스택 오버플로우가 지식의 접근성을 높였지만, 이 모든 것은 같은 패러다임 안에서의 점진적 개선이었습니다. 마차의 바퀴를 더 가볍게 만든 것이지, 말을 엔진으로 바꾼 것이 아니었습니다.

그런데 쿤이 말한 ‘이상 현상’이 나타나기 시작합니다. 소프트웨어의 복잡도가 폭발적으로 증가하면서, 인간 인지 능력의 한계에 봉착한 것입니다. 수십만 줄의 코드베이스, 수백 개의 마이크로서비스, 끊임없이 변하는 요구사항 – 인간이 문법 수준에서 이 모든 것을 통제하는 것은 물리적으로 불가능해졌습니다. 이상 현상이 누적되었고, 패러다임 쉬프트의 조건이 무르익었습니다.

제3 패러다임: AI와 의도 중심 개발 시대 (The Era of Intent)

그리고 지금, 세 번째 패러다임이 도래했습니다.

이전 전환이 “인간의 언어로 코딩할 수 있을까?”라는 질문에서 시작되었다면, 이번 전환의 질문은 이것입니다. **“어떻게 짤 것인가(How to write)”에서 “무엇을 만들 것인가(What to build)”로.**

From Syntax to Intent. 문법에서 의도로. 이것이 지금 일어나고 있는 전환의 본질입니다.

이 패러다임에서 인간과 AI의 역할은 명확하게 분리됩니다.

인간의 역할: 의도(Intent/Why)를 정의합니다. 설계(Design)를 수행하고, 결과를 검증(Review)합니다. 올바른 질문을 던지고, 올바른 맥락을 제공합니다.

AI의 역할: 구현(Implementation/How)을 처리합니다. 문법, 알고리즘, 보일러플레이트 – 한때 개발자의 시간 대부분을 차지하던 이 작업들을 AI가 수행합니다.

이것은 자동완성이 좀 더 똑똑해진 수준이 아닙니다. **개발자의 정의가 바뀌는 것입니다.** 마치 자동차가 ‘이동’의 정의를 바꾸었듯, 사진술이 ‘그림’의 정의를 바꾸었듯, 인쇄술이 ‘지식인’의 정의를 바꾸었듯.

코드를 보지 마십시오

저 자신이 이 전환을 체감한 순간이 있었습니다. 어느 날 밤, 새 프로젝트의 인증 모듈을 설계하고 있었습니다. 평소대로라면 Spring Security 설정 클래스를 열고, 필터 체인을 하나씩 직접 구성했을 겁니다. 그 날은 처음으로 AI 코딩 도구에 제가 원하는 인증 흐름을 자연어로 설명해 봤습니다. JWT 기반 인증, 리프레시 토큰 로테이션, Redis 세션 관리까지. 3분도 안 돼서 제가 한 시간은 걸렸을 코드가 화면에 나타났습니다.

그런데 저는 반사적으로 코드를 한 줄씩 읽기 시작했습니다. “이 부분은 토큰 만료 시간을 환경변수로 빼야지”, “여기는 예외 처리가 빠졌네.” 처음에는 이것이 ‘판단하는 사람’으로의 전환이라고 생각했습니다. 코드를 치는 사람에서 코드를 읽고 검증하는 사람으로. 꽤 그럴듯한 자기 서사였죠.

더 불편한 깨달음은 그 다음에 왔습니다. 제가 코드를 한 줄씩 읽고 있는 그 행위 자체가 이미 이전 패러다임의 습관이었던다는 것입니다. 그 시간에 저는 테스트 시나리오를 정의하고, 엡지 케이스를 설계하고, 시스템 전체의 인증 흐름이 비즈니스 요구사항과 맞는지를 검증해야 했습니다. 코드가 아니라 행동을 봐야 했습니다.

여기서 한 가지 분명히 하고 싶은 것이 있습니다.

“AI는 ○○○이고, 인간은 ○○○이다” - 이 문장 구조를 보셨을 때 익숙하지 않습니까? AI가 구현을 담당하고 인간이 설계를 담당한다, AI가 코드를 짜고 인간이 검증한다, AI는 도구이고 인간은 운전자다. 이 이분법은 듣기에는 그럴듯합니다. 하지만 이것 자체가 이전 패러다임의 프레임에 갇힌 사고입니다. 마차를 자동차로 바꾸면서 “말의 역할은 ○○이고, 엔진의 역할은 ○○이다”라고 역할 분담표를 만드는 것이 무의미했듯이요.

더 급진적으로 말하겠습니다.

‘코드’에 대해 이야기하는 것 자체가 이미 낡은 질문입니다.

“누가 코드를 짜느냐”가 중요한 게 아닙니다. “왜 아직도 코드를 보고 있느냐”가 진짜 질문입니다.

어셈블리 프로그래머는 기계어를 읽지 않습니다. C 프로그래머는 어셈블리를 읽지 않습니다. 고수준 언어 프로그래머는 바이트코드를 읽지 않습니다. 매번 추상화 레이어가 올라갈 때마다, 아래 레이어는 ‘불필요가 없는 것’이 되었습니다. 그 패턴이 지금, 코드 자체에 적용되고 있는 것입니다.

IDE를 켜지 마십시오. 코드를 스크롤하지 마십시오. 당신이 봐야 할 것은 코드가 아니라 시스템의 행동입니다. 의도대로 동작하는가? 엡지 케이스에서 무너지지 않는가? 비즈니스 요구사항을 충족하는가? 이 질

문들 사이에 코드가 끼어들 자리는 없습니다. 코드는 의도와 행동 사이에 자동으로 생성되는 중간 산출물입니다. 어셈블리가 그랬듯, 바이트코드가 그랬듯.

이것이 급진적으로 들립니까? 어셈블리 프로그래머에게 “앞으로는 기계어를 직접 쓸 일이 없어집니다”라고 말했을 때도 같은 반응이었을 겁니다. “기계어를 모르면 진짜 프로그래머가 아니지.” 그 말이 틀렸다는 건 역사가 이미 증명했습니다. 지금 “코드를 안 읽으면 진짜 개발자가 아니지”라고 느끼는 감정도, 같은 궤적 위에 있습니다.

부산물을 장인 정신이라 부르지 마십시오. 문제를 정의하고, 시스템의 행동을 설계하고, 결과가 의도와 일치하는지 증명하는 것. 그 과정에서 코드는 자동으로 생성됩니다. 보지 않아도 됩니다.

존재론적 변화

이 장을 마무리하며, 한 가지 분명히 해두어야 할 것이 있습니다.

우리가 지금 겪고 있는 변화는 단순히 IDE가 좋아진 수준이 아닙니다. 자동완성이 조금 더 똑똑해진 것도 아니고, 검색이 조금 더 편리해진 것도 아닙니다. 이것은 천동설이 지동설로 바뀌는 수준의 **존재론적 변화**입니다.

마차 제작자가 아무리 뛰어나도 자동차 시대에는 운전을 배워야 했듯, 사실주의 화가가 아무리 뛰어나도 사진술 이후에는 자기만의 시각 언어를 찾아야 했듯, 필사생이 아무리 정확해도 인쇄술 이후에는 편집과 큐레이션을 배워야 했듯 - 지금의 개발자도 새로운 패러다임에 맞는 새로운 역량을 갖춰야 합니다.

이 책의 나머지 장들에서 그 구체적인 방법을 다룰 것입니다. AI와 함께 설계하는 법, 의도를 정확하게 전달하는 법, AI가 생성한 코드를 검증하는 법, 그리고 이 모든 것을 하나의 워크플로우로 엮어내는 법을.

마차 시대가 끝났는데 말 엉덩이를 때리고 있지 마십시오. AI라는 엔진에 올라타서 어디로 갈지 (Direction)를 정하는 운전자가 되십시오.

이 선택은 당신의 몫입니다.

다음 장에서는 이 패러다임 전환이 개발자 시장에서 어떤 양극화를 만들고 있는지, 냉정한 데이터를 통해 보여드리겠습니다.

이 장을 마치며

셀프 체크

- ☐ “패러다임 쉬프트”의 개념을 내 상황에 대입해서 이해했다
- ☐ 마차/회화/인쇄술 유비 중 하나가 내 현재 상황과 겹치는 부분을 찾았다
- ☐ “코드를 짜는 사람”과 “코드의 방향을 정하는 사람”의 차이를 설명할 수 있다
- ☐ 내가 지금 하고 있는 개발 업무 중 AI가 대체할 수 있는 부분과 없는 부분을 구분할 수 있다

이 장의 미션

1. 내가 현재 하는 개발 업무를 두 가지로 분류해 보십시오. “마차의 바퀴를 개선하는 일”과 “자동차를 운전하는 일”. 각각 3개씩 적어보십시오.
2. 자신의 하루 업무 시간 중 “코드를 직접 타이핑하는 시간”의 비율을 대략이라도 추정해 보십시오. 그 시간이 줄어든다면, 나머지 시간에 무엇을 해야 할지 한 줄로 적어보십시오.

AI와 함께 활용하기

- **패러다임 자기 진단:** “나는 [연차]년차 [직무] 개발자입니다. 주로 하는 업무는 [업무 내용]입니다. 이 업무 중 AI가 대체할 수 있는 부분과, 인간만이 할 수 있는 부분을 분류해줘.”
- **역할 재정의 연습:** “다음은 내 일주일 업무 목록입니다: [목록]. 이 중 ‘구현(How)’ 중심인 것과 ‘의도(What/Why)’ 중심인 것을 분류하고, 구현 업무를 AI에게 위임할 방법을 제안해줘.” # 2장. 가진 자는 더 갖고, 없는 자는 더 잃는다 - AI 시대의 개발자 양극화

마태 효과: 기술 세계의 잔인한 법칙

성경 마태복음 25장 29절에는 이런 구절이 있습니다.

“무릇 있는 자는 받아 넉넉하게 되되, 없는 자는 그 있는 것까지 빼앗기리라.”

사회학에서는 이를 '마태 효과(Matthew Effect)'라 부릅니다. 부익부 빈익빈. 가진 자는 더 갖게 되고, 없는 자는 더 잃게 되는 구조적 현상. 이 현상이 지금, 소프트웨어 개발자 세계에서 가장 극적인 형태로 벌어지고 있습니다.

1장에서 “코드를 보지 마십시오”라고 했습니다. 코드는 중간 산출물일 뿐이라고요. 그런데 이 말의 이면에는 더 냉혹한 진실이 있습니다. AI는 곱셈기입니다. 덧셈이 아닙니다.

당신의 역량이 10이면, AI는 그것을 100으로 만들어 줍니다. 하지만 당신의 역량이 0이면, AI를 곱해도 여전히 0입니다. 이것이 AI 시대의 가장 냉혹한 진실입니다. AI는 실력 차이를 줄여주지 않거든요. 오히려 기존의 실력 차이를 기하급수적으로 증폭시킵니다.

시스템 아키텍처를 이해하는 개발자가 AI를 사용하면, 며칠 걸리던 설계를 몇 시간 만에 검증하고 구현합니다. 하지만 기본적인 코딩 문법만 아는 개발자가 같은 AI를 사용하면, AI가 생성한 코드가 왜 그렇게 작동하는지조차 이해하지 못합니다. 버그가 생기면 AI에게 “고쳐 달라”고 반복 요청하다가, 결국 더 큰 버그를 만들어냅니다.

같은 도구인데 결과는 정반대입니다. 이것이 곱셈기의 본질입니다.

그리고 이 곱셈 효과는 복리로 작용합니다. AI를 잘 활용하는 개발자는 더 빠르게 학습하고, 더 빠르게 성장하고, 더 어려운 문제를 맡게 되고, 그 과정에서 또다시 AI를 더 잘 활용하게 됩니다. 반대쪽은? 정확히 반대 방향으로 복리가 작동합니다. 성장 속도가 느려지고, 루틴한 업무에 머물고, AI를 활용할 기회조차 줄어들고, 격차는 매 분기마다 벌어집니다.

두 개의 속도로 달리는 시장

숫자를 보겠습니다. 감정이 아니라 데이터가 말하는 현실입니다.

사라지는 주니어, 치솟는 시니어

2024년부터 2025년 사이, 미국 상위 15개 테크 기업의 엔트리 레벨 채용 공고는 25% 감소했습니다 (SignalFire, 2024). 25%라는 숫자를 잠깐 생각해 보십시오. 4자리 중 1자리가 아예 사라진 것입니다. 그것도 구글, 메타, 아마존, 마이크로소프트 같은 글로벌 테크 기업에서 말입니다.

경기 탓이 아닙니다. 구조가 바뀐 겁니다.

엔지니어링 리더의 54%가 향후 주니어 개발자 채용을 줄일 계획이라고 응답했습니다(LeadDev Engineering Survey, 2025). 절반이 넘는 기업이 “주니어 개발자를 덜 뽑겠다”고 공식적으로 선언한 셈입니다. 그 이유는 명확합니다. AI 코딩 도구가 주니어 개발자가 하던 일의 상당 부분을 대체할 수 있게 되었기 때문입니다.

더 충격적인 숫자가 있습니다. 미국 노동통계국(BLS) 데이터에 따르면, 미국 프로그래머 고용은 2023년부터 2025년 사이에 27.5% 감소했습니다. 거의 3분의 1에 가까운 일자리가 2년 만에 증발한 것입니다. 스탠퍼드 대학 디지털 경제 연구소(Stanford Digital Economy Lab)의 분석(‘Canaries in the Coal Mine’, Brynjolfsson 외, 2025)에 따르면, 특히 22~25세 소프트웨어 개발자 고용은 2022년 말 대비 약 20% 감소했습니다.

한쪽에서 일자리가 사라지는 동안, 다른 쪽에서는 정반대의 현상이 벌어지고 있습니다.

폭등하는 AI 전문가 수요

AI 전문가에 대한 수요는 약 50% 증가했습니다(Deloitte Generative AI Market Report, 2024). 같은 시장, 같은 시기, 같은 산업에서 한쪽은 27.5% 줄고 한쪽은 약 50% 늘는 것입니다. 이것을 양극화라고 하지 않으면 무엇이라 부르겠습니까.

그리고 이 양극화는 연봉에서 더 극명하게 드러납니다. Agentic AI를 다룰 수 있는 개발자, 즉 AI 에이전트를 설계하고 오케스트레이션할 수 있는 개발자는 기존 동일 경력 대비 15~20%의 연봉 프리미엄을 받고 있습니다(업계 채용 시장 분석 기준). 같은 5년차 개발자인데, AI 네이티브 역량의 유무에 따라 한국 시장 기준으로 환산하면 연봉이 1,500만 원에서 2,000만 원까지 차이가 납니다.

정리하면 이렇습니다.

한쪽은 연봉이 올라갑니다. 다른 한쪽은 채용 공고 자체가 사라집니다.

이것이 2026년 2월 현재, 개발자 시장의 현실입니다.

환경이 만드는 격차: 노출 갭(Exposure Gap)

“AI를 잘 쓰는 것은 결국 개인의 노력 아닌가?”

아닙니다. 적어도 절반은 아닙니다.

멘토링에서 비슷한 시기에 만난 두 명의 3년차 개발자가 있었습니다. 둘 다 같은 대학 출신에, 비슷한 역량이었습니다. 한 명은 AI 도구 도입에 적극적인 스타트업으로 갔고, 다른 한 명은 SI 프로젝트 위주의 중견 기업에 갔습니다. 1년 후 다시 만났을 때, 스타트업에 간 친구는 Cursor로 하루에 PR을 서너 개씩 올리고 있었고, SI 쪽 친구는 AI 코딩 도구라는 카테고리 자체를 처음 들어봤습니다. 둘의 잠재력은 같았습니다. 달랐던 건 환경뿐이었습니다.

AI 도구에 대한 노출(exposure)은 개인의 의지보다 환경에 의해 결정되는 경우가 훨씬 많습니다. 이것을 저는 **노출 갭(Exposure Gap)**이라 부릅니다.

두 개의 세계

세계 A: 빅테크 또는 자금이 풍부한 스타트업. 이곳에서는 AI 코딩 도구가 기본입니다. 회사가 Cursor Business, GitHub Copilot Enterprise, Claude Code 비즈니스 계약으로 무제한 사용 환경을 전 직원에게 제공합니다. 팀 문화가 AI 활용을 장려하고, 코드 리뷰에서 “이 부분 AI에게 먼저 돌려봤어?”가 일상적인 질문입니다. 주간 회의에서 새로운 AI 워크플로우를 공유하고, AI 활용 우수 사례를 내부 위키에 기록합니다. 매달 새로운 AI 도구를 실험하는 것이 업무의 일부입니다.

세계 B: 전통적인 SI 업체, 중소 IT 기업, 또는 레거시 시스템을 유지보수하는 팀. 이곳에서는 여전히 기본 IDE만 사용합니다. AI 도구 예산이 없고, 도입 논의 자체가 없습니다. “AI는 아직 실무에서 쓰기엔 이릅니다”는 인식이 팀 전체에 퍼져 있습니다. 일부 개인이 몰래 무료 ChatGPT를 사용하지만, 체계적인 활용과는 거리가 멉니다.

같은 3년차 개발자입니다. 같은 대학을 나왔을 수도 있습니다. 비슷한 잠재력을 가지고 있을 수도 있습니다. 하지만 세계 A의 개발자는 Cursor와 Claude Code를 활용해 하루에 10배의 코드를 생산하고, 세계 B의 개발자는 IntelliJ에서 한 줄 한 줄 타이핑하고 있습니다.

이것은 재능의 차이가 아닙니다. 환경의 차이입니다.

그리고 이 환경의 차이가 시간이 지날수록 재능의 차이보다 훨씬 더 큰 실력 격차를 만들어냅니다. 세계 A의 개발자는 AI와 협업하며 프롬프트 엔지니어링, 코드 리뷰 자동화, 아키텍처 설계 보조 등 새로운 역량을 자연스럽게 습득합니다. 세계 B의 개발자는 그 역량이 존재한다는 사실조차 모를 수 있습니다.

가장 잔인한 점은 이것입니다. 세계 B에 있는 개발자는 자신이 뒤처지고 있다는 사실을 인식하기 어렵습니다. 주변 동료들도 같은 방식으로 일하고 있기 때문입니다. 물이 서서히 끓으면 개구리가 알아채지 못하듯이, 격차는 보이지 않게 벌어집니다. 그리고 어느 날 이직 시장에 나왔을 때, 비로소 현실과 마주합니다.

“왜 면접에서 AI 워크플로우를 물어보는 거지?” “왜 포트폴리오에 AI 활용 사례를 요구하는 거지?” “왜 나는 이런 질문에 대답할 수 없는 거지?”

그때는 이미 격차가 벌어질 대로 벌어진 후입니다.

지식의 복리: 2년 후의 갈림길

복리(compound interest)는 “세계 8번째 불가사의”라 불릴 만큼 강력한 개념입니다. 금융에서만 적용되는 개념이 아니잖아요. 지식과 역량에도 복리는 정확히 동일하게 작동합니다.

두 명의 개발자를 비교해 보겠습니다.

개발자 A: AI 활용 개발자

- AI를 활용해 기존 대비 3배의 피처를 출시합니다.
- 빠르게 성과를 내니, 팀에서 더 어려운 과제를 맡깁니다.
- 시스템 설계, 성능 최적화, 분산 아키텍처 같은 고난도 문제를 경험합니다.
- AI를 활용해 이 문제들을 분석하고 해결하면서, 아키텍처 감각이 빠르게 성장합니다.
- 1년 후: 시니어급 역량에 도달합니다.
- 2년 후: 아키텍트 역할을 수행하고, AI 에이전트를 활용한 개발 파이프라인을 설계합니다.

개발자 B: 전통적 개발자

- 기존 방식대로 안정적이지만 느린 속도로 피처를 출시합니다.
- 속도가 느리니, 루틴한 유지보수 업무가 주어집니다.
- CRUD 작성, 버그 수정, 테스트 케이스 추가 같은 반복 작업을 수행합니다.
- 아키텍처나 설계에 대한 경험이 쌓이지 않습니다.
- 1년 후: 여전히 같은 수준의 업무를 하고 있습니다.
- 2년 후: 본인이 하던 루틴한 업무가 AI로 자동화되기 시작합니다.

이것이 복리의 힘입니다. 처음에는 미미한 차이가 시간이 지나면서 건널 수 없는 간극이 됩니다.

개발자 A는 2년 후 아키텍트가 됩니다. 개발자 B는 2년 후 AI가 대체하는 일을 하고 있습니다.

과장이라고 생각하실 수 있습니다. 하지만 숫자로 계산해 보겠습니다.

설명을 위한 가정을 하나 세워 보겠습니다. AI 활용 개발자의 역량 성장률을 월 5%로, AI를 활용하지 않는 개발자의 성장률을 월 1%로 잡겠습니다. 이 수치는 멘토링 경험에 기반한 추정치이며, 실제 성장률은 개인과 환경에 따라 다릅니다. 하지만 복리의 원리를 이해하기에는 충분합니다.

12개월 후: - 개발자 A: 1.05의 12승 = 1.80배 (80% 성장) - 개발자 B: 1.01의 12승 = 1.13배 (13% 성장)

24개월 후: - 개발자 A: 1.05의 24승 = 3.23배 (223% 성장) - 개발자 B: 1.01의 24승 = 1.27배 (27% 성장)

소수점까지 정확한 숫자가 중요한 건 아닙니다. 핵심은 이겁니다. 2년이면 둘 사이에 2.5배 이상의 차이가 발생합니다. 이것은 단순히 코딩 속도의 차이가 아닙니다. 다룰 수 있는 문제의 복잡도, 설계 능력, 시스템 사고력 전반의 차이입니다.

그리고 이 격차는 3년, 5년이 지나면 말 그대로 ‘건널 수 없는’ 수준이 됩니다. 36개월 후에는 4.3배, 60개월 후에는 11.5배. 같은 시작점에서 출발했는데, 5년 후에는 한 명이 다른 한 명의 11배 역량을 가지게 됩니다.

잠깐, 이 숫자가 의미하는 것을 한 번 더 짚겠습니다. AI를 활용하는 개발자가 2년 만에 도달하는 3.23배 성장. 전통적인 방식으로 같은 수준에 도달하려면 얼마나 걸릴까요? 월 1%씩 성장한다고 가정하면, 약 118개월. 거의 **10년**입니다.

2년 대 10년. AI를 내 편으로 만든 사람은, 그렇지 않은 사람의 10년을 2년 만에 따라잡습니다.

이것이 복리입니다. 그리고 이것이 지금 이 순간 AI를 배워야 하는 수학적 이유입니다.

채용 패러독스: 주니어가 사라지면 시니어도 사라진다

여기에 아이러니가 하나 있습니다. 기업들이 주니어 개발자 채용을 줄이고 있는 현상, 단기적으로는 합리적으로 보입니다. AI가 주니어급 업무를 처리할 수 있으니, 비용을 절감할 수 있습니다. 하지만 이 결정에는 치명적인 맹점이 있습니다.

오늘의 주니어가 내일의 시니어입니다.

주니어를 뽑지 않으면 5년 후 시니어가 없습니다. 아무리 AI가 발전해도, 시스템을 설계하고 비즈니스의 사결정을 내리고 기술적 방향을 제시하는 시니어 아키텍트는 하루아침에 만들어지지 않습니다. 그것은 수년간의 실전 경험, 실패와 성공, 멘토링과 코드 리뷰를 통해서만 길러지는 역량입니다.

아마존 웹 서비스(AWS)의 CEO 매트 가먼(Matt Garman)은 이 현상에 대해 다음과 같이 경고했습니다.

“주니어 개발자를 AI로 대체하는 것은 가장 어리석은 결정 중 하나가 될 것입니다.”

그의 논리는 단순합니다. AI가 코드를 작성할 수 있지만, AI는 비즈니스 맥락을 이해하지 못합니다. AI는 고객의 고통을 모릅니다. AI는 레거시 시스템의 역사를 기억하지 못합니다. 이런 것들은 오직 사람만이, 그리고 오랜 경험을 통해서만 체득할 수 있는 것입니다.

하지만 시장은 이 경고를 무시하고 있습니다. 기업들은 여전히 주니어 채용을 줄이고 있고, 그 결과는 이미 교육 시장에 나타나고 있습니다.

포레스터(Forrester)는 컴퓨터공학 전공 입학률이 향후 수 년 내 20% 감소할 것으로 전망했습니다 (Forrester Predictions 2026). “개발자 취업이 어렵다”는 인식이 퍼지면서, 젊은 세대가 아예 개발자의 길을 선택하지 않기 시작한 것입니다.

이것은 악순환입니다.

주니어 채용 감소 -> “개발자 전망이 어둡다”는 인식 확산 -> CS 전공 지원 감소 -> 미래 인재 파이프라인 고갈 -> 5년 후 시니어 부족 -> 기업의 기술 경쟁력 약화

가장 먼저 이 악순환의 피해를 받는 것은 지금 현재 주니어 또는 취준생인 분들입니다. 하지만 아이러니하게도, 이 악순환이 만들어내는 기회도 존재합니다.

모두가 도망칠 때 남아 있는 사람이 가장 큰 기회를 잡습니다.

생각해 보면, 이 악순환에는 역설적인 희망이 숨어 있습니다. 인재 파이프라인이 줄어들다는 것은, 그 파이프라인 안에 남아 있는 사람의 가치가 올라간다는 뜻이기도 합니다. AI 역량까지 갖춘 채 버터낸 개발자는, 5년 후 시장에서 가장 귀한 존재가 됩니다. 지금의 혼란은 영원하지 않지만, 지금 쌓는 역량은 영구적입니다.

충격적 전망: 2028년의 개발자 지형

지금까지의 데이터와 트렌드를 연장하면, 2028년의 개발자 시장은 다음과 같은 모습이 될 것입니다.

두 개의 계층

AI 증폭형 아키텍트 (AI-Amplified Architects)

이들은 AI를 자유자재로 활용하는 개발자입니다. 코드를 직접 타이핑하는 시간보다 AI에게 지시하고 결과를 검증하는 시간이 더 깁니다. 시스템 아키텍처를 설계하고, AI 에이전트 파이프라인을 구축하며, 비즈니스 로직을 코드로 번역하는 역할을 합니다. 이들의 생산성은 2020년 기준 개발자의 5~10배입니다.

당연히 이들은 귀할 수밖에 없겠죠. 기업들이 서로 모시려 합니다. 연봉은 계속 오릅니다.

코드 상품화 노동자 (Code-Commodity Workers)

이들은 AI가 생성한 코드를 약간 수정하거나, AI가 아직 완벽하게 처리하지 못하는 레거시 시스템의 단순 유지보수를 담당합니다. 이들의 업무는 점점 더 정형화되고, AI가 발전할수록 이들이 할 수 있는 일의 범위는 줄어듭니다.

이들의 공급은 넘치고, 수요는 줄어듭니다. 연봉은 정체되거나 하락합니다.

다만 오해하지 마십시오. 이것은 현재 시장이 만들어내는 구조적 위치이지, 누군가의 정체성이 아닙니다. 지금 이 글을 읽고 있는 당신은 이미 방향을 바꾸기 시작한 사람입니다.

연봉 격차의 현실

현재도 AI 역량 유무에 따른 15~20% 프리미엄이 존재합니다. 이 격차가 3년 후에는 어떻게 될까요?

저자 개인의 추정임을 밝히고 말씀드리겠습니다. 같은 “개발자”라는 직함 안에서 연봉이 2~3배 차이나는 시대가 오고 있습니다. 이미 15~20% 프리미엄이 존재하는 현재에서, 그 격차가 가속될 방향은 명확합니다.

이것이 공포를 조장하는 것이라고 생각하실 수 있습니다. 하지만 이미 다른 분야에서 이런 현상을 보았습니다. 같은 “의사”인데 전공 분야에 따라 연봉이 5배 이상 차이나는 현실. 같은 “변호사”인데 영역에 따라 수입이 10배 차이나는 현실. “개발자”라는 직군도 예외가 될 이유가 없습니다.

하지만, 이것은 공포가 아니라 기회입니다

여기까지 읽으셨다면, 불안하실 수 있습니다. 그것이 정상입니다. 그리고 솔직히 말씀드리면, 그 불안감이 이 장을 쓴 목적이기도 합니다.

하지만 저는 여기서 가장 중요한 사실을 말씀드려야 합니다.

격차는 아직 건널 수 있습니다. 지금은 아직 초기입니다.

AI 도구가 개발 현장에 본격적으로 들어온 것은 2023년부터입니다. 불과 2~3년밖에 되지 않았습니다. 앞서 말한 복리 효과가 “건널 수 없는” 수준으로 벌어지려면 아직 시간이 있습니다. 지금 시작하면, 충분히 AI 증폭형 아키텍트 쪽에 설 수 있습니다.

왜 그렇게 확신하느냐고요?

첫째, 무료 또는 저렴한 AI 도구가 쏟아지고 있습니다. 2023년만 해도 비싸고 접근이 어려웠지만, 2026년 2월 현재 강력한 AI 코딩 도구를 월 몇 만 원이면 쓸 수 있습니다. 환경의 장벽이 빠르게 낮아지고 있습니다.

둘째, AI 네이티브 개발은 기존 프로그래밍 지식 위에 쌓이는 것입니다. 완전히 새로운 것을 처음부터 배우는 것이 아닙니다. 방향은 명확합니다. **Syntax에서 Intent로**. 문법을 외우는 사람에서, 의도를 설계하는 사람으로. 이미 개발자이신 분들은 그 기초 위에 새로운 층을 올리는 것이기 때문에, 학습 곡선이 생각보다 완만합니다.

셋째, 그리고 이것이 가장 중요한데, 대부분의 개발자가 아직 시작하지 않았습니다. 2026년 2월 현재, AI를 체계적으로 활용하는 개발자는 전체의 20~30%에 불과합니다. 나머지 70~80%는 아직 관망하거나, 산발적으로만 사용하고 있습니다. 지금 진지하게 시작하면 상위 30%에 빠르게 진입할 수 있습니다.

이 책을 읽고 있는 당신은 이미 인식의 첫 발을 뗐습니다.

양극화의 현실을 인식하는 것. 그것 자체가 이미 대다수보다 앞선 것입니다. 대부분의 사람들은 변화를 인식하기까지가 가장 오래 걸립니다. 인식한 후의 행동은 비교적 빠릅니다.

하지만 창문은 빠르게 닫히고 있습니다.

앞서 계산한 복리를 기억하십시오. 지금 시작하면 24개월 후 3.2배 성장할 수 있습니다. 6개월 후에 시작하면? 18개월밖에 남지 않아 2.4배에 그칩니다. 1년 후에 시작하면? 12개월로 1.8배. 숫자가 점점 작아집니다. 그리고 그 사이에 먼저 시작한 사람들은 계속 앞으로 나아갑니다.

지금 이 가장 빠른 때입니다. 어제보다는 늦었지만, 내일보다는 빠릅니다.

다음 장에서는 이 양극화의 핵심 원인인 ‘도구 갭’ - 같은 AI를 쓴다고 말하면서도 하늘과 땅 차이가 나는 현실 - 을 파헤치겠습니다. 공포는 이 장에서 끝입니다. 다음부터는 행동입니다.

이 장을 마치며

셀프 체크

- ☐ 자신이 “세계 A”인지 “세계 B”인지 판단했다
- ☐ AI 곱셈기 개념을 이해하고, 자신의 “기초 역량 값”이 0인지 10인지 생각해봤다
- ☐ 복리 효과를 자신의 상황에 대입해서 계산해봤다
- ☐ “모두가 도망칠 때 남아 있는 사람”이 되겠다는 결심이 있다

이 장의 미션

1. 자신이 “세계 B”에 있다면, 세계 A로 이동하기 위해 이번 주에 할 수 있는 행동 1가지를 정하고 실행하십시오. 예: AI 코딩 도구 하나를 설치하고 30분간 써보기.
2. 주변에 AI 코딩 도구를 체계적으로 쓰는 동료 있다면, 30분만 시간을 내달라고 요청하십시오. 어떻게 쓰는지 직접 보는 것이 백 마디 설명보다 낫습니다.

AI와 함께 활용하기

- **나의 격차 진단:** “나는 [연차]년차 [직무] 개발자입니다. 현재 AI 코딩 도구는 [사용 현황]입니다. 내 상황에서 가장 빠르게 AI 역량을 키우려면 어떤 순서로 무엇을 해야 하는지 3단계 로드맵을 짜줘.”
- **복리 시뮬레이션:** “내가 지금부터 매일 30분씩 AI 코딩 도구를 연습하면, 3개월/6개월/12개월 후에 어떤 수준에 도달할 수 있는지 현실적인 시나리오를 그려줘. 현재 레벨: [설명]” # 3장. 당신이 아직도 ChatGPT만 쓰는 동안 - 도구 갭이 만드는 생산성 절벽

“AI를 쓰면 실력이 안 느는 거 아니가요?”

멘토링을 하다 보면, 이 질문을 정말 많이 받습니다.

“솔직히 말씀드리면요, 요즘 코딩할 때 제가 직접 치는 코드가 별로 없어요. AI가 다 짜주는데… 이래도 되는 건가요?”

눈을 내리깔고, 마치 고백하듯이 말합니다. 죄책감이 묻어나는 목소리입니다. 본인이 뭔가 잘못하고 있다는 확신이 이미 얼굴에 써 있습니다. 이런 질문을 수백 번은 들었을 겁니다. 그리고 매번 같은 생각을 합니다.

그 죄책감이야말로, 지금 당신의 성장을 가장 크게 방해하고 있는 것이라고.

잠깐 다른 분야의 이야기를 해보겠습니다. 디지털 카메라가 처음 나왔을 때, 필름 사진작가들 사이에서 이런 말이 돌았습니다. “디지털로 찍으면 진짜 사진이 아니다.” 포토샵이 등장했을 때는 “보정하면 사기다”라는 말도 있었습니다. 지금 와서 보면 어떨습니까? 디지털 카메라를 거부한 사진작가는 시장에서 사라졌고, 포토샵을 쓰지 않는 프로 사진작가는 사실상 존재하지 않습니다.

코딩도 마찬가지입니다. AI를 쓰는 것이 곧 새로운 기본기입니다.

AI를 쓰지 않는 것은 순수함이 아니라, 거부입니다.

구글의 웹 성능 엔지니어링 리더(Google Web Performance Engineering Lead) Addy Osmani는 이런 통찰을 남겼습니다. “AI는 사실 좋은 엔지니어링 관행을 더 보상해 준다.” 무슨 뜻일까요? AI에게 명확한 요구사항을 전달하려면 본인이 문제를 정확히 이해해야 합니다. AI가 생성한 코드를 제대로 검증하려면 기초 지식이 있어야 합니다. AI 도구를 효과적으로 쓰려면 시스템 설계 감각이 있어야 합니다.

2장에서 이야기한 곱셈기를 떠올려 보십시오. 기초가 없으면 AI를 곱해도 0입니다. 기초가 있으면 10배, 100배가 됩니다.

문제는 “AI를 쓰느냐 안 쓰느냐”가 아닙니다. “어떤 수준의 AI 도구를 쓰느냐”입니다. 그리고 바로 여기에서, 대부분의 신입 개발자가 모르는 거대한 격차가 벌어지고 있습니다.

도구 스펙트럼: 당신은 지금 어디에 서 있습니까?

AI 코딩 도구에는 명확한 단계가 있습니다. 대부분의 사람들이 “나도 AI 쓰고 있어요”라고 말하지만, 같은 “AI 사용”이라는 단어 안에 하늘과 땅 차이가 숨어 있습니다.

Level 0 – AI 미사용

“순수하게 내 힘으로 해야 실력이 늡니다.”

구글 검색과 Stack Overflow만으로 코딩합니다. AI 도구를 의도적으로 거부합니다. 본인의 순수한 실력을 지키고 싶다는 신념이 있습니다. 놀랍게도, 2025년에도 이 입장을 고수하는 개발자가 생각보다 많거든요. 학교에서 “AI 쓰면 안 된다”는 교육을 받았거나, 커뮤니티에서 “AI 의존하면 실력이 안 늡다”는 분위기에 영향을 받은 경우가 대부분입니다.

안타깝지만, 이것은 성실함이 아니라 시대착오입니다. 마치 GPS가 있는데 종이 지도만 고집하는 택시 기사와 같습니다. 목적지에 도착은 하겠지만, 옆 차선의 택시가 두 배 빠르게 달리고 있다는 사실은 변하지 않습니다.

Level 1 – ChatGPT 웹 인터페이스로 질문

“검색 대신 ChatGPT에 물어봐요.”

에러 메시지를 복사해서 ChatGPT에 붙여 넣고, 답변을 읽고, 수동으로 코드에 반영합니다. Stack Overflow의 대체재로 사용하는 수준입니다. “AI를 쓰고 있다”고 말하는 사람의 대다수가 바로 이 단계에 있습니다.

솔직하게 말씀드리겠습니다. 현재 대부분의 신입 개발자가 여기에 머물러 있습니다. 멘토링에서 만나는 분들 대부분이 여기에 계시거든요. 처음 이 현실을 인식했을 때 꽤 당황했습니다.

Level 2 – 코드 자동완성

“GitHub Copilot이 알아서 다음 줄을 제안해줘요.”

에디터 안에서 AI가 코드를 제안하고, Tab 키를 눌러 수락합니다. 코드 작성 속도가 체감될 정도로 빨라 집니다. GitHub의 자체 조사에 따르면 Copilot 사용자가 코딩 작업을 최대 55% 더 빠르게 완료한다고 합니다. 상당한 향상입니다. 하지만 여전히 한 줄, 한 블록 단위의 도움입니다. 작업의 전체 흐름을 바꾸

지는 못합니다. 비유하자면, 자동완성은 빠른 타자기이고, 에이전트는 비서입니다. 타자를 빨리 치는 것과 비서에게 일을 맡기는 것은 차원이 다른 이야기입니다.

Level 3 – AI 페어 프로그래머

“Cursor나 Claude Code로 여러 파일을 동시에 작업해요.”

여기서부터 질적인 변화가 시작됩니다. AI가 단순히 자동완성을 넘어, 프로젝트의 맥락을 이해하고 여러 파일에 걸친 변경을 수행합니다. “이 API의 응답 형식을 바꾸고, 프론트엔드에서 호출하는 부분도 같이 수정해줘”라고 하면, 관련 파일을 찾아서 일관되게 수정합니다. 개발자는 더 이상 코드를 한 줄씩 작성하는 것이 아니라, AI와 대화하며 작업 단위를 통째로 위임하기 시작합니다. AI가 단순 도구에서 작업 파트너로 전환되는 순간입니다. 이 단계를 처음 경험한 멘티가 이런 말을 한 적이 있습니다. “이거 치트키 아니가요?” 치트키가 아니라, 새로운 기본기입니다.

Level 4 – 에이전틱 코딩 (AI 팀원)

“AI가 계획을 세우고, 구현하고, 테스트하고, 오류를 고칩니다.”

이 단계에서 AI는 지시를 받으면 스스로 계획을 수립하고, 코드를 작성하고, 테스트를 실행하고, 실패하면 원인을 분석해서 다시 고칩니다. 개발자는 최종 결과물을 리뷰하고 방향을 교정하는 역할입니다. 지금 이 순간, 업계의 선두 주자들이 바로 이 단계에서 일하고 있습니다.

Level 5 – 멀티 에이전트 오케스트레이션 (AI 팀 리더)

“여러 AI 에이전트가 동시에 다른 작업을 수행합니다.”

하나의 에이전트가 백엔드 API를 구현하는 동안, 다른 에이전트가 프론트엔드 UI를 만들고, 또 다른 에이전트가 테스트 코드를 작성합니다. 개발자는 오케스트라의 지휘자처럼 전체 흐름을 조율합니다. 에이전트들 사이의 의존성을 관리하고, 결과물이 하나의 일관된 시스템으로 통합되는지 감독합니다. 현재 시니어 개발자들이 빠르게 이 단계로 이동하고 있으며, 이 단계에 도달한 개발자는 혼자서 예전의 소규모 팀만큼의 산출물을 낼 수 있습니다. 아직은 시니어 영역이지만, 이런 방향으로 발전하고 있다는 것만 알아두십시오.

이 스펙트럼을 보고 어떤 생각이 드십니까?

냉정한 현실을 말씀드리겠습니다. 대부분의 신입 개발자는 Level 1에 머물러 있습니다. 그러면서 Level 3-4에 있는 동료에게 3배에서 5배의 생산성 격차로 밀리고 있다는 사실을 모릅니다. 아니, 더 정확하게 말하면, 그런 격차가 존재한다는 것조차 인식하지 못하고 있습니다.

이게 도구 갭입니다. 그리고 이 갭은 시간이 지날수록 좁아지는 것이 아니라 기하급수적으로 벌어집니다. Level 1에서 Level 2로 올라가는 것은 쉽습니다. Copilot을 설치하면 됩니다. 하지만 Level 2에서 Level 3으로의 전환은 도구만 바꾸는 것이 아니라 일하는 방식 자체를 재설계해야 합니다. AI에게 맥락을 제공하는 법, 프롬프트를 구조화하는 법, 결과물을 검증하는 프로세스를 만드는 법을 새로 배워야 합니다. 그리고 대부분은 그 전환이 필요하다는 사실조차 모른 채, Level 1에서 “나도 AI 쓰고 있어요”라고 말하고 있습니다.

한 가지 더 불편한 진실을 말씀드리겠습니다. 도구 갭은 단순히 속도의 문제가 아닙니다. 결국 기회의 문제가 됩니다. Level 4에서 일하는 개발자는 같은 시간에 더 많은 프로젝트를 경험합니다. 더 많은 시스템을 설계하고, 더 많은 문제를 풀고, 더 빠르게 성장합니다. 속도가 빠른 것이 끝이 아니라, 속도가 빠르기 때문에 더 많은 경험을 축적하게 되는 것입니다. 복리 효과가 발생합니다. 1년 후에 Level 1 개발자와 Level 4 개발자의 이력서에 적힌 프로젝트 수, 해결한 문제의 깊이, 다뤄본 기술의 범위는 비교할 수 없을 정도로 차이가 납니다.

AI 네이티브 개발자의 하루

도구 스펙트럼의 차이가 실제 업무에서 어떻게 나타나는지, 보여드리겠습니다.

Level 1 개발자의 아침은 이렇습니다. 지라 티켓을 확인하고, 관련 코드를 읽고, 어디를 수정해야 할지 파악합니다. 코드를 작성하다가 막히면 ChatGPT에 질문합니다. 답변을 읽고, 필요한 부분을 수동으로 코드에 반영합니다. 테스트를 직접 작성하고, 돌려보고, 실패하면 다시 코드를 고칩니다. PR을 올리고, 리뷰를 기다립니다.

Level 4 개발자의 아침은 다릅니다. 어젯밤 에이전트가 자동으로 올린 PR을 리뷰하는 것부터 시작합니다. 새 기능이 필요하면, 마크다운으로 스펙을 작성하고 에이전트에게 전달합니다. 에이전트가 구현하는 동안 다른 티켓의 아키텍처를 설계합니다. 에이전트가 구현을 완료하면 코드를 리뷰하고, 설계 의도와 맞는지 확인하고, 엡지 케이스를 검증합니다. 디버깅이 필요하면 에러와 함께 전체 컨텍스트를 에이전트에게 전달하고, 수정 제안을 검증한 뒤 적용합니다.

차이가 보이십니까?

같은 8시간을 일하고, 같은 연차이고, 같은 기술 스택을 사용합니다. 하지만 하루가 끝났을 때 아웃풋은 전혀 다릅니다. Level 1 개발자가 하나의 티켓을 처리하는 동안, Level 4 개발자는 세 개의 티켓을 동시에 진행합니다. 그리고 Level 4 개발자가 코드를 직접 타이핑하는 시간은 전체 업무 시간의 10%도 되지 않습니다. 나머지 90%는 설계, 검증, 의사결정인 셈이죠.

코드를 “직접 짜는” 시간이 10%라는 것은, 코드가 이미 이 사람의 업무에서 부산물이 되었다는 뜻입니다. 여기서 “AI가 대체할 수 없는 영역”이라는 익숙한 프레이밍을 쓰고 싶은 유혹이 있지만, 1장에서 말했듯 그 이분법 자체가 낡았습니다. 대체의 문제가 아닙니다. 레벨의 문제입니다. 이 개발자는 코드라는 추상화 레이어 위로 올라간 겁니다. 문제를 정의하고, 시스템의 행동을 설계하고, 결과가 비즈니스 목표와 일치하는지 검증하는 레벨. 코드를 보느냐 안 보느냐의 차이가 아니라, 어떤 높이에서 일하느냐의 차이입니다.

Vibe Coding이 보여준 것, 그리고 빠뜨린 것

2025년 초, 테슬라 AI 디렉터 출신이자 OpenAI 창립 멤버인 Andrej Karpathy가 'Vibe Coding'이라는 용어를 만들었습니다. “코드를 정확히 이해하지 않더라도 AI에게 대화하듯 지시하면서 소프트웨어를 만드는 방식.” 이 단어는 순식간에 개발자 커뮤니티를 뜨겁게 달궜습니다.

시니어 개발자들의 반응은 예상 가능했습니다. “코드를 이해하지 못하면서 어떻게 개발자라고 할 수 있느냐.” 이 비판은 틀렸습니다. 1장에서 말했듯이, 코드를 보느냐 안 보느냐는 이미 잘못된 질문입니다. 어셈블리 프로그래머가 C 프로그래머를 보며 “기계를 모르면서 어떻게 프로그래머냐”고 했던 것과 같은 반응입니다.

Vibe Coding의 방향은 맞습니다. 코드에서 눈을 떼는 것. 자연어로 의도를 전달하는 것. 이것은 1장에서 이야기한 추상화 레이어의 상승과 정확히 같은 궤적입니다.

그런데 Vibe Coding이 빠뜨린 것이 있습니다. **의도(Intent)**입니다.

Y Combinator의 파트너 Tim Brady는 이런 사실을 공유했습니다. 2025년 배치에 참여한 스타트업 중 25%가 코드베이스의 95% 이상이 AI로 생성되었다고. 코드를 AI가 짠다는 것은 이미 현실입니다. 하지만 이 스타트업들 중 성공하는 곳과 실패하는 곳의 차이는, 코드를 얼마나 이해하느냐가 아니었습니다. **만들려는 것이 무엇인지, 왜 만드는지, 어떤 문제를 해결하는지를 얼마나 명확하게 정의했느냐**였습니다.

Vibe Coding의 진짜 문제는 코드를 안 보는 것이 아닙니다. 의도 없이 코드를 안 보는 것입니다.

“이것저것 넣어봐” “느낌대로 해봐” “뭔가 그럴듯하게 만들어줘” – 이것은 코드에서 해방된 것이 아니라, 방향 없이 표류하는 것입니다. 재즈에 비유하면, 악보를 태운 것이 아니라 악보를 배운 적이 없는 것입니다. 둘은 완전히 다릅니다.

반면 이렇게 하는 것은 다릅니다. “사용자가 결제 후 3초 안에 확인 이메일을 받아야 해. 이벤트 기반으로 비동기 처리하고, 실패 시 3회 재시도 후 DLQ로 빠져야 해.” 이 사람도 코드를 안 봅니다. 하지만 의도가 정확합니다. 시스템이 어떻게 행동해야 하는지를 정의했고, 엣지 케이스까지 설계했습니다. 코드는 이 의도의 부산물로 자동 생성됩니다.

둘 다 코드를 안 봅니다. 하지만 한쪽은 의도 없는 자유이고, 다른 쪽은 의도 있는 자유입니다. Vibe Coding이 위험한 것은 전자를 후자와 구분하지 않기 때문입니다.

실제로 저는 후자의 방식으로 일하고 있습니다. 한 가지 사례를 들겠습니다. 여러 마이크로서비스가 물려있는 결제 플로우가 있었습니다. 사용자 인증부터 결제 요청, 외부 PG사 웹훅 수신, 비동기 이벤트 처리, 최종 확인 이메일 발송까지. 이 전체 흐름을 자동으로 검증하는 시스템이 필요했습니다. 지시는 이랬습니

다. “결제 플로우를 시나리오 기반으로 자동 검증할 수 있는 시스템 만들어줘. 시나리오는 YAML로 정의하고, 실행 결과는 대시보드에서 볼 수 있게. 외부 PG사는 모킹해서 격리된 환경에서 돌아가게 해.” 이 한 문단의 의도에서 249개 파일, 24,700줄의 시스템이 나왔습니다. 시나리오 실행 엔진, 웹 대시보드, 스케줄링 시스템, PG사 모킹 서버, 컨테이너 기반 격리 환경까지. 코드를 한 줄도 직접 치지 않았습니다. 하지만 의도는 처음부터 분명했습니다. 어떤 흐름을, 어떤 방식으로, 왜 검증하는지.

또 하나의 예를 들면, 저는 '사이클 돌려'라는 세 글자로 프로젝트 전체 검수를 실행합니다. 8개 체크리스트가 병렬로 돌아가고, 에이전트 그룹이 동시에 코드를 수정하고, 최종 보고서가 자동으로 생성됩니다. 이것은 Vibe Coding이 아닙니다. 의도가 시스템에 내장된 것입니다. 세 글자면 충분한 이유는, 그 세 글자 뒤에 '무엇을 검수하고, 어떤 기준으로 판단하고, 어떤 순서로 수정할지'가 이미 설계되어 있기 때문입니다.

에이전틱 혁명: 신입 개발자가 완전히 놓치고 있는 것

2024년 말부터 2025년에 걸쳐, AI 코딩 도구에 근본적인 전환이 일어났습니다. 챗봇에서 에이전트로의 전환입니다.

챗봇은 질문에 답합니다. “이 코드가 왜 에러가 나요?”라고 물으면, 원인을 설명해줍니다. 그러면 개발자가 직접 코드를 수정합니다.

에이전트는 다릅니다. 에이전트는 계획을 세우고, 실행하고, 반복합니다. “이 API에 OAuth 인증을 추가 해줘”라고 지시하면, 에이전트는 이렇게 움직입니다.

먼저 기존 코드 구조를 분석합니다. 어떤 파일을 수정해야 하는지 계획을 세웁니다. 필요한 패키지를 설치합니다. 코드를 작성합니다. 여러 파일에 걸쳐 일관되게 수정합니다. 테스트를 실행합니다. 테스트가 실패하면 원인을 분석하고 코드를 수정합니다. 테스트가 통과할 때까지 이 과정을 반복합니다.

개발자는 이 전체 과정이 끝난 뒤에, 최종 결과물을 리뷰합니다.

이것이 에이전틱 코딩입니다. Claude Code가 대표적입니다. 터미널에서 자연어로 지시하면, 프로젝트의 파일 구조를 이해하고, 여러 파일을 동시에 수정하고, 셸 명령어를 실행하고, 빌드와 테스트까지 수행합니다. Cursor의 Agent Mode도 마찬가지입니다. 편집기 안에서 멀티 파일 리팩토링을 수 초 만에 처리합니다.

Anthropic이 발표한 내부 사용 통계에 따르면, 자사 코드의 상당 부분이 AI 지원으로 작성되고 있다고 합니다. 이건 AI 회사의 개발자라서 그런 것이 아닙니다. 에이전틱 도구의 성능이 실제로 그 수준에 도달했기 때문입니다.

그리고 이 변화를, 대부분의 신입 개발자가 완전히 놓치고 있습니다.

여전히 ChatGPT 웹 창에 코드를 복사 붙여넣기 하면서 “AI 쓰고 있어요”라고 말하고 있습니다. 에이전트가 5분 만에 끝낼 작업을 두 시간에 걸쳐 수동으로 처리하고 있습니다. 세탁기가 있는데 손빨래를 하는 것과 같습니다. 빨래는 됩니다. 다만 세탁기를 돌린 사람은 그 시간에 다른 일을 하고 있을 뿐입니다.

에이전트를 안 쓴다고 해서 개발을 못 하는 것은 아닙니다. 하지만 에이전트를 쓰는 동료가 당신보다 3배, 5배 빠르게 같은 품질의 결과물을 내놓고 있다면, 당신의 상대적 위치는 매일 뒤로 밀리고 있는 것입니다.

더 무서운 것은, 에이전트 도구가 매달 더 좋아지고 있다는 사실입니다. 2024년 초의 에이전트와 2025년 말의 에이전트는 능력 차이가 극적입니다. 컨텍스트 윈도우가 커지고, 도구 사용 능력이 향상되고, 자율적으로 판단하고 실행하는 범위가 넓어지고 있습니다. 에이전트를 일찍 쓰기 시작한 사람은 도구의 진화

와 함께 자신의 워크플로우도 진화시킵니다. 늦게 시작하면, 도구의 능력을 따라가는 것만으로도 벅차게 됩니다.

그리고 이제, 한 단계 더 올라간 변화가 시작되었습니다.

에이전트에서 에이전트 팀으로

2026년 2월, Anthropic이 Claude Code에 'Agent Teams'라는 기능을 출시했습니다. 단일 에이전트가 아닌, 여러 에이전트가 팀으로 협업하는 기능입니다.

작동 방식은 이렇습니다. 리드 에이전트가 작업을 분해하고, 팀원 에이전트들에게 할당합니다. 각 팀원은 독립된 컨텍스트에서 병렬로 작업하면서, 공유 태스크 리스트를 통해 진행 상황을 조율합니다. 팀원끼리 직접 메시지를 주고받기도 합니다. 리드를 거치지 않고요. 마치 실제 개발팀이 작동하는 방식과 같습니다.

Anthropic은 이 기능의 스트레스 테스트로, 16개의 에이전트가 동시에 협업하여 10만 줄 규모의 C 컴파일러를 작성하게 했습니다. 약 2,000개의 세션을 거쳐 완성된 이 컴파일러는 x86, ARM, RISC-V 아키텍처에서 Linux 6.9 커널을 컴파일할 수 있습니다. 에이전트 한 명이 아니라, 에이전트 팀이 프로젝트를 수행하는 시대가 된 것입니다.

이것이 의미하는 바를 생각해 보십시오. 과거에는 '나 + AI 에이전트 1명'이었습니다. 지금은 '나 + AI 에이전트 팀'입니다. 한 에이전트가 백엔드를 구현하는 동안 다른 에이전트가 프론트엔드를 만들고, 또 다른 에이전트가 테스트를 작성합니다. 개발자 한 명이 소규모 팀의 산출물을 내는 것이 더 이상 비유가 아니라 현실입니다.

당신이 아직 ChatGPT 웹 창에 코드를 복사 붙여넣기 하고 있는 동안, 옆자리 동료는 에이전트 팀을 지휘하며 하루에 세 개의 피처를 동시에 진행하고 있습니다. 이 격차는 선형이 아닙니다. 지수적입니다.

“그러면 나는 뭘 하는 거죠?” – 새로운 역할 정의

1장에서 “코드를 보지 마십시오”라고 했습니다. 에이전트가 코드를 짜고, 에이전트 팀이 프로젝트를 수행합니다. 그렇다면 당신은 뭘 하는 겁니까?

이 질문에 대한 답이, 이 장의 핵심이자 앞으로 이 책 전체를 관통하는 메시지입니다.

당신은 팀 리드입니다. 에이전트 팀의 리드.

이전 섹션에서 보았듯이, 에이전트는 이제 혼자가 아니라 팀으로 움직입니다. 리드 에이전트가 작업을 분배하고, 팀원 에이전트들이 병렬로 실행하고, 공유 태스크 리스트로 조율합니다. 하지만 이 팀에는 한 가지가 빠져 있습니다. 방향입니다. 이 팀이 무엇을 만들어야 하는지, 왜 만들어야 하는지, 어떤 문제를 해결해야 하는지. 그것을 정의하는 것이 당신의 역할입니다.

구체적으로 말하면 이렇습니다.

첫째, 의도를 정의합니다. 비즈니스가 원하는 것을 에이전트가 실행할 수 있는 명세로 번역합니다. “사용자가 결제 후 3초 안에 확인 이메일을 받아야 해. 이벤트 기반 비동기 처리, 실패 시 3회 재시도 후 DLQ.” 이 수준의 의도 정의가 없으면 에이전트 팀은 방향 없이 움직입니다.

둘째, 시스템의 행동을 설계합니다. 컴포넌트 간의 관계, 데이터의 흐름, 확장성과 성능의 트레이드오프. 코드 수준이 아니라 시스템 수준에서 행동을 정의하고, 에이전트에게 실행을 맡깁니다.

셋째, 결과를 검증합니다. 에이전트가 만든 결과물이 의도대로 행동하는지 확인합니다. 1장에서 말했듯이 코드를 한 줄씩 읽는 것이 아닙니다. 시스템이 의도대로 동작하는지, 엣지 케이스에서 무너지지 않는지, 비즈니스 목표와 일치하는지를 검증합니다.

넷째, 의사결정을 내립니다. “이 기능에 캐시를 적용할 것인가?”, “마이크로서비스로 분리할 것인가?”, “기술 부채를 지금 갚을 것인가?” 이런 판단은 비즈니스 맥락과 팀 상황을 종합적으로 고려해야 합니다. 에이전트는 선택지를 제시할 수 있지만, 결정은 당신이 합니다.

Addy Osmani는 자신의 글 ‘The 70% Problem’에서 이것을 “70% 문제”라고 표현했습니다. AI가 전체 작업의 70%를 처리해 줍니다. 하지만 나머지 30%가 진짜 어려운 부분입니다. 아키텍처 결정, 엣지 케이스 처리, 시스템 통합, 성능 최적화, 보안 검증. 에이전트 팀이 아무리 강력해도, 이 30%의 방향을 잡아주는 사람이 없으면 70%의 실행도 의미가 없습니다.

역설적이지만, AI 시대에 기초가 더 중요해진 이유가 여기에 있습니다. AI가 70%를 처리해주니까 기초가 필요 없는 것이 아닙니다. AI가 70%를 처리해주기 때문에, 나머지 30%에 집중할 수 있는 기초 체력이 필수인 것입니다.

당신의 역할은 바뀌었습니다. 코드를 짜는 사람에서, 에이전트 팀을 지휘하는 사람으로.

From Syntax to Intent. 문법에서 의도로. 1장에서 이야기한 이 전환이 바로 여기서 현실이 됩니다. 에이전트 한 명을 부리는 것에서 에이전트 팀을 오케스트레이션하는 것으로. 이 전환을 빨리 받아들이는 사람이 AI 네이티브 시대의 승자가 됩니다.

시대착오적 죄책감을 버려라

마지막으로, 이 장의 시작점이었던 질문으로 돌아가겠습니다.

“AI를 쓰면 실력이 안 느는 거 아니가요?”

이 질문의 본질은 죄책감입니다. 내가 직접 하지 않으면 진짜가 아니라는 생각. 도구의 힘을 빌리면 부정 행위라는 감각. 이 감각이 어디에서 오는지, 역사를 통해 살펴보겠습니다.

1980년대, 스프레드시트 소프트웨어가 등장했을 때 일입니다. 회계사들 사이에서 논란이 있었습니다. “컴퓨터가 계산해주면, 우리의 전문성은 어디에 있는 거냐?” 주판과 계산기로 수치를 검증하던 시절의 회계사들은, 엑셀이 모든 것을 자동으로 계산해주는 것에 불안감을 느꼈습니다. 하지만 지금 와서 생각해 보면, 엑셀이 나왔을 때 주판을 안 쓰는 것에 죄책감을 느낀 회계사가 있었을까요? 그런 죄책감을 느꼈다면, 그것은 올바른 감정이었을까요?

더 가까운 예를 들어보겠습니다. 컴파일러가 처음 등장했을 때, 어셈블리 프로그래머들은 비슷한 반응을 보였습니다. “고급 언어로 프로그래밍하면 진짜 프로그래머가 아니다.” 기계어 수준에서 하드웨어를 이해하지 못하면 실력이 없다는 것이었습니다. 컴파일러가 나왔을 때 어셈블리를 안 쓰는 게 죄책감이었을까요? 지금 그 질문이 얼마나 우스운지, 아마 누구나 알 것입니다.

패턴이 보이십니까?

기술의 매 전환점마다, 새로운 도구를 쓰는 것에 대한 죄책감이 있었습니다. 그리고 예외 없이, 그 죄책감을 빨리 버린 사람이 다음 시대를 이끌었습니다. 주판에 대한 죄책감을 버린 회계사가 재무 분석가가 되었고, 어셈블리에 대한 죄책감을 버린 프로그래머가 소프트웨어 아키텍트가 되었습니다.

지금 이 순간, AI 도구에 대한 죄책감은 그 역사의 반복입니다.

“AI가 코드를 짜주면 내가 뭘 배우는 거지?”라는 질문은 잘못된 질문입니다. 올바른 질문은 이것입니다.

“AI와 함께 일하는 방식을 배우면, 내가 얼마나 더 큰 문제를 풀 수 있을까?”

지금 AI를 쓰지 않는 것이 성실함이라고 착각하는 것은, 미래에서 돌아보면 가장 게으른 선택이 될 것입니다. 성실함의 정의가 바뀌었기 때문입니다. 과거의 성실함은 “내 손으로 직접 코드를 많이 짜는 것”이었습니다. 지금의 성실함은 “가장 효과적인 도구를 사용해서 가장 좋은 결과를 내는 것”이거든요.

AI를 쓰는 것은 부정행위가 아닙니다. AI를 쓰지 않는 것은 공부가 아닙니다. 그것은 진화를 거부하는 것입니다.

한 가지 더 말씀드리겠습니다. 이 죄책감의 가장 교활한 점은, 그것이 “성실함”이라는 가면을 쓰고 있다는 것입니다. “나는 정직하게 코딩하고 있어”라는 자기 서사는 기분이 좋습니다. 도덕적 우월감까지 줍니다. 하지만 그것은 위안일 뿐, 전략이 아닙니다. 취업 시장에서, 프로젝트 현장에서, 성과 평가에서 “저는 AI 안 쓰고 정직하게 했습니다”가 면책 사유가 되지는 않습니다. 결과물의 양과 질로 평가받는 세상에서, 도구 선택은 도덕의 문제가 아니라 전략의 문제입니다.

물론, AI를 무비판적으로 수용하라는 말이 아닙니다. AI가 생성한 코드를 이해 없이 복사 붙여넣기만 하는 것은 위험합니다. 하지만 그 위험을 피하는 방법은 AI를 거부하는 것이 아니라, AI를 더 잘 쓰는 것입니다. AI의 출력을 검증하는 능력, AI에게 더 정확한 지시를 내리는 능력, AI의 한계를 파악하는 능력. 이것들이 새 시대의 기초 체력입니다.

이 장을 읽고 나서, 한 가지만 기억해주시기 바랍니다.

도구 갭은 실력 갭이 됩니다. Level 1에 머물면서 Level 4의 동료와 같은 평가를 받을 수는 없습니다. 죄책감에 사로잡혀서 도구의 진화를 거부하는 것은, 결국 본인의 성장을 스스로 제한하는 것입니다.

시대착오적 죄책감을 버리십시오. 그리고 도구 스펙트럼에서 한 단계, 아니 두 단계 위로 올라가십시오. 그것이 2025년 신입 개발자가 해야 할, 가장 시급하고 가장 확실한 투자입니다.

다음 장에서는, 그렇다면 구체적으로 어떤 도구를 어떻게 세팅하고, 어떻게 일하는 방식을 재설계해야 하는지 이야기하겠습니다.

이 장을 마치며

셀프 체크

- ☐ 자신의 현재 도구 Level(0~5)을 판단했다
- ☐ “AI를 쓰면 실력이 안 는다”는 생각에서 벗어났다
- ☐ Vibe Coding과 에이전틱 코딩의 차이를 설명할 수 있다
- ☐ “벽돌공이 아니라 건축가”라는 역할 전환을 이해했다
- ☐ 도구 갭이 단순한 속도 문제가 아니라 기회의 문제임을 인식했다

이 장의 미션

1. 현재 자신의 도구 Level을 판단하고, 한 단계 위로 올라가기 위해 이번 주에 시도할 도구 1개를 정하십시오. Level 0~1이라면 Cursor 또는 GitHub Copilot 설치부터. Level 2라면 Claude Code를 터미널에서 써보기부터.
2. AI에게 죄책감을 느끼고 있었다면, 이 문장을 소리 내어 읽으십시오. “AI를 쓰지 않는 것은 성실함이 아니라 거부다.” 그리고 오늘 하나의 업무를 AI와 함께 처리해보십시오.

AI와 함께 활용하기

- **도구 Level 진단:** “나는 현재 AI 코딩 도구를 [사용 방식]으로 쓰고 있어. 도구 스펙트럼 Level 0~5 중 내가 어디에 있는지 판단하고, 다음 Level로 올라가기 위한 구체적 행동 3가지를 알려줘.”
- **에이전틱 코딩 체험:** “이 프로젝트에서 [기능]을 구현해야 해. 파일 구조를 분석하고, 구현 계획을 세우고, 코드를 작성하고, 테스트까지 실행해줘.” (Claude Code 또는 Cursor Agent Mode에서)
- **Vibe Coding 실험:** “다음 아이디어를 30분 안에 작동하는 프로토타입으로 만들어줘: [아이디어 설명]. 완벽하지 않아도 되고, 핵심 기능만 동작하면 됨.” # 4장. 환경 세팅 - AI와 일하는 개발 환경 구축하기

3장의 마지막에서 약속드렸습니다. 구체적으로 어떤 도구를 어떻게 세팅하고, 어떻게 일하는 방식을 재 설계해야 하는지 이야기하겠다고.

여기서부터가 2부입니다. 1부에서는 “세상이 바뀌었다”는 인식을 드렸습니다. 2부에서는 “그래서 이렇게 하면 된다”를 보여드리겠습니다. 2장에서 이야기한 세계 B에서 세계 A로 이동하는 여정이 여기서 시작됩니다.

그런데 한 가지 미리 말씀드려야 할 것이 있습니다. 이 장은 “여기 클릭하고, 저기 설정하세요” 같은 튜토리얼이 아닙니다. 도구 설치법은 각 도구의 공식 문서가 훨씬 잘 설명합니다. 이 장에서 다루려는 것은, 왜 이 도구인지, 그리고 도구를 설치한 다음에 해야 할 진짜 중요한 것은 무엇인지입니다.

4.1 결론부터 말하겠습니다

개인적인 의견을 먼저 말씀드리겠습니다.

2026년 2월 현재, AI 코딩 에이전트의 1위는 **Claude Code**입니다. 압도적으로.

이것은 객관적 벤치마크와 개인적 경험 모두에서 나온 결론입니다. 먼저 데이터를 보겠습니다.

SWE-bench Verified는 실제 오픈소스 프로젝트의 GitHub 이슈를 AI가 자동으로 해결할 수 있는지를 측정하는 벤치마크입니다. 인위적인 코딩 테스트가 아니라, 현실의 버그와 기능 요청을 다룹니다. 2026년 2월 기준 Claude Opus 4.5의 점수는 80.9%입니다. 이것은 역대 최고 성적입니다. OpenAI의 최신 모델인 GPT-5 기반 Codex가 74.5%이고, 그 외 모델들은 이보다 낮습니다.

숫자만으로는 감이 잘 안 올 수 있으니 구체적으로 말씀드리겠습니다. SWE-bench에서 6.4%p의 차이는 단순한 오차 범위가 아닙니다. 80% 이상의 영역에서는 남은 문제들이 점점 더 어려워지기 때문에, 이 차이는 체감상 훨씬 큼니다. 마치 100m 달리기에서 9.8초와 10.1초의 차이처럼, 숫자로는 작아 보이지만 실제로는 세대의 차이입니다.

하지만 벤치마크보다 중요한 것은 실제로 써본 경험입니다.

저는 Claude Code Max 플랜(월 200달러)을 쓰고 있습니다. 이 플랜은 Pro 대비 20배의 사용량을 제공합니다. 충분하냐고요? 이 사용량으로 **1주일에 상용 서비스 3개를 동시에 개발하고 운영**하고 있습니다. 과장이 아닙니다. 이 책의 저자 프로필에 적힌 서비스들이 실제로 이 방식으로 만들어지고 있는 것들입니다.

월 200달러. 연봉으로 환산하면 약 320만 원입니다. 주니어 개발자 한 명의 월급에도 미치지 못하는 금액으로, 소규모 개발팀의 산출물을 내고 있습니다. 3장에서 이야기한 에이전트 팀 오케스트레이션이 이론이 아니라 현실인 이유가 여기에 있습니다.

4.2 왜 Claude Code인가

“그래서 왜 하필 Claude Code냐”는 질문에 답하겠습니다.

터미널 네이티브

1장에서 “IDE를 켜지 마십시오”라고 했습니다. Claude Code는 그 주장을 현실로 만드는 도구입니다. 에디터가 아닙니다. 터미널입니다. 자연어로 지시하면, 파일을 읽고, 수정하고, 빌드하고, 테스트하고, 커밋까지 합니다. 개발자는 코드를 타이핑하지 않습니다. 의도를 전달합니다.

Cursor나 Windsurf 같은 AI 에디터들도 강력합니다. 하지만 이것들은 본질적으로 IDE입니다. 에디터 안에서 AI를 부르는 방식입니다. “코드를 보면서” 작업하는 패러다임의 연장선에 있습니다. Claude Code는 반대입니다. 코드를 보지 않는 것이 기본값입니다. 코드는 에이전트가 다루는 것이고, 개발자는 의도와 검증에 집중합니다. 1장에서 이야기한 추상화 레이어의 상승이 도구 레벨에서 실현되는 것입니다.

에이전트 팀

3장에서 소개한 Agent Teams. 리드 에이전트가 작업을 분해하고, 팀원 에이전트들이 병렬로 실행하고, 공유 태스크 리스트로 조율하는 기능. 2026년 2월 현재, 이 수준의 멀티 에이전트 오케스트레이션을 프로덕션 레벨로 제공하는 도구는 Claude Code뿐입니다.

한 에이전트가 백엔드 API를 구현하는 동안, 다른 에이전트가 프론트엔드 컴포넌트를 만들고, 또 다른 에이전트가 E2E 테스트를 작성합니다. 이것이 “1주일에 3개 서비스”가 가능한 이유입니다. 혼자서 하는 것이 아닙니다. 에이전트 팀을 지휘하는 것입니다.

컨텍스트 이해

Claude Code는 프로젝트 전체를 읽습니다. 파일 구조, 의존성, 빌드 설정, 테스트 구조를 파악한 상태에서 작업합니다. “이 프로젝트에서 결제 관련 코드를 모두 찾아서 분석해줘”라고 하면, grep을 돌리는 것이 아니라 코드의 의미를 이해하면서 찾습니다. 30개 이상의 서비스가 물려 있는 모노레포에서도, 서비스 간 의존성을 파악하고 일관된 수정을 합니다. 이 수준의 컨텍스트 이해 능력은 현재 다른 도구에서 경험하기 어렵습니다.

자율적 문제 해결

에이전트에게 작업을 맡기면, 중간에 빌드가 실패하거나 테스트가 깨지는 일이 빈번합니다. 이때 에이전트의 진짜 역량이 드러납니다. Claude Code는 에러를 분석하고, 원인을 추론하고, 수정하고, 다시 테스트합니다. 이 사이클을 스스로 반복합니다. 개발자가 개입하지 않아도 됩니다. 단, 결과가 의도와 맞는지 최종 검증은 반드시 해야 합니다.

4.3 다른 도구들의 위치

Claude Code가 압도적이라고 해서, 다른 도구들이 쓸모없다는 것은 아닙니다. 각 도구에는 자기 자리가 있습니다. 다만 같은 레벨에서 비교하는 것은 맞지 않습니다.

OpenAI Codex

솔직하게 말하면, Codex는 유일하게 Claude Code와 같은 문장에서 언급할 만한 도구입니다.

OpenAI가 2025년에 출시한 클라우드 기반 에이전트로, 샌드박스 환경에서 코드를 작성하고 테스트합니다. GPT-5 기반 모델의 코딩 능력은 분명 강력합니다. 가격도 월 200달러(Pro 플랜)로 Claude Code Max와 동일한 가격대입니다.

하지만 결정적인 차이가 있습니다. Codex는 클라우드에서 실행됩니다. 코드가 샌드박스 환경에 올라가서 처리됩니다. Claude Code는 로컬에서 실행됩니다. 내 터미널에서, 내 프로젝트 디렉토리에서, 내 빌드 환경 그대로 작동합니다. 프로덕션 환경과 동일한 조건에서 테스트하고 빌드할 수 있다는 것은, 실무에서 체감되는 차이가 큼니다. 또한 SWE-bench 성적에서도 6.4%p의 격차가 있습니다.

Codex가 나쁜 도구라는 것이 아닙니다. 강력한 도구입니다. 다만 현재 시점에서 “어떤 도구 하나만 고르라면”이라는 질문에, 저의 답은 명확합니다.

GitHub Copilot

Copilot은 자동완성 도구입니다. 무료 플랜이 있고, Pro는 월 10달러, Pro+는 월 39달러입니다. 2025년부터 Agent Mode가 추가되었고, 프리미엄 요청을 통해 에이전틱 기능을 쓸 수 있습니다.

가장 큰 장점은 접근성입니다. VS Code에 확장 프로그램 하나 설치하면 됩니다. 회사에서 VS Code만 쓸 수 있는 환경이라면, Copilot은 현실적인 선택입니다. 3장의 도구 스펙트럼에서 Level 2에 해당합니다.

하지만 명확히 해두겠습니다. Copilot의 자동완성과 Claude Code의 에이전틱 코딩은 차원이 다릅니다. Tab 키를 눌러서 다음 줄을 수락하는 것과, 자연어로 시스템 전체를 설계하고 구현을 위임하는 것을 같은 “AI 코딩 도구”라는 범주로 묶는 것 자체가 오해를 만듭니다.

Cursor

AI 네이티브 에디터 중에서는 가장 완성도가 높습니다. Pro 월 20달러, Ultra 월 200달러. Agent Mode를 통해 멀티 파일 수정이 가능하고, 에디터 안에서 작업하는 것을 선호하는 개발자에게 익숙한 경험을 제공합니다.

다만 앞서 말했듯이, 에디터 기반이라는 것 자체가 “코드를 보면서 작업하는” 패러다임의 연장입니다. 그리고 과금 방식이 크레딧 기반이라, 사용량이 많아지면 비용 예측이 어렵습니다. Ultra 플랜(월 200달러)의 실제 사용 가능량은 Claude Code Max와 직접 비교하기 어렵습니다.

Gemini CLI

Google이 오픈소스로 공개한 터미널 기반 AI 도구입니다. Apache 2.0 라이선스이며, 무료(분당 60회, 일 1,000회 요청 가능)입니다. 무료라는 점은 분명 매력적입니다. Claude Code나 Codex에 월 200달러를 투자하기 어려운 상황이라면, 에이전틱 코딩을 경험해보는 입문용으로 의미가 있습니다.

하지만 에이전트로서의 역량, 특히 복잡한 멀티 파일 작업이나 대규모 리팩토링에서의 정확도는 현재 Claude Code와 비교할 수준이 아닙니다.

4.4 200달러의 경제학

“월 200달러는 비싸지 않습니까?”

이 질문을 많이 받습니다. 그래서 계산을 해보겠습니다.

주니어 개발자의 평균 연봉을 4,000만 원으로 잡겠습니다. 월급은 약 333만 원입니다. 여기에 4대 보험, 퇴직금, 복리후생을 포함한 실제 인건비는 월 450만 원 이상입니다.

Claude Code Max 플랜은 월 200달러, 약 27만 원입니다. 주니어 개발자 인건비의 6%입니다.

이 6%의 비용으로 생산성이 3배에서 5배 올라갑니다. 투자 대비 수익률(ROI)로 따지면, 이보다 확실한 투자는 거의 없습니다.

“그래도 월 27만 원이 부담됩니다.”

이해합니다. 취업 준비 중이거나 신입이라면 부담이 될 수 있습니다. 선택지가 있습니다.

Claude Code Pro(월 20달러, 약 2.7만 원)로 시작할 수 있습니다. Max 대비 사용량은 1/20이지만, 에이전트 코딩의 핵심 기능은 동일합니다. 학습과 사이드 프로젝트에는 충분합니다. Pro에서 사용량이 부족하다고 느끼는 순간이 오면, 그때가 Max로 올려야 할 시점입니다. 그리고 그 시점이 오면, 월 27만 원이 아깝다는 생각은 사라질 겁니다. “이게 27만 원이라고?”가 됩니다.

무료로 시작하고 싶다면 Gemini CLI가 있습니다. 에이전트 코딩이 무엇인지 체험하는 데는 충분합니다.

한 가지 더. 월 200달러가 비싸다고 느끼는 이유 중 하나는, “도구에 돈을 쓴다”는 것에 익숙하지 않기 때문입니다. 개발자들은 IDE에 돈을 쓰는 것에는 익숙합니다. IntelliJ IDEA Ultimate가 연 149달러입니다. 하지만 AI 에이전트에 돈을 쓰는 것은 아직 낯선 개념입니다.

관점을 바꿔보십시오. 이것은 도구 비용이 아니라 팀 비용입니다. 월 27만 원으로 에이전트 팀을 고용하는 것입니다. 그 팀이 코드를 작성하고, 테스트하고, 디버깅하고, 리팩토링합니다. 당신은 팀 리더로서 의도를 정의하고 결과를 검증합니다. 이 관점에서 보면, 월 27만 원은 비싼 것이 아니라 말도 안 되게 싼 것입니다.

4.5 AI에게 맥락을 주는 법

도구를 정했으면, 이제 진짜 세팅을 해야 합니다.

도구를 설치하는 것과 도구를 **세팅**하는 것은 다릅니다. 대부분의 사람들이 도구를 설치하자마자 “이 코드 수정해줘”라고 지시합니다. 그리고 결과가 엉망입니다. “이 도구 별로네” 하면서 다시 ChatGPT 웹창으로 돌아갑니다.

왜 그럴까요? AI에게 **맥락**을 주지 않았기 때문입니다.

신입 개발자가 팀에 합류했다고 상상해 보십시오. 첫날, 아무런 온보딩 없이 “이 버그 좀 고쳐주세요”라고 하면 어떻게 될까요? 프로젝트 구조도 모르고, 코딩 컨벤션도 모르고, 기술 스택이 뭔지도 모르는데 코드를 수정하라고 합니다. 당연히 엉뚱한 방향으로 고칩니다.

AI도 마찬가지입니다. 그래서 필요한 것이 **프로젝트 컨텍스트 파일**입니다.

Claude Code에서는 `CLAUDE.md` 라는 파일을 프로젝트 루트에 만듭니다. Cursor에서는 `.cursorrules`, Codex에서는 `AGENTS.md` 입니다. 이름은 도구마다 다르지만, 하는 일은 같습니다. AI에게 “이 프로젝트는 이런 프로젝트야”라고 알려주는 것입니다.

구체적으로 뭘 적어야 하느냐. 실전 예시를 보여드리겠습니다.

```
# 프로젝트 개요
- Spring Boot 3.x + Kotlin 기반 백엔드 서비스
- PostgreSQL, Redis 사용
- 포트: 8080

# 코딩 컨벤션
- 함수명: camelCase
- 패키지 구조: 도메인별 분리 (user/, order/, payment/)
- 테스트: JUnit 5 + MockK 사용

# 빌드 & 실행
- 빌드: ./gradlew build
- 테스트: ./gradlew test
- 로컬 실행: docker-compose up -d && ./gradlew bootRun

# 주의사항
- 기존 API 응답 형식(ApiResponse<T>)을 반드시 따를 것
- 날짜는 모두 UTC, ISO-8601 형식
- 커밋 메시지는 Conventional Commits 형식
```

이 10줄짜리 파일 하나 만들어두면, AI가 생성하는 코드의 품질이 극적으로 달라집니다. 온보딩을 제대로 받은 팀원처럼, 프로젝트의 규칙을 이해한 상태에서 작업하기 때문입니다.

컨텍스트 파일에 적어야 할 핵심 요소 다섯 가지입니다.

1. **기술 스택과 버전** – AI가 맞는 API를 사용하도록
2. **프로젝트 구조** – AI가 파일을 어디에 만들지 알도록
3. **코딩 컨벤션** – AI가 팀 스타일에 맞는 코드를 짜도록
4. **빌드/테스트 명령어** – AI가 직접 빌드하고 테스트할 수 있도록
5. **주의사항/금지사항** – AI가 실수하기 쉬운 부분을 미리 막도록

그리고 여기에 하나를 더 추가하겠습니다. Claude Code에는 다른 도구에 없는 강력한 패턴이 있습니다. 컨텍스트 파일의 **계층 구조**입니다.

프로젝트 루트에 `CLAUDE.md` 를 만들고, 각 서비스 디렉토리에도 `CLAUDE.md` 를 만들 수 있습니다. 루트 파일에는 전체 프로젝트의 공통 규칙을, 서비스별 파일에는 해당 서비스의 고유한 규칙을 적습니다. 모노레포나 멀티 서비스 환경에서 이 계층 구조는 체감되는 차이를 만듭니다.

처음부터 완벽할 필요는 없습니다. 작업하면서 AI가 자주 실수하는 부분을 발견할 때마다, 컨텍스트 파일에 한 줄씩 추가하면 됩니다. 시간이 지날수록 이 파일은 프로젝트의 “살아있는 온보딩 문서”가 됩니다. 저의 경우, 가장 오래된 프로젝트의 `CLAUDE.md` 는 50줄이 넘습니다. 그 50줄 안에 수백 번의 실수에서 얻은 교훈이 담겨 있습니다.

4.6 첫 번째 세팅: 30분 안에 시작하기

구체적인 순서를 알려드리겠습니다. Claude Code 기준입니다.

0~5분: 설치

터미널에서 `npm install -g @anthropic-ai/claude-code` 를 실행합니다. 끝입니다. Anthropic 계정으로 로그인하고, 구독 플랜을 선택합니다. Pro(월 20달러)로 시작해도 충분합니다.

5~15분: CLAUDE.md 작성

재미있는 방법을 하나 알려드리겠습니다. 컨텍스트 파일 자체를 Claude Code에게 작성하게 할 수 있습니다. 프로젝트 디렉토리로 이동한 뒤 첫 번째 지시를 이렇게 내려보십시오.

“이 프로젝트의 구조를 분석하고, 기술 스택, 빌드 방법, 코딩 컨벤션을 정리해서 CLAUDE.md 파일을 작성해줘.”

Claude Code가 프로젝트 파일을 읽고 초안을 만들어줍니다. 그 초안을 검토하고 수정하면, 첫 컨텍스트 파일이 10분 안에 완성됩니다.

15~30분: 첫 번째 작업 위임

컨텍스트 파일이 준비되었으면, 간단한 작업 하나를 맡겨보십시오.

- “이 프로젝트의 README를 업데이트해줘”
- “이 함수에 대한 단위 테스트를 작성해줘”
- “이 API 엔드포인트에 입력값 검증을 추가해줘”

핵심은 AI가 당신의 프로젝트 맥락을 이해한 상태에서 작업하는 경험을 처음으로 해보는 것입니다.

ChatGPT 웹 창에 코드를 복사 붙여넣기 하던 것과, 에이전트가 프로젝트 전체를 보면서 작업하는 것의 차이를 직접 체감해보십시오.

이 30분이 3장의 도구 스펙트럼에서 Level 1과 Level 3의 경계를 넘는 순간입니다.

4.7 첫 1주일: 에이전트와 일하는 감각 익히기

30분 세팅으로는 시작일 뿐입니다. 도구를 “설치”한 것과 도구를 “체화”한 것은 다릅니다.

첫 1주일이 결정적입니다. 매일 30분, 7일간의 루틴을 제안합니다.

1일차: 탐색 – 에이전트에게 기존 코드를 분석하게 합니다. “이 프로젝트의 전체 구조를 설명해줘”, “이 서비스의 API 엔드포인트 목록을 정리해줘.” 코드를 직접 작성하지 않고, 에이전트의 이해도를 확인하는데 집중합니다.

2일차: 작은 수정 – 간단한 버그 수정이나 코드 개선을 맡깁니다. 결과물을 검토하면서, 에이전트가 컨텍스트 파일의 규칙을 따르는지 확인합니다. 안 따르는 부분이 있으면, 컨텍스트 파일에 규칙을 추가합니다.

3일차: 테스트 작성 – 테스트 코드가 없는 함수를 하나 골라, 에이전트에게 테스트를 작성하게 합니다. 에이전트가 생성한 테스트가 실제로 의미 있는 케이스를 다루는지 판단해보십시오. 이 과정에서 “검증 능력”이 생기기 시작합니다.

4일차: 기능 구현 – 작은 기능 하나를 에이전트에게 맡깁니다. 한 번에 완벽한 결과를 기대하지 마십시오. 결과를 리뷰하고, 의도와 다른 부분을 지적하고, 수정을 요청하는 대화의 과정이 핵심입니다.

5일차: 디버깅 – 실제 버그를 에이전트에게 전달하고 해결을 맡깁니다. “이 에러의 원인을 찾고 수정해줘.” 에이전트가 에러 로그를 분석하고, 원인을 추론하고, 코드를 수정하고, 테스트하는 전체 과정을 관찰합니다.

6일차: 멀티 파일 리팩토링 – 여러 파일에 걸친 변경을 맡깁니다. “이 공통 유틸 함수의 시그니처를 변경하고, 호출하는 모든 곳을 같이 수정해줘.” 에이전트가 프로젝트 전체를 파악하면서 일관된 수정을 하는 것을 확인합니다.

7일차: 회고 – 이번 주 에이전트와 함께 작업한 내용을 돌아봅니다. 어떤 지시가 잘 통했고, 어떤 지시는 결과가 나빴는지 기록합니다. 컨텍스트 파일에 추가할 내용이 있으면 업데이트합니다.

이 7일이 끝나면, “이건 에이전트에게 맡기면 되겠다”라는 판단이 자연스럽게 나오기 시작합니다. 그 감각이 생기면, 3장에서 이야기한 Level 전환의 절반은 이미 이루어진 것입니다.

이 장을 마치며

이 장에서 다룬 것은 두 가지입니다.

첫째, 도구의 선택. 결론은 명확합니다. Claude Code가 현재 최고의 에이전틱 코딩 도구이고, 200달러 Max 플랜의 투자 대비 가치는 압도적입니다. 다른 도구들도 각자의 자리가 있지만, 같은 레벨에서 비교하는 것은 맞지 않습니다.

둘째, 도구의 세팅. 설치는 5분이면 끝나지만, 세팅은 프로젝트를 이해하는 과정입니다. 프로젝트 컨텍스트 파일 하나가 AI와의 모든 작업의 품질을 결정합니다. 그리고 그 파일은 작업하면서 계속 진화합니다.

“그런데 이렇게 특정 도구를 강하게 추천해도 되는 겁니까?”

이런 의문이 들 수 있습니다. 이 분야는 빠르게 변합니다. 6개월 후에는 판도가 달라질 수 있습니다. 하지만 서문에서 말씀드렸듯이, 이 책의 유효기간은 1년입니다. 1년 후를 걱정하면서 “글쎄요, 뭐든 다 좋아요, 상황에 맞게 골라보세요” 같은 안전한 답을 드리는 것은 이 책의 역할이 아닙니다.

2026년 2월 현재, 이 순간 가장 확실한 투자는 Claude Code입니다. 이것이 저의 답입니다.

다음 장에서는, 환경이 갖춰진 상태에서 AI와 함께 코드를 짜기 전에 해야 할 가장 중요한 단계를 이야기합니다. 코드 한 줄 치기 전에 먼저 AI와 설계부터 하는 법. 1장에서 이야기한 “의도를 정확하게 전달하는 법”이 5장에서 본격적으로 시작됩니다.

셀프 체크

- ☐ Claude Code가 다른 AI 코딩 도구와 차별화되는 이유를 설명할 수 있다
- ☐ 프로젝트 컨텍스트 파일(CLAUDE.md)의 역할과 작성법을 이해했다
- ☐ 컨텍스트 파일에 들어가야 할 5가지 핵심 요소를 알고 있다
- ☐ 도구를 “설치”하는 것과 “세팅”하는 것의 차이를 설명할 수 있다
- ☐ 월 200달러 투자의 ROI를 계산하는 관점을 이해했다

이 장의 미션

1. Claude Code를 설치하고(Pro 플랜이면 충분합니다), 현재 작업 중인 프로젝트에 CLAUDE.md 파일을 만드십시오. 기술 스택과 빌드 명령어, 이 두 가지만이라도 적는 것부터 시작하십시오.

2. 4.7의 7일 루틴 중 1일차를 오늘 실행하십시오. 에이전트에게 자신의 프로젝트 구조를 분석하게 하는 것부터. 30분이면 됩니다.

AI와 함께 활용하기

- **컨텍스트 파일 작성:** “내 프로젝트의 구조를 분석하고, 기술 스택, 빌드 방법, 코딩 컨벤션, 주의사항을 정리해서 CLAUDE.md를 작성해줘.”
 - **도구 경험 비교:** “이 프로젝트에서 [기능]을 구현해야 해. 파일 구조를 분석하고, 구현 계획을 세우고, 코드를 작성하고, 테스트까지 실행해줘.” - ChatGPT 웹 인터페이스로 같은 작업을 했을 때와 비교해보십시오.
 - **에이전트 팀 체험:** “이 작업을 병렬로 진행해줘. 백엔드 API와 프론트엔드 컴포넌트를 동시에 만들어줘.” - Max 플랜의 Agent Teams 기능으로 멀티 에이전트 오케스트레이션을 직접 경험해보십시오.
- # 5장. AI와 함께 설계하기 - 코드 한 줄 치기 전에 해야 할 것

코드부터 치는 사람, 질문부터 하는 사람

멘토링을 하다 보면 패턴이 보입니다. 기능 요구사항을 받았을 때, 주니어 개발자 대부분은 IntelliJ를 열고 컨트롤러 클래스부터 만듭니다. 파일을 생성하고, 어노테이션을 붙이고, 메서드 시그니처를 적기 시작합니다. “일단 짜보면서 생각하자”는 접근입니다.

한번은 멘토링 세션에서 경력 2년차 백엔드 개발자에게 간단한 과제를 드린 적이 있습니다. “회원 탈퇴 API를 만들어 보세요.” 그분은 바로 IDE를 열었습니다. 30분 뒤, `DELETE /api/users/{id}` 엔드포인트가 만들어져 있었습니다. 기술적으로 틀린 건 없었습니다. 하지만 저는 이렇게 물었습니다.

“탈퇴한 회원의 결제 내역은요? 작성한 게시글은 어떻게 되나요? 탈퇴 후 동일 이메일로 재가입하면요? 탈퇴 사유는 수집하나요?”

그분의 표정이 굳었습니다. 코드는 완성했지만, 정작 이 기능이 왜 필요하고 어떤 상황에서 호출되는지를 한 번도 생각하지 않았던 겁니다.

이것이 “코드를 짜는 사람”과 “시스템을 설계하는 사람”의 차이입니다. 1장에서 건축가 비유를 들었습니다. 건축가는 벽돌을 쌓기 전에 도면을 그립니다. AI 시대의 개발자도 마찬가지입니다. 코드를 짜기 전에 먼저 설계를 해야 합니다. 그리고 이 설계 과정에서, AI는 당신이 만날 수 있는 가장 접근성 높은 첫 번째 동료가 됩니다.

5.1 “Why”를 먼저 정의하는 습관

1장에서 **From Syntax to Intent** - 문법에서 의도로 - 라는 전환을 이야기했습니다. 이 전환의 가장 실용적인 출발점이 바로 여기입니다. 코드를 짜기 전에, 이 기능이 왜 필요한지를 먼저 정의하는 것.

“왜”를 정의한다는 건 거창한 일이 아닙니다. 마크다운 문서 하나에 세 가지만 적으면 됩니다.

1. 이 기능이 해결하는 문제는 무엇인가? (유저의 어떤 불편을 해소하는가)
2. 이 기능이 없으면 어떻게 되는가? (현재 상태의 문제점)
3. 성공 기준은 무엇인가? (이 기능이 잘 동작하면 어떤 상태가 되는가)

예시를 들어보겠습니다. “쿠폰 적용 API를 만들어야 한다”는 요구사항을 받았다고 가정합니다.

나쁜 스펙: 무엇(What)만 있는 경우

쿠폰 적용 API

- POST /api/coupons/apply
- Request: couponCode, orderId
- Response: discountAmount
- 쿠폰 코드를 검증하고 할인 금액을 반환한다

이것은 스펙이 아니라 API 명세입니다. AI에게 이걸 던지면 “동작은 하지만 현실에서 못 쓰는” 코드가 나옵니다. 쿠폰 중복 사용은? 유효기간 만료는? 최소 주문 금액 조건은? 이미 결제가 완료된 주문에 적용하면? 이런 맥락이 전부 빠져 있으니깐요.

좋은 스펙: 왜(Why)와 제약(Constraints)이 포함된 경우

쿠폰 적용 기능

배경 (Why)

- 현재 프로모션 쿠폰을 발급하고 있지만, 적용 과정이 수동(CS팀 직접 처리)
- 월 평균 200건의 쿠폰 적용 요청이 CS팀에 들어오고 있음
- 유저가 결제 시점에 직접 쿠폰을 적용할 수 있어야 CS 부담 감소

제약 조건 (Constraints)

- 1주문 1쿠폰 (중복 적용 불가)
- 쿠폰에 유효기간, 최소 주문 금액, 최대 할인 금액 조건이 있음
- 이미 결제 완료된 주문에는 적용 불가
- 동시에 같은 쿠폰을 여러 유저가 사용하는 경우 처리 필요 (선착순 쿠폰)

성공 기준 (What)

- 유저가 결제 화면에서 쿠폰 코드를 입력하면 즉시 할인 금액이 반영됨
- 유효하지 않은 쿠폰에 대해 명확한 에러 메시지 제공
- CS팀 수동 처리 건수 80% 감소

차이가 보이십니까? 두 번째 스펙에는 **이 기능이 존재해야 하는 이유, 현실 세계의 제약 조건, 측정 가능한 성공 기준**이 들어 있습니다. 이 문서를 AI에게 주면, AI는 “쿠폰 중복 적용 방지를 위한 유니크 제약”, “동시성 제어를 위한 비관적 락 또는 Redis 분산 락”, “쿠폰 상태 검증 로직” 같은 것들을 알아서 고려하기 시작합니다.

1장에서 **질문의 격차가 실력의 격차**라고 했습니다. 좋은 스펙을 쓸 수 있는 사람이 AI에게 좋은 질문을 던질 수 있는 사람입니다. 스펙이 구체적일수록 AI의 출력은 현실에 가까워집니다. 스펙이 모호하면 AI의 출력도 모호합니다. 입력의 질이 출력의 질을 결정하는 원리는, AI 시대에 더 극적으로 작동합니다.

5.2 AI를 첫 번째 동료로 삼기: 설계 토론의 기술

스펙을 작성했으면, 다음 단계는 **구현 전에 AI와 설계 토론**을 하는 것입니다.

여기서 많은 분들이 실수하는 지점이 있습니다. “이 기능 어떻게 구현하면 좋을까?”라고 묻고 끝내는 겁니다. AI가 답을 주면 “오, 좋다” 하고 바로 코드를 짜기 시작합니다. 이것은 설계 토론이 아니라 답안지 베끼기입니다.

진짜 설계 토론은, AI에게 **트레이드오프를 분석**하게 하는 겁니다. 모든 기술적 결정에는 장단점이 있습니다. 시니어 개발자가 주니어와 다른 점은 그 장단점을 알고 상황에 맞는 선택을 할 수 있다는 것인데, 이 과정을 AI와 함께 훈련할 수 있습니다.

설계 토론 프롬프트 패턴 3가지

패턴 1: 트레이드오프 분석 요청

우리 서비스에서 쿠폰 적용 시 동시성 문제를 해결해야 해.
현재 환경: Spring Boot + PostgreSQL + Redis 사용 중.
TPS는 평균 50, 피크 시 200 정도.

다음 선택지를 비교 분석해줘:

1. DB 비관적 락 (SELECT FOR UPDATE)
2. Redis 분산 락 (Redisson)
3. DB 유니크 제약 + 재시도

각각의 장단점, 구현 난이도, 우리 환경에서의 적합도를 표로 정리해줘.

이 프롬프트의 핵심은 **선택지를 내가 먼저 제시한다**는 점입니다. AI에게 “어떻게 하면 돼?”라고 물으면 AI는 하나의 답을 줍니다. “이것들 중 뭐가 나을까?”라고 물으면 AI는 분석을 줍니다. 전자는 수동적 소비이고, 후자는 능동적 판단의 재료를 얻는 것입니다.

패턴 2: 반론 요청

나는 이 기능을 이렇게 설계하려고 해: [설계 내용].
이 설계의 약점이나 놓치고 있는 부분을 찾아줘.
특히 확장성, 장애 대응, 엣지 케이스 관점에서 검토해줘.

혼자 설계하면 자기 설계의 약점을 보기 어렵습니다. AI에게 “내 설계를 공격해줘”라고 요청하면, 상당히 유용한 피드백을 받을 수 있습니다. 시니어 동료에게 설계 리뷰를 받는 것과 비슷한 효과입니다.

패턴 3: 단계별 설계 확장

이 기능의 v1 (MVP)은 이렇게 설계했어: [v1 설계].

이 설계를 기반으로, 향후 다음 요구사항이 추가될 때를 고려해줘:

- 쿠폰 종류 확장 (정률 할인, 정액 할인, 배송비 무료)
- 쿠폰 조합 규칙 (특정 쿠폰끼리만 중복 허용)

v1을 어떻게 설계해야 v2, v3로 확장할 때 최소한의 수정으로 가능할까?

과도한 추상화 없이, 현실적인 수준에서 제안해줘.

이 패턴이 강력한 이유는, “지금은 간단하게 가되 미래를 고려하라”는 현실적 설계 감각을 AI와 함께 연습할 수 있기 때문입니다. 마지막 문장 “과도한 추상화 없이”가 중요합니다. 이걸 빼면 AI는 팩토리 패턴과 전략 패턴을 도입한 지나치게 복잡한 구조를 제안하는 경향이 있거든요.

설계 토론의 핵심은 **AI의 답을 그대로 채택하는 것이 아니라, AI의 분석을 재료로 내가 판단하는 것**입니다. AI가 “Redis 분산 락이 적합합니다”라고 답해도, 최종 결정은 당신이 합니다. 그 과정에서 당신은 트레이드오프를 고려하는 설계 감각을 쌓게 됩니다. 이것이 1장에서 말한 “건축가”의 구체적 실천입니다.

5.3 에이전트와 대화하는 법: 명령이 아니라 질문부터

설계가 끝나고 구현 단계에 들어갔습니다. 여기서 대부분의 사람이 하는 행동이 있습니다.

“쿠폰 적용 API 만들어줘.”

명령입니다. 그리고 이것이 문제의 시작입니다.

4장에서 CLAUDE.md 같은 프로젝트 컨텍스트 파일을 세팅했습니다. 에이전트는 당신의 프로젝트 구조를 읽을 수 있고, 기술 스택을 알고, 기존 코드의 패턴을 파악하고 있습니다. 그런데 왜 “Spring Boot 기반 이커머스 서비스에서 Controller - Service - Repository 3계층이고, JPA + PostgreSQL을 사용 중이야”라고 매번 다시 알려줘야 합니까? 에이전트가 이미 아는 것을 사람이 일일이 타이핑하는 것은 맥락 파일의 존재 이유를 부정하는 것입니다.

더 근본적인 문제가 있습니다. 명령은 개발자가 이미 답을 알고 있다는 전제에서 출발합니다.

“CouponApplyService를 만들고, POST /api/orders/{orderId}/coupons 엔드포인트를 만들어줘.” 이렇게 지시하려면, 어떤 클래스가 필요하고, 어떤 경로가 적절한지를 개발자가 먼저 결정해야 합니다. 결국 코드 레벨의 설계를 사람이 하고, 타이핑만 에이전트에게 시키는 겁니다. 이것은 1장에서 말한 추상화 레이어의 상승이 아니라, 고급 자동완성입니다.

좋은 상호작용은 명령이 아니라 질문에서 시작합니다.

질문부터 시작한다

에이전트에게 처음 꺼내야 할 말은 이런 겁니다.

“야, 쿠폰 적용 기능을 만들려고 하는데. 우리 프로젝트에서 쿠폰 관련 코드 지금 어떻게 되어 있어?”

이 한 문장이 명령과 다른 이유는, **에이전트에게 탐색의 기회를 주기 때문**입니다. 에이전트는 프로젝트를 읽고 이렇게 답할 겁니다. “Coupon 엔티티가 이미 있고, CouponRepository도 있습니다. 현재 쿠폰 발급 기능은 있지만 적용 기능은 없습니다. Order 엔티티에 couponId 필드는 있지만 아직 사용되지 않고 있네요.”

이 답변을 듣는 순간, 당신은 프로젝트의 현재 상태를 에이전트의 눈으로 파악하게 됩니다. 그리고 여기서 대화가 자연스럽게 이어집니다.

“그러면 기존 Coupon 엔티티에 유효기간이랑 최소주문금액 조건이 있어?”

“네, expiresAt, minOrderAmount, maxDiscountAmount 필드가 있습니다.”

“좋아. 그러면 쿠폰 적용 기능 만들어줘. 1주문 1쿠폰이고, 동시에 같은 쿠폰 쓰는 경우도 처리해야 해. 에러는 사유별로 분리해서 내려줘.”

이것이 전부입니다. 기술 스택을 다시 설명할 필요도, 클래스명을 지정할 필요도, 엔드포인트 경로를 정해줄 필요도 없습니다. 에이전트는 프로젝트의 기존 패턴을 이미 파악했기 때문에, 기존 컨벤션에 맞는 코드를 생성합니다. ErrorResponse 형식이 프로젝트에 이미 있다면 그 형식을 따릅니다. 패키지 구조가 도메인별로 분리되어 있다면 그 구조를 따릅니다.

왜 질문이 명령보다 강력한가

세 가지 이유가 있습니다.

첫째, **에이전트가 당신보다 프로젝트를 빠르게 읽습니다.** 30개 서비스가 물려 있는 모노레포에서 “쿠폰 관련 코드가 어디에 있는지”를 사람이 찾으면 10분 걸립니다. 에이전트는 10초 걸립니다. 질문을 통해 에이전트의 탐색 능력을 활용하는 것이, 사람이 미리 다 파악해서 명령하는 것보다 효율적입니다.

둘째, **잘못된 가정을 방지합니다.** “CouponApplyService를 만들어줘”라고 명령했는데, 사실 프로젝트에 이미 CouponService가 있고 거기에 apply 메서드를 추가하는 것이 기존 패턴에 맞는 경우가 있습니다. 질문을 먼저 하면, 에이전트가 기존 구조를 보고 적절한 위치를 판단합니다. 개발자가 미리 결정하면, 오히려 기존 패턴과 어긋나는 코드가 나옵니다.

셋째, **대화를 통해 의도가 정제됩니다.** 처음부터 완벽한 스펙을 작성하는 것은 어렵습니다. 하지만 에이전트와 질문을 주고받는 과정에서, “아, 이것도 고려해야 하네”, “이 부분은 기존 코드에 이미 있구나” 같은 발견이 일어납니다. 대화 자체가 설계를 정제하는 과정이 됩니다.

대화의 패턴

에이전트와의 대화에는 자연스러운 흐름이 있습니다.

1단계: 탐색 질문 - “우리 프로젝트에서 결제 관련 코드 어떻게 되어 있어?” - “기존에 비슷한 기능 구현한 적 있어? 참고할 만한 패턴 있으면 알려줘.” - “이 서비스에서 이벤트 처리를 어떻게 하고 있어? Kafka 쓰고 있어?”

2단계: 의도 전달 - “사용자가 결제 시점에 쿠폰을 적용할 수 있게 만들어야 해.” - “동시에 같은 쿠폰을 여러 명이 쓰는 상황도 처리해야 하고.” - “에러 메시지는 사유별로 구분해서 프론트가 바로 쓸 수 있게.”

3단계: 피드백 - “그 방식도 좋은데, 락 범위가 너무 넓지 않아? 더 좁힐 수 있을까?” - “테스트는 동시성 시나리오도 포함해줘.”

이 흐름에서 개발자가 하는 것은 클래스명을 짓는 것도, 엔드포인트를 설계하는 것도 아닙니다. **의도를 전달하고, 결과를 판단하는 것**입니다. 3장에서 이야기한 “에이전트 팀의 리드”가 바로 이런 모습입니다.

“프롬프트 엔지니어링”이라는 환상

한 가지 더 짚고 넘어가겠습니다. 인터넷에 “AI 프롬프트 엔지니어링” 가이드가 넘쳐납니다. “이런 형식으로 써라”, “역할을 부여하라”, “단계를 나눠라”. 틀린 말은 아니지만, 본질을 빗나가고 있습니다.

프롬프트를 정교하게 다듬는 것보다 중요한 것은, **에이전트가 이미 알고 있는 것을 활용하는 것**입니다. 4장에서 세팅한 CLAUDE.md, 프로젝트 구조, 기존 코드 패턴. 이것들이 에이전트의 맥락입니다. 좋은 프롬프트 템플릿을 외우는 것보다, 에이전트에게 “너 이거 알아?”라고 먼저 물어보는 것이 훨씬 강력합니다.

그리고 이 “질문하는 능력”의 본질은 사실 프롬프트 작성 능력이 아닙니다. **자신이 원하는 것이 무엇인지 아는 것, 그리고 상대가 이미 무엇을 알고 있는지 파악하는 것**입니다. 이 능력은 AI가 아니라 사람에게 설명할 때도, 기획서를 쓸 때도, 팀을 리드할 때도 동일하게 작동합니다.

5.4 실전 시나리오: 질문에서 구현까지

지금까지 이야기한 내용을 하나의 흐름으로 엮어보겠습니다. 간단한 기능 하나를 예시로, “설계 토론 -> 질문 -> 구현 위임”까지의 전체 과정을 시연합니다.

상황 설정

당신은 이커머스 서비스의 백엔드 개발자입니다. PM이 이렇게 요청합니다.

“유저가 주문 내역을 조회할 수 있는 API를 만들어주세요. 마이페이지에서 쓸 거예요.”

1단계: Why를 정의한다

코드를 열기 전에, 마크다운 파일을 열고 이렇게 씁니다.

주문 내역 조회 기능

배경

- 현재 유저가 자신의 주문 내역을 확인하려면 CS에 문의해야 함
- 마이페이지에 주문 내역 탭을 추가하여 셀프서비스 전환
- 대상: 로그인한 유저만 접근 가능

의문점 (아직 모르는 것)

- 주문 내역을 얼마나 과거까지 보여줄 것인가?
- 페이지네이션이 필요한가? (주문이 많은 유저가 있는가?)
- 주문 상태별 필터링이 필요한가?
- 주문 상세(개별 상품 정보)까지 한 번에 내려줄 것인가?

“의문점”이 핵심입니다. 이것이 PM에게 역으로 질문할 항목이기도 하고, 에이전트와 대화를 시작할 재료이기도 합니다.

2단계: 에이전트에게 질문한다

에디터를 열지 않습니다. 터미널에서 에이전트와 대화를 시작합니다.

“마이페이지에서 주문 내역 조회 기능을 만들려고 하는데. 우리 프로젝트에서 주문 관련 코드 어떻게 되어 있어? Order 엔티티 구조 보여줘.”

에이전트는 프로젝트를 탐색하고 답합니다. “Order 엔티티에는 이런 필드가 있고, OrderItem과 OneToMany 관계이고, 현재 주문 생성 API는 있지만 조회 API는 없습니다. 기존 API들은 이런 패턴으로 페이지네이션을 구현하고 있습니다.”

이 답변만으로 설계의 절반이 끝났습니다. 기술 스택? 에이전트가 이미 알고 있습니다. 페이지네이션 방식? 기존 프로젝트에서 이미 쓰고 있는 패턴을 에이전트가 알려줬습니다. 인증 방식? CLAUDE.md에 적혀 있습니다.

3단계: 의도를 전달하고 토론한다

“마이페이지용이니까 본인 주문만 보여줘야 하고, 목록이랑 상세는 분리하는 게 맞겠지? 그리고 주문 상태별 필터링도 있으면 좋겠는데, 기존 프로젝트에서 필터링 어떻게 하고 있어?”

에이전트가 기존 패턴을 기반으로 설계를 제안합니다. 여기서 5.2에서 이야기한 설계 토론이 자연스럽게 이어집니다. “이 규모에서 offset 기반과 cursor 기반 중 뭐가 맞을까?” 같은 후속 질문을 던질 수도 있습니다.

4단계: 구현을 맡긴다

토론이 끝나면, 한마디면 됩니다.

“좋아, 그 방향으로 만들어줘.”

에이전트는 토론 내용을 전부 기억하고 있습니다. 배경도, 제약도, 합의된 설계도 대화 안에 이미 들어 있습니다. 별도의 스펙 문서를 작성해서 넘길 필요가 없습니다. 대화 자체가 스펙입니다.

이것이 5.1에서 말한 “Why를 먼저 정의하는 습관”, 5.2에서 말한 “AI와 설계 토론”, 5.3에서 말한 “질문부터 시작하는 대화”가 하나로 연결되는 전체 워크플로우입니다. 코드는 한 줄도 치지 않았지만, 가장 중

요한 일은 이미 끝났습니다. 이후 에이전트가 생성한 코드를 검증하고 다듬는 과정은 다음 장에서 다루겠습니다.

이 장을 마치며

이 장의 핵심을 한 문장으로 요약하면 이것입니다. **코드를 짜기 전에 생각하고 질문하는 시간이, AI 시대에 가장 ROI가 높은 시간이다.**

많은 가이드가 “좋은 프롬프트를 쓰는 법”을 가르칩니다. 기술 스택을 명시하고, 제약 조건을 나열하고, 기대 결과를 정의하라고. 그것이 틀린 것은 아니지만, 에이전틱 도구 시대에는 한 세대 뒤쳐진 접근입니다. 에이전트는 당신의 프로젝트를 읽을 수 있습니다. 기술 스택을 알고 있고, 기존 코드의 패턴을 파악하고 있습니다. 매번 이걸 다시 설명하는 것은, 온보딩을 마친 팀원에게 매일 아침 “우리 회사는 이런 일을 하는 곳이야”라고 설명하는 것과 같습니다.

대신, 질문하십시오. “우리 프로젝트에서 이거 어떻게 되어 있어?” “기존에 비슷한 패턴 있어?” “이렇게 하려는데 기존 구조에서 가능해?” 에이전트의 탐색 능력을 먼저 활용하고, 그 위에서 의도를 전달하십시오. 그것이 에이전트와 일하는 사람의 방식입니다.

다음 장에서는 이렇게 설계한 것을 바탕으로 에이전트가 실제로 코드를 생성하고, 그 코드를 검증하고 디버깅하는 과정을 다루겠습니다.

셀프 체크

- ☐ 기능 요청을 받았을 때 코드부터 치지 않고, “왜 이 기능이 필요한가”를 먼저 정리할 수 있다
- ☐ 에이전트에게 명령 대신 질문부터 하는 습관의 의미를 설명할 수 있다
- ☐ AI에게 트레이드오프 분석을 요청하는 설계 토론을 할 수 있다
- ☐ 프로젝트 컨텍스트를 매번 수동으로 제공하지 않고, 에이전트의 탐색 능력을 활용할 수 있다

이 장의 미션

1. 자신이 다음에 구현할 기능 1개를 골라, **코드를 한 줄도 치지 말고** Why 문서부터 작성하십시오. 배경, 의문점, 성공 기준을 포함해야 합니다.
2. 에이전트에게 명령 대신 질문부터 던져보십시오. “우리 프로젝트에서 이 기능 관련 코드 어떻게 되어 있어?”부터 시작하십시오. 에이전트의 답변을 듣고, 그 위에서 의도를 전달하고, 대화를 통해 구현까지 이어가 보십시오.

AI와 함께 활용하기

- **탐색 질문으로 시작하기:** “내가 [기능]을 만들려고 하는데, 우리 프로젝트에서 [관련 도메인] 코드 어떻게 되어 있어? 기존에 비슷한 패턴 있으면 알려줘.”
- **설계 토론하기:** “이 기능 이렇게 만들까 생각 중인데, 기존 프로젝트 구조에서 맞는 방향이야? 다른 방법이 더 나을 수도 있으면 알려줘.”
- **내 설계 검증받기:** “이렇게 설계하려고 하는데, 약점이나 놓치고 있는 엣지 케이스 있어? 솔직하게 말해줘.” # 6장. 코드 작성과 디버깅 - AI가 짠 코드를 ‘부리는’ 법

AI가 코드를 짜는 시대, 당신의 진짜 일은 무엇인가

5장에서 우리는 코드를 한 줄 치기 전에 해야 할 것들을 다뤘습니다. Why를 먼저 정의하고, 에이전트와 설계를 토론하고, 질문을 통해 대화를 이끌어가는 법을 배웠습니다. 에이전트에게 질문하고, 프로젝트 현황을 파악하고, 의도를 전달하고, “좋아, 그 방향으로 만들어줘”까지 갔습니다. 이제 에이전트가 코드를 생성합니다.

그런데 여기서 많은 분들이 착각하는 지점이 하나 있습니다. “대화를 잘 이끌면 되는 거 아닌가? 에이전트가 알아서 잘 만들어주겠지?”

아닙니다. 그건 절반입니다.

3장에서 다룬 '70% 문제'를 기억하십시오. AI가 70%를 해줍니다. 하지만 나머지 30% - 검증, 교정, 판단 - 가 진짜 어려운 부분이고, 바로 그것이 당신의 일입니다. 에이전트가 아무리 강력해도, 결과물이 의도대로 동작하는지, 엣지 케이스에서 무너지지 않는지, 보안 구멍은 없는지를 판단하는 것은 여전히 사람의 몫입니다.

이 장에서는 에이전트에게 구현을 맡기는 실전 워크플로우부터, 결과물을 검증하는 체크리스트, 디버깅의 새로운 방식, 그리고 실전 시나리오까지 다룹니다. 코드를 ‘짜는’ 법이 아니라, 에이전트가 짠 코드를 ‘부리는’ 법을 이야기합니다.

6.1 에이전틱 코딩 실전: 구현을 맡기는 워크플로우

대화에서 코드까지의 흐름

5장에서 에이전트와 대화를 통해 설계까지 마쳤습니다. “좋아, 그 방향으로 만들어줘.” 에이전트가 코드를 작성하기 시작합니다. 여기서 중요한 판단이 하나 있습니다.

위임의 범위를 어떻게 잡느냐입니다.

통째로 맡기기 vs 중간중간 확인하기

통째로 맡기기가 유효한 경우가 있습니다. 범위가 작고, 기존 코드와의 의존성이 낮을 때. 독립된 유틸리티 함수, 단순한 CRUD API, 명확한 입출력이 정의된 컴포넌트 같은 것들입니다. 이런 경우에는 에이전트에게 맡기고, 결과물을 한 번에 리뷰하는 것이 효율적입니다.

중간중간 확인하기가 필요한 경우는 더 많습니다. 여러 레이어에 걸치는 기능, 기존 코드와 복잡하게 연결되는 변경, 비즈니스 로직이 복잡한 경우. 이런 상황에서 통째로 맡기면, 에이전트가 중간에 방향을 잘못 잡았을 때 되돌리기가 어렵습니다.

실전에서 효과적인 패턴은, 에이전트와의 대화를 단계적으로 이어가는 것입니다.

“먼저 데이터 모델부터 잡아보자. 기존 엔티티 구조 보고 어떻게 확장해야 할지 제안해줘.” 에이전트가 제안하면 검토합니다. 방향이 맞으면 다음 단계로 넘어갑니다. “좋아. 이제 핵심 비즈니스 로직 구현해줘. 옛지 케이스는 일단 TODO로 남기고 메인 흐름부터.” 에이전트가 구현하면 다시 검토합니다. “이제 TODO로 남긴 부분들 채워줘. 동시성이라든 입력 검증 특히 신경 써서.” 마지막으로 테스트입니다. “테스트 작성해줘. 정상 케이스, 경계값, 실패 케이스 다 포함해서.”

이 방식의 핵심은 각 단계에서 에이전트의 방향을 검증하고 교정할 수 있다는 것입니다. 데이터 모델이 잘못되었으면 첫 단계에서 바로 잡을 수 있습니다. 모든 것을 만든 뒤에 전체를 뒤집는 것보다 훨씬 효율적입니다.

AI가 막힐 때 대처법

AI에게 구현을 맡기다 보면, 분명히 막히는 순간이 옵니다. 같은 에러를 반복하거나, 엉뚱한 방향으로 코드를 수정하거나, 점점 복잡해지지만 하고 해결은 안 되는 상황. 이럴 때 “계속 고쳐봐”를 반복하는 것은 최악의 선택입니다.

세 가지 대처법을 기억하십시오.

첫째, 범위를 축소합니다. AI가 세 가지를 동시에 해결하려다 꼬이는 경우가 많습니다. “지금 이 에러 하나만 해결하자. 나머지는 나중에”라고 범위를 좁혀주면 훨씬 잘 됩니다.

둘째, 맥락을 재설정합니다. 대화가 길어질수록 AI의 맥락이 오염됩니다. 이전에 시도했다 실패한 접근이 맥락에 남아서, 같은 실수를 반복하는 것이죠. 새 대화를 시작하고, 현재 상태와 원하는 결과만 깨끗하게 전달하는 것이 효과적입니다.

셋째, 단계를 더 잘게 쪼갭니다. AI가 한 번에 해결하지 못하는 문제를 만났다면, 그 문제를 더 작은 단위로 분해해서 하나씩 해결하도록 유도합니다. 이것은 사실 인간 개발자가 복잡한 문제를 풀 때 쓰는 방법과 동일합니다.

6.2 AI 결과물 검증의 기술

“왜 이렇게 짰지?”를 물어보는 습관

AI가 코드를 생성하면, 대부분의 사람은 “돌아가는지”부터 확인합니다. 돌아가면 넘어갑니다. 여기서 차이가 갑니다.

코드 리뷰를 해본 분이라면 아실 겁니다. 좋은 리뷰어는 “이 코드가 왜 이런 구조인지”를 묻습니다. AI가 짠 코드에도 같은 질문을 던져야 합니다. “왜 이 라이브러리를 선택했지?”, “왜 이 패턴을 적용했지?”, “이 구조가 이 맥락에서 최선인가?” AI가 만든 코드를 그대로 받아들이는 것은 동료가 올린 PR을 읽지도 않고 승인하는 것과 같습니다.

이 습관이 중요한 이유는, 단순히 버그를 잡기 위해서만이 아닙니다. AI의 설계 판단을 이해하는 과정에서 자신의 설계 감각이 성장하기 때문입니다. AI가 왜 이런 선택을 했는지 따져보다 보면, “아, 이런 상황에서는 이 패턴이 더 적합하구나”를 체득하게 됩니다.

AI 코드 검증 체크리스트

AI가 생성한 코드를 받았을 때, 다음 항목들을 순서대로 확인하십시오.

1. **의도 일치 확인:** 내가 요청한 것과 AI가 구현한 것이 일치하는가? 스펙의 핵심 요구사항이 빠지지 않았는가?
2. **존재하지 않는 것 확인:** AI가 호출하는 API, 메서드, 라이브러리가 실제로 존재하는가? (AI는 그럴듯하지만 존재하지 않는 것을 만들어내는 경우가 있습니다)
3. **보안 점검:** 사용자 입력을 검증하고 있는가? SQL 인젝션, XSS 등의 기본적인 보안 취약점은 없는가? 민감 정보가 로그에 노출되지 않는가?
4. **에러 처리 확인:** 예외 상황이 적절히 처리되고 있는가? 에러가 삼켜지고(swallow) 있지는 않은가?
5. **성능 감각 점검:** 불필요한 반복, N+1 쿼리, 과도한 메모리 사용은 없는가?
6. **불필요한 복잡도 확인:** 과도한 추상화나 불필요한 디자인 패턴이 적용되어 있지 않은가? 5줄이면 될 것을 20줄로 짜지 않았는가?
7. **의존성 점검:** 불필요한 외부 라이브러리가 추가되지 않았는가? 그 라이브러리가 유지보수되고 있는 것인가?

이 7개 항목이 전부는 아니지만, 이것만 확인해도 가장 흔한 AI 코드 함정의 80%는 잡을 수 있습니다.

AI 코드의 대표적 함정들

실무에서 반복적으로 마주치는 AI 코드의 함정을 정리하겠습니다.

과도한 추상화. AI는 “좋은 코드”를 짜려는 경향이 있습니다. 문제는, 그 “좋은”이 때때로 과도하다는 것입니다. 간단한 CRUD에 팩토리 패턴, 전략 패턴, 추상 클래스를 겹겹이 씌우는 경우가 적지 않습니다. 3개의 엔드포인트를 위해 5개의 인터페이스를 만들 필요는 없습니다.

존재하지 않는 API 호출. AI는 자신이 학습한 데이터를 기반으로 코드를 생성합니다. 라이브러리의 버전이 바뀌면서 사라진 메서드, 혹은 애초에 존재하지 않았지만 그럴듯한 이름의 메서드를 호출하는 경우가 있습니다. 컴파일은 되는 것처럼 보이지만 런타임에 실패합니다. 반드시 공식 문서와 대조하십시오.

보안 취약점. AI는 기능 구현에는 뛰어나지만, 보안에는 소홀한 경우가 많습니다. 사용자 입력을 그대로 쿼리에 넣거나, 비밀번호를 평문으로 저장하거나, 인가 검사를 빠뜨리거나. 특히 프로토타입을 프로덕션으로 전환할 때 이 부분을 놓치면 사고가 됩니다.

불필요한 의존성. 표준 라이브러리로 충분히 해결할 수 있는 것에 외부 라이브러리를 끌어오는 경우.

`commons-lang3` 로 한 줄이면 되는 것에 낫선 유틸리티 라이브러리를 추가하거나, 심지어 유지보수가 멈춘 라이브러리를 가져오기도 합니다.

“돌아가니까 됐다”의 위험성

“동작하는 코드”와 “좋은 코드”는 다릅니다. AI가 생성한 코드가 테스트를 통과하면, “됐다!”고 넘어가고 싶은 유혹이 강합니다. 하지만 “돌아가니까 됐다”는 기술 부채의 씨앗입니다.

지금 동작하는 코드가 3개월 뒤에도 유지보수 가능한가? 다른 팀원이 이 코드를 읽었을 때 의도를 파악할 수 있는가? 요구사항이 변경되었을 때 수정이 용이한가? 이 질문들에 “아마도”라고 답한다면, 그 코드는 검증이 더 필요합니다.

AI가 짜준 코드라서 검증을 소홀히 하는 것은, 외주 업체가 납품한 코드를 테스트도 안 하고 프로덕션에 올리는 것과 같습니다. 누가 짰느냐가 중요한 것이 아닙니다. 코드의 품질을 판단하고 보증하는 것은 여전히 당신의 책임입니다.

6.3 디버깅의 새로운 방식

에이전트와 디버깅하기

디버깅은 에이전트와의 협업에서 가장 극적인 생산성 차이가 나는 영역입니다.

에이전트 도구에서 디버깅은 이전 세대와 근본적으로 다릅니다. ChatGPT에 에러 메시지를 복사 붙여넣기 하던 시대에는 환경 정보, 관련 코드, 시도한 방법을 일일이 제공해야 했습니다. 에이전트는 다릅니다. 프로젝트를 직접 읽고, 빌드를 실행하고, 에러를 직접 확인합니다. 환경이 뭔지, 관련 코드가 어디에 있는지를 따로 알려줄 필요가 없습니다. CLAUDE.md에 프로젝트 정보가 있고, 에이전트가 파일 시스템에 접근할 수 있으니까요.

그래서 에이전트에게 에러를 전달하는 방법은 간단합니다.

“주문 조회 API에서 특정 사용자만 500 에러가 나. 로그에 이런 에러가 찍혀 있어. 원인 찾아서 고쳐줘.”

에이전트는 해당 코드를 읽고, 에러 원인을 분석하고, 코드를 수정하고, 테스트를 실행합니다. 이 모든 것을 자율적으로 수행합니다.

진짜 중요한 것은 “확인”이다

그런데 여기서 위험한 순간이 옵니다. 에이전트가 수정을 완료하고 “해결했습니다”라고 말합니다. 테스트도 통과합니다. 대부분의 사람은 여기서 “됐다!” 하고 넘어갑니다.

이것이 함정입니다.

에이전트가 에러를 고쳤다는 것과, 에이전트가 올바르게 고쳤다는 것은 다릅니다. 증상을 없앤 것과 원인을 해결한 것도 다릅니다. null이 들어와서 에러가 났는데, 에이전트가 null 체크를 추가해서 에러를 삼켜 버렸을 수 있습니다. 테스트는 통과하지만, 근본 원인은 그대로입니다.

그래서 에이전트가 버그를 고친 뒤에 해야 할 것은, **“왜 이 에러가 났고, 네가 한 수정이 왜 올바른 해결인지 설명해줘”**라고 묻는 것입니다. 이 질문 하나가 결정적입니다. 에이전트의 설명을 듣고, 그 논리가 납득이 되면 수정을 수락합니다. 납득이 안 되면 다시 물어봅니다.

이 과정에서 두 가지가 동시에 일어납니다. 문제가 해결되고, 시스템에 대한 당신의 이해도가 깊어집니다. 다음에 비슷한 에러를 만났을 때, 에이전트의 설명을 듣기 전에 원인을 추론할 수 있게 됩니다. 이것이 에이전트를 통해 배우면서 동시에 문제를 해결하는, AI 네이티브 디버깅의 핵심입니다.

에이전트를 더 신뢰해야 할 때 vs 더 의심해야 할 때

에이전트가 수정한 결과를 어느 수준까지 확인해야 하는지는, 문제의 성격에 따라 달라집니다. 이 감각이 중요합니다.

에이전트를 신뢰해도 되는 영역: - 에러 메시지가 명확하고, 원인이 코드 수준에 있을 때 - 라이브러리나 프레임워크의 설정 문제일 때 - 문법 오류, 타입 불일치, 잘못된 API 사용 같은 패턴 기반 오류일 때 - 스택 트레이스가 명확하게 문제 위치를 가리키고 있을 때

이런 경우에는 에이전트가 고치고 테스트 통과하면, 수정 내용을 간단히 확인하는 정도로 충분합니다.

더 깊이 의심해야 하는 영역: - 비즈니스 로직의 흐름이 잘못되었을 때 (에이전트는 비즈니스 의도를 모릅니다) - 동시성 문제, 레이스 컨디션 같은 비결정적 버그 - 인프라 환경에 따라 달라지는 문제 (로컬에서는 되고 서버에서 안 되는 경우) - 여러 서비스에 걸쳐 발생하는 분산 시스템의 버그

이런 경우에는 에이전트가 “고쳤습니다”라고 해도, 수정의 논리를 반드시 확인해야 합니다. 에이전트에게 “이 수정이 근본 원인을 해결하는 거야, 아니면 증상만 가리는 거야?”라고 물어보는 것이 좋습니다. 핵심은 **검증의 강도를 문제의 성격에 맞게 조절하는 것**입니다.

6.4 실전 시나리오: 버그 하나를 에이전트와 함께 잡기

이론은 충분합니다. 실제 상황을 하나 살펴보겠습니다.

상황

QA 팀에서 리포트가 들어왔습니다. “특정 사용자의 주문 조회 시 500 에러가 발생합니다. 다른 사용자는 정상입니다.”

로그를 확인하니 이런 스택 트레이스가 남아 있습니다.

```
org.springframework.dao.InvalidDataAccessApiUsageException:
  Parameter value [null] did not match expected type [java.lang.Long];
  nested exception is java.lang.IllegalArgumentException
    at com.example.order.repository.OrderRepository.findById(...)
    at com.example.order.service.OrderService.getOrders(...)
    at com.example.order.controller.OrderController.getUserOrders(...)
```

1단계: 30초만 생각한다

에이전트에게 바로 넘기기 전에, 잠깐 멈추고 머릿속으로 정리합니다. 이 30초가 이후의 검증 품질을 결정합니다.

- 무엇이 문제인가: 특정 사용자의 주문 조회 시 500 에러
- 언제 발생하는가: 특정 사용자에게서만. 다른 사용자는 정상
- 에러의 핵심: `Parameter value [null]`, Long 타입을 기대했는데 null이 들어옴
- 내 가설: `userId`가 null인 채로 리포지토리에 전달되고 있다

이 가설을 갖고 에이전트에게 넘깁니다.

2단계: 에이전트에게 말린다

“주문 조회에서 특정 사용자만 500 에러가 나. 에러 로그 이거야. [에러 스택 트레이스] `userId`가 null로 넘어가는 것 같은데, `OrderController`부터 추적해서 원인 찾고 고쳐줘.”

기술 스택을 설명하지 않습니다. 에이전트는 프로젝트를 읽을 수 있으니까요. 관련 코드를 복사해서 넘기지 않습니다. 에이전트가 직접 파일을 열어서 확인합니다.

에이전트는 `OrderController` → `OrderService` → `OrderRepository` 경로를 추적하고, `JwtTokenProvider` 의 `getUserId()` 메서드까지 파고들어갑니다. 그리고 원인을 찾습니다. 토큰의 `sub` 클레임이 숫자가 아닌 UUID 형태인 사용자가 있어서, `Long.parseLong()` 이 실패하면서 catch 블록에서 null을 반환하고 있었다고.

에이전트가 코드를 수정하고, 테스트를 돌려서 통과시킵니다. “해결했습니다.”

3단계: 여기서 멈추지 않는다

대부분의 사람은 여기서 “좋아, 됐다”하고 넘어갑니다. 하지만 앞서 이야기한 것을 기억하십시오. **에이전트가 고쳤다는 것과, 올바르게 고쳤다는 것은 다릅니다.**

“수정 내용 설명해줘. 근본 원인이 뭐였고, 네 수정이 왜 올바른 해결인지.”

에이전트가 설명합니다. UUID 형태의 `userId`를 가진 사용자가 있었고, 기존 코드는 모든 `userId`가 Long이라고 가정하고 있었다고. 수정 방법은 두 가지가 있었는데, UUID를 Long으로 변환하는 방법과 파싱 실패 시 명확한 에러를 던지는 방법 중 후자를 선택했다고.

여기서 당신의 판단이 들어갑니다. “잠깐, 그 UUID 형태의 `userId`는 왜 존재하는 거지? 마이그레이션 과정에서 잘못 들어간 데이터인가, 아니면 원래 의도된 것인가?” 이 질문은 에이전트가 스스로 할 수 없는 질문입니다. 비즈니스 맥락을 알아야 하니까요.

데이터를 확인해보니, OAuth 소셜 로그인으로 가입한 사용자의 경우 외부 provider의 ID가 UUID 형태로 저장되어 있었습니다. 에이전트의 수정은 증상은 해결했지만, 근본적으로는 소셜 로그인 사용자의 `userId` 처리 로직을 재설계해야 하는 문제였습니다.

이 시나리오에서 배울 것

에이전트가 버그를 잡는 속도는 놀랍습니다. 코드를 추적하고, 원인을 찾고, 수정하고, 테스트하는 과정을 자율적으로 수행합니다. 하지만 이 시나리오에서 진짜 중요한 순간은 에이전트가 코드를 고친 뒤에 일어났습니다. “왜 이런 수정을 했는지 설명해줘”라고 물었고, 그 설명을 듣고 “잠깐, 근본 원인이 다른 곳에 있는 것 같은데”라는 판단을 한 것.

이것이 이 장 전체를 관통하는 메시지입니다. 에이전트가 코드를 짜고, 에러를 분석하고, 수정을 적용합니다. 하지만 그 수정이 증상 해결인지 근본 해결인지를 판단하는 것은 개발자입니다. 3장에서 말한 “70% 문제의 나머지 30%”가 바로 이것입니다. 에이전트의 70%를 신뢰하되, 검증하고 판단하는 30%에서 당신의 전문성이 발휘됩니다.

이 장을 마치며

이 장에서 우리는 에이전트에게 코드를 맡기고, 그 결과를 부리는 전체 과정을 다뤘습니다. 핵심을 정리하겠습니다.

복잡한 기능은 대화를 이어가면서 단계별로 확인해야 합니다. 에이전트의 결과물은 반드시 검증해야 합니다. 디버깅에서는 에이전트가 수정한 뒤에 “왜 이렇게 고쳤는지 설명해줘”라고 묻는 것이 결정적입니다. 그리고 이 모든 과정에서, 에이전트의 수정이 증상 해결인지 근본 해결인지를 판단하는 것은 당신의 몫입니다.

에이전트가 코드를 짜주는 시대에 개발자의 가치는 “코드를 짤 수 있느냐”가 아닙니다. **“이 코드가 맞는 지 판단할 수 있느냐”**입니다. 그 판단력은 기초 지식, 경험, 그리고 “왜?”를 묻는 습관에서 나옵니다.

다음 장에서는 이 결과물의 품질을 한 단계 더 끌어올리는 방법 - 리팩터링, 테스트, 코드 리뷰를 AI와 함께 수행하는 워크플로우를 다루겠습니다.

셀프 체크

- ☐ 에이전트에게 구현을 맡길 때, 통째로 맡기기와 단계별 확인의 기준을 설명할 수 있다
- ☐ 에이전트가 생성한 코드를 검증할 때 확인해야 할 핵심 항목을 알고 있다
- ☐ 에이전트가 버그를 고친 뒤에 “왜 이렇게 고쳤는지” 확인하는 습관의 의미를 이해했다
- ☐ 에이전트를 신뢰해도 되는 영역과 더 깊이 의심해야 하는 영역을 구분할 수 있다

이 장의 미션

1. 에이전트에게 간단한 기능 구현을 하나 맡겨보십시오. 결과물을 받은 뒤, 이 장의 “검증 체크리스트” 7개 항목으로 코드를 리뷰하십시오. 최소 2가지 개선점 또는 잠재적 문제를 찾아서 기록하십시오.
2. 최근에 만난 에러를 에이전트에게 맡기고 수정을 받은 뒤, “왜 이렇게 고쳤는지, 근본 원인이 뭐였는지 설명해줘”라고 물어보십시오. 에이전트의 설명이 납득이 되는지 직접 판단해보십시오.

SI와 함께 활용하기

- **코드 검증:** “방금 만든 코드, 보안 취약점이나 성능 병목 없는지 확인해줘. 특히 존재하지 않는 API 호출하고 있으면 알려줘.”
- **디버깅:** “이 에러 나는데 원인 찾아서 고쳐줘. [에러 메시지]. 고친 다음에 왜 이런 에러가 났는지, 네 수정이 왜 올바른지 설명해줘.”
- **코드 개선:** “방금 구현한 것, 과도한 추상화 있으면 단순화하고, 에러 처리 빠진 곳 보완하고, 불필요한 의존성 있으면 제거해줘. 뭘 왜 바꿨는지 설명해줘.” # 7장. 리팩터링과 테스트 - 본질을 묻다

누가 코드를 읽는가

6장에서 우리는 에이전트가 짠 코드를 검증하고 디버깅하는 법을 다뤘습니다. 코드가 돌아갑니다. 테스트도 통과합니다. 에이전트가 “완료했습니다”라고 말합니다.

여기서 대부분의 책이라면, “일단 돌아가게 만들었으니 이제 코드 품질을 올리자”고 말할 겁니다. 리팩터링을 하고, 테스트 커버리지를 높이고, 클린 코드 원칙을 적용하라고. 그것이 “좋은 개발자”의 덕목이라고.

잠깐 멈추겠습니다. 한 가지 질문을 던지겠습니다.

리팩터링은 누구를 위한 것입니까?

이 질문에 대한 기존의 답은 명확했습니다. “다른 개발자가 읽기 쉽도록.” “6개월 후의 나 자신을 위해.” “팀의 생산성을 위해.” 리팩터링의 본질은 **사람과 사람 사이의 커뮤니케이션**이었습니다. 코드를 읽는 것이 사람이고, 수정하는 것이 사람이었으니, 사람이 이해하기 쉬운 구조로 정리하는 것이 곧 품질이었습니다.

그런데 지금, 코드를 읽고 수정하는 주체가 바뀌고 있습니다.

4장에서 에이전트가 프로젝트 전체를 읽는다고 했습니다. 5장에서 에이전트와 대화를 통해 설계를 한다고 했습니다. 6장에서 에이전트가 코드를 작성하고, 디버깅하고, 테스트한다고 했습니다. 코드를 가장 많이 읽고, 가장 많이 수정하는 주체가 사람에서 에이전트로 이동하고 있습니다.

그렇다면 질문을 바꿔야 합니다. **사람이 읽기 쉬운 코드가 정말로 가장 중요한 코드 품질인가? 아니면, 에이전트가 이해하고 작업하기 쉬운 구조가 더 중요한 것인가?**

이 질문에서 출발하겠습니다.

7.1 리팩터링이라는 환상

“좋은 코드”는 왜 좋았는가

리팩터링의 역사를 잠깐 돌아보겠습니다.

Martin Fowler가 1999년에 “Refactoring”을 출판했을 때, 소프트웨어 개발의 가장 큰 문제 중 하나는 “코드를 이해하는 비용”이었습니다. 팀원이 작성한 코드를 다른 팀원이 수정하려면, 먼저 그 코드를 읽고 이해해야 했습니다. 변수명이 `a` 이면 뭘지 모릅니다. 함수가 200줄이면 흐름을 따라가기 어렵습니다. 같은 로직이 세 곳에 복사되어 있으면 하나만 고치고 나머지를 놓칩니다.

리팩터링은 이 문제에 대한 해법이었습니다. 코드를 읽기 쉽게 만들어서, 사람이 이해하는 비용을 줄이자. 함수를 작게 쪼개고, 의미 있는 이름을 붙이고, 중복을 제거하고, 계층을 분리하자. “Clean Code”라는 개념 자체가, 본질적으로 **사람의 인지 부하를 줄이기 위한 것이었습니다**.

그리고 이것은 20년간 맞는 답이었습니다. 코드를 읽고 수정하는 것이 사람이었으니까요.

패러다임이 바뀌었다

지금은 상황이 다릅니다.

에이전트는 200줄짜리 함수를 읽는 데 0.1초 걸립니다. 변수명이 `a` 여도 문맥에서 의미를 추론합니다. 코드가 여러 파일에 흩어져 있어도 전체를 동시에 파악합니다. 사람에게 읽기 어려운 코드가 에이전트에게는 아무 문제가 되지 않습니다.

반대로, 에이전트에게 진짜 중요한 것은 다른 것입니다. 프로젝트 컨텍스트 파일(CLAUDE.md)이 잘 정리되어 있는지, 빌드와 테스트 명령어가 명확한지, 서비스 간 의존성이 문서화되어 있는지, 코딩 컨벤션이 일관되게 적용되어 있는지. 에이전트의 생산성을 결정하는 것은 코드의 “아름다움”이 아니라, **프로젝트의 맥락이 얼마나 잘 전달되는지**입니다.

4장에서 세팅한 CLAUDE.md가 바로 이것이었습니다. 에이전트를 위한 온보딩 문서. 사람을 위한 클린 코드가 아니라, 에이전트를 위한 클린 컨텍스트.

그래서 리팩터링은 필요 없는 건가?

아닙니다. 항상 불필요한 것은 아닙니다. 하지만 **무조건 해야 하는 것도 아닙니다**. 핵심은 구분입니다.

리팩터링이 여전히 필요한 경우:

사람이 직접 읽고 판단해야 하는 코드. 비즈니스의 핵심 로직, 보안 관련 코드, 아키텍처의 경계를 정의하는 코드. 이런 코드는 사람이 이해할 수 있어야 합니다. 에이전트가 아무리 잘 짜도, 이 부분에서 의도가 불명확하면 사람이 검증할 수 없습니다. 여기서는 여전히 리팩터링의 가치가 있습니다.

팀원 간 공유되는 인터페이스도 마찬가지입니다. API 계약, 데이터 모델의 공개 인터페이스, 팀 내 공통 규약. 이런 것들은 사람이 읽어야 하니까, 가독성이 중요합니다.

리팩터링이 불필요한 경우:

에이전트가 생성하고 에이전트가 수정하는 코드. 내부 구현의 디테일, 유틸리티 함수의 내부 로직, 한 번 만들고 에이전트가 알아서 유지보수하는 코드. 이런 코드를 사람이 읽기 쉽게 정리하는 데 시간을 쓰는 것은, 투자 대비 회수가 낮습니다.

한 번 쓰고 다시 볼 일 없는 스크립트, 곧 교체될 예정인 코드, 프로토타입. 이런 코드에 클린 코드 원칙을 적용하는 것은 교과서적으로는 옳지만, 현실적으로는 낭비입니다.

리팩터링 대신 해야 할 것

리팩터링에 쓸 시간이 있다면, 이것에 투자하는 편이 ROI가 높을 수 있습니다.

프로젝트 컨텍스트를 정교하게 만드는 것. CLAUDE.md에 한 줄을 추가하는 것이, 코드 한 함수를 리팩터링하는 것보다 에이전트의 전체 생산성에 더 큰 영향을 줍니다. “이 서비스에서 결제 관련 변경 시 반드시 PG사 웹훅 핸들러도 확인할 것” - 이 한 줄이 에이전트의 실수를 방지합니다.

빌드와 테스트 파이프라인을 자동화하는 것. 에이전트가 코드를 수정한 뒤 자동으로 빌드가 돌아가고, 테스트가 실행되고, 결과를 확인할 수 있는 환경. 이것이 코드 품질을 보장하는 가장 확실한 방법입니다. 코드가 아무리 더러워도, 테스트가 통과하면 동작은 보장됩니다.

에이전트 사이클을 설계하는 것. 8장에서 자세히 다루겠지만, “만들고 - 리뷰하고 - 수정하는” 사이클을 에이전트에 내장하는 것. 이것이 코드를 예쁘게 만드는 것보다 실질적인 품질 향상을 가져옵니다.

물론, 리팩터링이 필요한 상황이 오면 에이전트에게 맡기면 됩니다. “이 함수 200줄인데, 책임별로 분리해줘. 기존 동작은 유지해.” 에이전트가 30초 만에 해냅니다. 리팩터링 자체의 실행 비용은 이미 거의 제로입니다. 중요한 것은 “언제 하느냐”의 판단이지, “어떻게 하느냐”의 기술이 아닙니다.

7.2 테스트: 본질로 돌아가기

테스트는 왜 생겨났는가

테스트의 역사를 잠깐 돌아보겠습니다. 이 역사를 이해하면, 지금 무엇이 진짜 필요하고 무엇이 관성인지가 보입니다.

소프트웨어 테스트가 체계화된 것은 1990년대 후반부터입니다. Kent Beck이 1999년에 “Extreme Programming Explained”를 출판하면서 TDD(Test-Driven Development)가 주목받기 시작했습니다. 왜 이 시기에 테스트가 중요해졌을까요?

소프트웨어가 복잡해지고, 팀이 커지고, 배포 주기가 빨라지면서, **“내가 고친 코드가 다른 곳을 깨뜨리지 않았다”는 확신**이 필요해졌기 때문입니다. 회귀 테스트(regression test)의 핵심 가치가 바로 이것입니다. “수정 전과 동일하게 동작한다”는 보장.

그리고 TDD는 여기서 한 걸음 더 나갔습니다. “테스트를 먼저 쓰면, 구현의 방향이 명확해진다.” 테스트가 곧 스펙이 된다는 발상이었습니다.

이 두 가지 - 회귀 방지와 스펙으로서의 테스트 - 가 테스트의 본질적 가치입니다.

통념에 도전하겠습니다

그런데 지금 개발 현장에서 “테스트”라고 하면 떠오르는 것은 무엇입니까?

단위 테스트 커버리지 80%. TDD를 반드시 해야 한다. 모든 public 메서드에 테스트를 작성해야 한다. 커버리지가 낮으면 PR을 머지할 수 없다.

이것들이 정말 본질입니까?

냉정하게 따져보겠습니다. 단위 테스트 커버리지 80%를 달성했는데, 프로덕션에서 결제가 두 번 되는 버그가 나왔습니다. 왜? 단위 테스트는 함수 하나의 동작을 검증하지, **시스템 전체의 행동**을 검증하지 않으니까요. 함수 A는 정상, 함수 B도 정상, 하지만 A와 B가 만나는 지점에서 문제가 생깁니다.

TDD를 철저히 했는데, 요구사항이 바뀌면서 테스트를 전부 고쳐야 합니다. 구현 코드 수정에 30분, 테스트 코드 수정에 2시간. 테스트가 변경을 돕는 것이 아니라, 변경을 방해하고 있습니다.

이런 상황을 겪어본 적이 있다면, 한 가지 불편한 진실을 인정해야 합니다. **많은 단위 테스트가 실제로는 코드의 “구현”을 테스트하고 있지, “행동”을 테스트하고 있지 않습니다.** 구현이 바뀌면 테스트도 깨지니까, 테스트가 리팩터링을 보호하는 것이 아니라 리팩터링을 가로막습니다.

AI가 만든 테스트는 한 번 더 의심해야 합니다

여기에 한 겹이 더 있습니다. 6장에서 에이전트에게 “테스트도 만들어줘”라고 맡길 수 있다고 했습니다. 편리합니다. 그런데 에이전트가 만든 테스트에는 공통적인 함정이 세 가지 있습니다.

함정 1: 구현을 복사한 테스트. 에이전트는 방금 자기가 짰 구현을 보면서 테스트를 만듭니다. 그래서 구현의 로직을 그대로 옮겨 적은 테스트가 나오기 쉽습니다. 구현이 틀렸으면 테스트도 같은 방식으로 틀려서, 절대 실패하지 않습니다.

함정 2: Happy Path만 테스트. 정상 입력, 정상 흐름, 정상 결과까지만 검증합니다. 그 바깥, 즉 만료된 쿠폰, 음수 금액, 동시 요청 같은 경계는 비어 있습니다.

함정 3: 모킹 과다. 의존성을 전부 모킹해서 “함수가 호출되었는가”만 검증하고, “결과가 올바른가”는 검증하지 않습니다.

세 함정의 공통점은 하나입니다. **녹색 바가 떠도 아무것도 보장하지 않는다는 것.** 테스트가 있으니 안심했는데 사실 아무것도 검증하고 있지 않았던 상태. 이것이 커버리지 숫자보다 위험한 이유는, 숫자는 멀쩡하게 채워져 있기 때문입니다.

테스트를 테스트하는 법: 뮤테이션

그러면 내 테스트가 허수인지 아닌지 어떻게 알 수 있을까요. 방법이 있습니다. **뮤테이션 테스트 (Mutation Testing)**, 코드를 의도적으로 조금씩 망가뜨리고(뮤테이션) 테스트가 그 변형을 잡아내는지 확인하는 기법입니다. 유통기한 검사 시스템이 제대로 작동하는지 확인하려면, 유통기한이 하루 지난 우유를 일부러 넣어보면 됩니다.

에이전트에게 이렇게 말합니다.

다음 함수의 코드를 의도적으로 변형한 버전 5개를 만들어줘.
각 변형은 미세하지만 버그를 포함해야 해.
그리고 현재 테스트가 각 변형을 잡아내는지 확인해줘.

변형 예시:

- > 를 >= 로 변경
- + 를 - 로 변경
- && 를 || 로 변경

- return 값을 null로 변경
- 조건문의 true/false를 반전

쿠폰 할인 함수로 실제 돌려본 결과입니다.

```
// 원본 함수
function applyDiscount(price: number, coupon: Coupon): number {
  if (coupon.expiresAt < new Date()) throw new Error('만료된 쿠폰');
  if (price < coupon.minOrderAmount) throw new Error('최소 주문 금액 미달');
  return Math.max(0, price - coupon.discountAmount);
}

// 뮤턴트 1: < 를 <= 로 변경 (만료일 당일에 쿠폰 사용 가능해짐)
// -> 현재 테스트가 잡는가? 잡지 못함. 만료일 당일 테스트가 없음
// 뮤턴트 2: < 를 > 로 변경 (최소 금액 조건이 반전됨)
// -> 잡음. 정상 주문 테스트에서 실패
// 뮤턴트 3: - 를 + 로 변경 (할인 대신 가격이 올라감)
// -> 잡음. 할인 금액 검증 테스트에서 실패
// 뮤턴트 4: Math.max(0, ...) 제거 (음수 결과 가능)
// -> 잡지 못함. 할인 금액이 가격보다 큰 케이스 테스트가 없음
// 뮤턴트 5: 에러 메시지를 바꿈 (로직은 동일)
// -> 잡지 못함. 에러 메시지를 검증하지 않음
```

5개 중 2개를 잡지 못했습니다. 만료일 당일, 그리고 할인 금액이 가격을 넘는 경우. 둘 다 프로덕션에서 돈이 걸리는 케이스입니다. 이제 무엇을 보강해야 하는지가 명확해졌습니다. 커버리지 숫자는 이것을 알려주지 않습니다. 뮤테이션은 알려줍니다.

함께 쓰면 좋은 질문이 하나 더 있습니다. **“현재 테스트는 통과하지만 실제로는 버그인 입력을 찾아줘.”** 에이전트를 테스트의 작성자에서 테스트의 공격자로 전환시키는 질문입니다. 같은 에이전트라도 역할이 바뀌면 다른 것을 봅니다. 이 두 가지, 뮤테이션과 공격자 전환은 “테스트가 있다”를 “테스트를 믿을 수 있다”로 바꾸는 가장 빠른 도구입니다.

진짜 필요한 테스트는 무엇인가

본질로 돌아가겠습니다. 테스트의 본질적 가치 두 가지를 다시 봅니다.

1. **회귀 방지:** 수정이 기존 동작을 깨뜨리지 않았다는 보장
2. **스펙으로서의 역할:** 시스템이 어떻게 행동해야 하는지의 정의

이 두 가지를 가장 효과적으로 달성하는 테스트는 무엇입니까?

인수 테스트(Acceptance Test)입니다. “사용자가 쿠폰을 적용하면 할인 금액이 반영된다”, “결제 완료되면 3초 안에 확인 이메일이 발송된다”, “재고가 0인 상품은 구매 버튼이 비활성화된다.” 이것들은 구

현의 디테일이 아니라, **시스템의 행동**을 정의합니다. 구현이 바뀌어도 행동이 같으면 테스트는 통과합니다.

그리고 한 단계 더 나아가면, **E2E(End-to-End) 테스트**입니다. API 호출부터 DB 저장, 이벤트 발행, 외부 서비스 연동까지 전체 흐름을 검증합니다. 단위 테스트 100개보다 잘 설계된 E2E 테스트 10개가 프로덕션 장애를 더 확실하게 방지합니다.

3장에서 제가 “사이클 돌려”라는 세 글자로 프로젝트 전체 검수를 실행한다고 했습니다. 그 검수의 핵심이 바로 이것입니다. 에이전트가 E2E 시나리오를 자동으로 실행하고, 결과를 보고합니다. 시스템이 의도대로 행동하는지를 자동으로 검증하는 것. 이것이 테스트의 궁극적 형태입니다.

TDD는 해야 하나, 말아야 하나

“그러면 TDD는 안 해도 되니까?”

질문 자체가 잘못되었습니다. “해야 한다 / 안 해야 한다”는 이분법이 문제입니다.

TDD가 효과적인 상황이 있습니다. 복잡한 비즈니스 규칙을 구현해야 할 때, 테스트를 먼저 정의하면 에이전트에게 의도를 전달하는 가장 정확한 방법이 됩니다. “이 입력에 이 결과가 나와야 해”를 테스트로 보여주면, 에이전트는 그 테스트를 통과하는 코드를 만듭니다. 5장에서 이야기한 “의도를 전달하는 법”의 가장 정밀한 형태가 TDD입니다.

하지만 모든 코드에 TDD를 적용해야 하는 것은 아닙니다. 간단한 CRUD, 에이전트가 알아서 만들고 테스트까지 하는 코드, 프로토타입. 이런 것에 TDD를 고집하는 것은 의식(ritual)이지 전략이 아닙니다.

TDD가 강력한 경우: 비즈니스 규칙이 복잡하고, 엣지 케이스가 많고, 에이전트에게 정확한 행동을 정의해줘야 할 때.

TDD가 불필요한 경우: 에이전트가 구현하고 에이전트가 테스트하면 충분한, 단순한 기능. 테스트를 먼저 쓰는 비용보다 에이전트에게 “만들고 테스트도 짜줘”라고 하는 것이 빠를 때.

에이전틱 테스트 자동화

테스트의 미래는 에이전트가 테스트를 “짜는” 것이 아닙니다. 에이전트가 테스트를 “실행하고 판단하는” 것입니다.

구체적으로 말하면 이렇습니다. 에이전트가 코드를 수정하면, 자동으로 빌드가 돌아가고, 기존 테스트가 실행되고, 실패하면 에이전트가 원인을 분석해서 수정합니다. 이 사이클이 자동으로 반복됩니다. 6장에서 이미 이 흐름을 봤습니다.

여기서 한 단계 더 나아가면, E2E 시나리오를 YAML이나 자연어로 정의하고, 에이전트가 그 시나리오를 자동으로 실행하는 시스템을 구축할 수 있습니다. 3장에서 보여드린 “결제 플로우 자동 검증 시스템”이 바로 이것이었습니다. 시나리오 정의 -> 자동 실행 -> 결과 보고. 이것이 단위 테스트 커버리지 80%보다 실질적인 안전망입니다.

궁극적으로 당신이 해야 할 것은 이것입니다. 에이전트에게 “코드 수정할 때마다 이 시나리오들이 통과하는지 확인해”라고 제약을 거는 것. 테스트를 일일이 작성하는 것이 아니라, **시스템의 행동을 정의하고 그 검증을 자동화하는 것**. 이것이 AI 시대의 테스트 전략입니다.

7.3 코드 리뷰: 무엇을 리뷰할 것인가

6장에서 에이전트의 결과물을 검증하는 법을 이미 다뤘습니다. 코드 리뷰는 그 검증의 공식적인 형태입니다. 여기서는 간결하게 핵심만 짚겠습니다.

리뷰의 초점이 이동했다

과거의 코드 리뷰: “이 코드가 올바르게 동작하는가? 읽기 쉬운가? 컨벤션을 따르는가?”

에이전트 시대의 코드 리뷰: “이 변경이 의도와 일치하는가? 시스템의 행동이 달라지지 않았는가? 비즈니스 맥락에서 올바른 결정인가?”

에이전트가 코드를 작성하는 시대에, 문법 실수나 컨벤션 위반은 거의 발생하지 않습니다. 에이전트는 CLAUDE.md의 규칙을 따르고, 기존 코드의 패턴을 학습해서 일관된 스타일로 작성합니다. 들여쓰기나 네이밍 같은 것을 사람이 리뷰할 필요는 거의 없습니다.

사람이 집중해야 하는 것은 고수준의 판단입니다. “이 기능이 비즈니스 요구사항과 맞는가?”, “이 아키텍처 결정이 장기적으로 유지 가능한가?”, “이 변경이 다른 서비스에 영향을 주지 않는가?” 6장에서 이야기한 “증상 해결 vs 근본 해결”의 판단이 코드 리뷰에서도 동일하게 적용됩니다.

에이전트를 1차 리뷰어로 활용하기

효과적인 패턴은 **1차 에이전트 리뷰, 2차 사람 리뷰**입니다.

에이전트에게 “이 변경사항 리뷰해줘. 보안, 성능, 에러 처리 관점에서”라고 말합니다. 에이전트가 기계적으로 잡을 수 있는 것들을 걸러냅니다. 사람은 에이전트가 잡지 못하는 고수준 판단에 집중합니다. 설계 의도, 비즈니스 맥락, 장기적 유지보수 가능성. 사소한 지적에 시간을 뺏기지 않으니, 정말 중요한 판단에 에너지를 쓸 수 있습니다.

7.4 실전: 프로토타입에서 프로덕션으로

이 장에서 이야기한 원칙을 하나의 흐름으로 엮어보겠습니다.

시나리오

에이전트와 함께 쿠폰 적용 기능을 만들었습니다. 5장의 대화형 설계, 6장의 구현 위임을 거쳐 코드가 완성되었습니다. 에이전트가 “완료했습니다”라고 말합니다.

1단계: 리팩터링이 필요한지 판단한다

먼저 묻습니다. “이 코드를 사람이 직접 읽고 이해해야 하는 부분이 있어?”

쿠폰 할인 계산 로직은 비즈니스의 핵심입니다. 할인율이 맞는지, 중복 적용이 정말 방지되는지를 사람이 검증해야 합니다. 이 부분은 가독성이 중요하니, 에이전트에게 맡깁니다. “할인 계산 로직 부분, 사람이 읽기 쉽게 정리해줘. 각 단계가 뭘 하는지 명확하게.”

나머지 - 컨트롤러 레이어, 입력 검증, 에러 응답 포맷 - 은 기존 프로젝트 패턴을 따르고 있고, 에이전트가 알아서 유지보수할 수 있는 부분입니다. 굳이 리팩터링할 필요 없습니다.

2단계: 진짜 필요한 테스트를 만든다

단위 테스트 커버리지 80%를 목표로 하지 않습니다. 대신, 이 기능의 **핵심 행동**을 정의합니다.

“쿠폰 적용 기능의 인수 테스트를 만들어줘. 시나리오: (1) 유효한 쿠폰 적용 시 할인 금액이 정확히 반영된다. (2) 만료된 쿠폰은 명확한 에러와 함께 거부된다. (3) 이미 사용된 쿠폰은 거부된다. (4) 최소 주문 금액 미달 시 거부된다. (5) 동시에 두 사용자가 같은 선착순 쿠폰을 사용하면 한 명만 성공한다.”

이 5개 시나리오가 단위 테스트 50개보다 강력합니다. 시스템의 행동을 정의하고, 구현이 바뀌어도 행동이 보장됩니다.

테스트를 만들었다면 마지막으로 그 테스트 자체를 시험합니다. “할인 계산 함수를 미세하게 변형한 버전 5개를 만들고, 방금 만든 테스트가 잡아내는지 확인해줘.” 잡지 못한 변형이 있다면 그것이 다음 보강 지점입니다. 테스트를 만드는 것과 테스트를 믿을 수 있게 만드는 것은 다른 작업입니다.

3단계: 에이전트 리뷰 + 사람 리뷰

에이전트에게 “보안, 성능, 에러 처리 관점에서 리뷰해줘”를 맡기고, 기계적 이슈를 먼저 해결합니다. 그 다음, 사람이 비즈니스 로직의 정합성을 확인합니다. “할인율 계산이 기획서와 일치하는가?”, “동시성 처리 방식이 우리 서비스의 트래픽 패턴에 적합한가?”

이 흐름에서 빠진 것

많은 것이 빠져 있습니다. 단위 테스트 커버리지 목표도 없고, 모든 메서드에 대한 테스트도 없고, 코드의 모든 부분을 리팩터링하지도 않았습니

다. 그리고 그것이 핵심입니다. **전부 하는 것이 아니라, 진짜 필요한 것만 하는 것.** 비즈니스 핵심 로직은 사람이 읽을 수 있게 정리하고, 시스템의 행동은 인수 테스트로 보장하고, 나머지는 에이전트에게 맡깁니다. 이것이 에이전트 시대의 코드 품질 전략입니다.

이 장을 마치며

이 장에서 도전한 것은 하나의 통념입니다. “리팩터링은 항상 좋고, 테스트는 많을수록 좋고, 커버리지는 높을수록 좋다.”

이것이 틀린 말은 아닙니다. 하지만 맹목적으로 따를 말도 아닙니다. 리팩터링은 누가 코드를 읽느냐에 따라 필요할 수도, 불필요할 수도 있습니다. 테스트는 무엇을 검증하느냐에 따라 가치가 달라집니다. 단위 테스트 커버리지 숫자보다, 시스템의 핵심 행동을 보장하는 인수 테스트와 E2E 테스트가 실질적입니다.

본질은 이것입니다. **코드 품질의 목적이 바뀌었습니다.** 사람이 읽기 쉬운 코드에서, 에이전트가 효과적으로 작업할 수 있는 프로젝트 구조로. 클린 코드에서 클린 컨텍스트로. 이 전환을 이해하면, 무엇에 시간을 쓸지가 명확해집니다.

8장에서는 이 관점을 더 넓혀서, 객체지향, 아키텍처, 엔지니어링 기법 전반의 가치가 어떻게 변하고 있는지를 이야기하겠습니다.

셀프 체크

- ☐ “좋은 코드”가 왜 좋았는지, 그 본질적 이유(사람 간 커뮤니케이션)를 설명할 수 있다
- ☐ 리팩터링이 필요한 경우와 불필요한 경우를 구분하는 기준을 알고 있다
- ☐ 단위 테스트 커버리지보다 인수 테스트/E2E 테스트가 왜 실질적인지 설명할 수 있다
- ☐ AI가 만든 테스트의 세 가지 함정(구현 복사, Happy Path만, 모킹 과다)을 알고, 뮤테이션 테스트로 허수 테스트를 적발할 수 있다
- ☐ TDD가 효과적인 상황과 불필요한 상황을 구분할 수 있다
- ☐ “클린 코드”에서 “클린 컨텍스트”로의 전환을 이해했다

이 장의 미션

1. 자신의 프로젝트에서 CLAUDE.md(또는 프로젝트 컨텍스트 파일)를 열고, “에이전트가 실수하기 쉬운 부분”을 한 줄 추가하십시오. 코드를 리팩터링하는 것보다 이 한 줄이 전체 생산성에 더 큰 영향을 줍니다.

2. 자신의 프로젝트에서 가장 중요한 기능 하나를 골라, 인수 테스트 시나리오 5개를 자연어로 작성하십시오. “사용자가 X를 하면 Y가 된다” 형식으로. 그리고 에이전트에게 이 시나리오를 테스트 코드로 만들게 하십시오.
3. 테스트가 이미 있는 함수 하나를 골라, 에이전트에게 뮤텐트 5개를 만들게 하고 현재 테스트가 몇 개를 잡는지 확인하십시오. 잡지 못한 뮤텐트가 곧 다음에 보강할 테스트입니다.

AI와 함께 활용하기

- **리팩터링 판단:** “이 코드에서 사람이 직접 읽고 검증해야 하는 부분이 어디야? 그 부분만 가독성 높게 정리해줘. 나머지는 그대로 뒤.”
- **인수 테스트 작성:** “이 기능의 핵심 행동을 검증하는 인수 테스트를 만들어줘. 정상 시나리오, 실패 시나리오, 동시성 시나리오 포함해서. 구현 디테일이 아니라 시스템 행동을 테스트해야 해.”
- **뮤테이션 테스트:** “이 함수를 미세하게 변형한 버그 버전 5개를 만들고, 현재 테스트가 각각을 잡아 내는지 확인해줘. 잡지 못한 변형을 방어하는 테스트를 추가해줘.”
- **E2E 자동화:** “이 서비스의 결제 플로우를 E2E로 검증하는 시나리오를 설계해줘. 사용자 인증부터 결제 완료 이메일까지 전체 흐름.” # 8장. 악보를 태우십시오 – 엔지니어링 기법의 역설

이 장의 존재 이유

1장에서 이렇게 썼습니다. “IDE를 켜지 마십시오.” “코드를 보지 마십시오.”

이 말을 듣고 불편했다면, 정상입니다. 아마 이런 생각이 떠오르셨을 겁니다.

“객체지향은? 테스트 코드는? 클린 아키텍처는? 대용량 트래픽? 이런 거 다 필요 없다는 건가?”

아닙니다.

객체지향, 테스트 코드, 헥사고널 아키텍처, DDD, 대용량 트래픽 처리, 동시성 제어 – 이 기법들은 여전히 시스템이 무너지는 순간에 당신을 살립니다. 서비스가 트래픽 100배를 견뎌야 할 때, 장애가 연쇄적으로 전파될 때, 레거시 코드 50만 줄을 리팩터링해야 할 때. 이런 순간에 기초 엔지니어링 역량이 없으면 AI도 소용없습니다.

무시하는 것이 아닙니다.

다만, 한 가지를 분명히 하겠습니다.

이것들의 '세부 지식'으로서의 가치가 예전과 같지 않습니다.

키워드면 충분합니다

과거에 헥사고날 아키텍처를 적용하려면 이런 과정이 필요했습니다. 개념을 이해하고, 포트와 어댑터 패턴을 학습하고, 인바운드와 아웃바운드 경계를 직접 설계하고, 수십 개의 인터페이스와 구현체를 일일이 작성하고, 테스트를 짜서 경계가 제대로 분리되었는지 검증합니다. 숙련된 개발자도 며칠이 걸렸습니다. 처음 적용하는 사람이라면 몇 주를 소모했습니다.

지금은 이렇게 합니다.

“이 프로젝트를 헥사고날 아키텍처로 구조 잡아줘. 인바운드 포트, 아웃바운드 포트, 어댑터 분리해서.”

끝입니다. 키워드만 알면 됩니다.

“DDD 기반으로 Aggregate Root 설계해줘.” “테스트 코드를 단위 테스트, 통합 테스트, E2E로 나눠서 짜줘.” “이 서비스의 동시성 이슈를 분산 락으로 해결해줘.”

세부 구현을 몰라도 됩니다. 키워드를 알면 방향을 지시할 수 있고, AI가 구현합니다.

더 나아가면 이렇게 됩니다. “헥사고날, DDD, 객체지향 원칙을 적용해서 만들고, 만든 결과물을 같은 기준으로 리뷰해” – 이런 사이클을 자동으로 돌리는 에이전트를 구축합니다. 한 번 돌리면 끝이 아니라, 만들고-리뷰하고-수정하는 사이클이 반복됩니다. 이 사이클을 설계할 줄 아는 것. 그것이 지금 시대의 엔지니어링 역량입니다.

세부 구현을 줄줄 외우는 것이 엔지니어링이었던 시대는 끝났습니다. 엔지니어링 원칙을 이해하고 그것을 자동화된 사이클에 내장하는 것. 그것이 같은 단어, 다른 의미입니다.

모른다면? 그것도 괜찮습니다

“근데 저는 헥사고날이 뭔지도 모르는데요.”

괜찮습니다. 진짜로요.

이렇게 물어보십시오. “나는 자바 스프링 프로젝트를 하고 있어. 백엔드 아키텍처에서 베스트 프랙티스가 뭐야?”

AI가 알려줍니다. 헥사고날 아키텍처, 클린 아키텍처, DDD, CQRS, 이벤트 소싱. 큰 숲이 펼쳐집니다. 각각이 대충 뭔지 이해됩니다. 그러면 그 다음은?

시키면 됩니다.

“우리 프로젝트에 헥사고날 아키텍처 적용해줘.”

적용된 결과물을 보면서 “아, 이게 포트고 이게 어댑터구나” 하고 체감합니다. 직접 몇 주 고민하면서 배우는 것이 아니라, AI가 만든 결과물을 보면서 반나절 만에 개념을 체득합니다. 그리고 다음 프로젝트에서는 키워드만으로 지시할 수 있게 됩니다.

이것이 AI와 함께 성장하는 것입니다.

순서가 바뀐 겁니다. 과거에는 “개념을 배우고 → 적용한다”였습니다. 지금은 “적용부터 하고 → 개념을 체득한다”입니다. 결과에서 원인으로. 구체에서 추상으로. 이 방향 전환이 학습 속도를 근본적으로 바꿔 놓습니다.

세부 지식의 바닥 수렴

불편하지만 직시해야 할 현실이 있습니다.

‘SOLID 원칙의 각 항목을 설명할 수 있다’, ‘팩토리 패턴과 빌더 패턴의 차이를 안다’, ‘JPA의 N+1 문제를 해결할 수 있다’ - 이런 세부 구현 지식의 시장 가치는 2025년을 통과하면서 바닥에 수렴하고 있습니다.

5년 전이라면, 면접에서 “DI 컨테이너의 동작 원리를 설명해보세요”라는 질문에 답할 수 있는 것이 실력이었습니다. 지금은 AI에게 물어보면 3초 만에 정확한 답이 나옵니다. 누구나 3초 만에 알 수 있는 것은, 더 이상 차별화 요소가 아닙니다.

그렇다면 지금 뭐에 집중해야 할까요?

이것이 진짜 질문입니다. “이 세부 지식이 중요한가 아닌가”가 아닙니다. **“지금 이 시간에 무엇에 투자해야 하는가”**입니다.

객체지향을 배우는 데 6개월을 쓸 것인가, 아니면 객체지향 키워드를 이해한 상태에서 AI와 함께 실전 프로젝트 10개를 만드는 데 6개월을 쓸 것인가. 테스트 코드를 완벽하게 작성하는 법을 마스터하는 데 3개월을 쓸 것인가, 아니면 테스트 전략을 설계하고 AI에게 실행을 맡기는 워크플로우를 3주 만에 구축할 것인가.

둘 다 객체지향을 알고, 둘 다 테스트를 합니다. 하지만 6개월 뒤 이 두 사람의 포트폴리오를 놓고 비교하면, 한 쪽은 개념을 설명하는 사람이고 다른 쪽은 시스템을 만든 사람입니다. 시장은 후자에게 돈을 지불합니다.

먼저 배우십시오. 그리고 전부 잊으십시오

여기서 한 가지 인정할 것이 있습니다.

훈련 과정에서는 이 세부 지식이 필요합니다. 키워드만 알아도 AI를 부릴 수 있는 것은 사실이지만, 그 키워드가 가리키는 개념의 깊이를 한 번은 경험해봐야 합니다. 경험하지 않은 사람이 던지는 키워드와, 경험하고 난 뒤에 던지는 키워드의 무게가 다릅니다. 결과가 나왔을 때 “이건 맞다, 이건 아니다”라고 판단하는 감각은, 한 번이라도 직접 부딪혀 본 사람에게서만 나옵니다.

그래서 저희(Team Grit)의 학습 프로그램은 이렇게 설계되어 있습니다. 가장 먼저, 이전 세대를 풍미했던 엔지니어링 핵심 무기들을 다룹니다. 객체지향, 테스트 코드, 클린 아키텍처, 동시성 제어, 대용량 트래픽 설계 - 실무에서 진짜 킷이 되는 것들을 깊게 파고듭니다.

단, AI와 함께합니다.

3년치 커리큘럼을 2주 만에 압축합니다. AI가 코드를 생성하고, 학습자는 그 코드의 구조를 분석하고, AI와 토론하며 원리를 체득합니다. 과거라면 혼자서 삼질하며 6개월 걸렸을 깨달음을 2주 만에 관통합니다. 이것이 AI 시대의 학습입니다. 속도의 차이가 아니라, 차원의 차이입니다.

그리고 그 다음 레벨에서는, 전부 잊습니다.

잊는다는 것이 무슨 뜻이냐. 객체지향의 SOLID 원칙을 줄줄 외우는 것을 멈추고, 한 단계 더 올라간다는 뜻입니다. 코드가 아니라 시스템을 봅니다. 구현이 아니라 의도를 정의합니다. 개발자의 시점에서 아키텍트의 시점으로, 아키텍트의 시점에서 의사결정자의 시점으로.

최종적으로 도달하는 레벨은 이것입니다. 1인 기업가. CEO의 시점. 이 시점에서 코드는 전혀 중요하지 않습니다. 시스템의 행동이 비즈니스 목표와 일치하느냐, 고객의 문제를 해결하느냐, 이 제품이 시장에서 살아남느냐 - 이것만 중요합니다. 코드는 그 과정에서 자동으로 생성되고 자동으로 배포되고 자동으로 모니터링되는 인프라일 뿐입니다.

재즈의 경지

이 과정을 가장 잘 설명하는 비유가 있습니다. 재즈입니다.

재즈 뮤지션의 성장 과정을 보십시오.

첫 번째 단계는 악보입니다. 한 음도 틀리지 않게 연주합니다. 박자를 정확히 맞추고, 다이내믹을 지시대로 따르고, 원곡 그대로를 재현합니다. 이 단계에서의 실력이란 '얼마나 정확하게 재현하느냐'입니다. 프로그래밍으로 치면 코드를 정확히 작성하는 단계입니다. 문법에 맞게. 패턴에 맞게. 베스트 프랙티스에 맞게. 이것이 지난 50년간 소프트웨어 엔지니어링이 '실력'이라 불려왔던 것입니다.

두 번째 단계는 즉흥입니다. 코드 진행(chord progression)의 구조를 이해하고, 그 위에서 자신만의 멜로디를 만들어냅니다. 악보에 없는 음을 칩니다. 하지만 아무렇게나 치는 것이 아닙니다. 화성학의 원리를 몸으로 체화했기 때문에, 어떤 음을 쳐도 전체 흐름을 벗어나지 않습니다. 프로그래밍으로 치면, 상황에 따라 "여기는 DDD로", "여기는 단순 CRUD로", "여기는 이벤트 소싱으로" - 유연하게 판단하고 AI에게 지시하는 단계입니다. 키워드만으로.

세 번째 단계는 자유입니다. 마일스 데이비스가 'Kind of Blue'를 녹음했을 때, 밴드 멤버들에게 건넨 것은 악보가 아니라 스케일 몇 개와 방향성뿐이었습니다. 나머지는 전부 즉흥이었습니다. 그런데 결과물은 재즈 역사상 가장 완벽한 앨범 중 하나가 되었습니다. 규칙을 완전히 내면화한 사람만이 규칙을 초월할 수 있습니다. 내가 하고 싶은 대로 해도 틀을 벗어나지 않는 경지. 이것은 규칙을 무시하는 것이 아닙니다. 규칙이 몸의 일부가 된 것입니다.

AI 네이티브 개발자가 도달해야 할 경지가 이것입니다.

첫 번째 단계에서는 악보대로 칩니다. 객체지향을 배우고, 테스트를 배우고, 아키텍처를 배웁니다. AI와 함께 압축적으로. 2주면 됩니다.

두 번째 단계에서는 즉흥을 시작합니다. 키워드로 판단하고, 에이전트 사이클로 실행합니다. 구현의 세부 사항에서 벗어나, 시스템의 행동을 설계합니다.

세 번째 단계에서는 악보를 태웁니다. 코드도, 아키텍처 패턴도, 엔지니어링 기법도 의식하지 않습니다. "이 비즈니스 문제를 해결해"라고만 합니다. 나머지는 시스템이 알아서 합니다. 당신이 한 것은 방향을 정한 것뿐입니다. 하지만 그 방향이 정확한 이유는, 첫 번째와 두 번째 단계를 통과했기 때문입니다.

악보를 배우지 않은 사람은 자유로울 수 없습니다. 하지만 악보에 갇힌 사람도 자유롭지 않습니다.

먼저 악보를 완벽하게 익히십시오. AI와 함께라면 2주면 됩니다. 그리고 악보를 태우십시오. 당신이 봐야 할 것은 음표가 아닙니다. 음악 그 자체입니다.

Team Gift

이 장을 마치며

이 장이 전달하고 싶은 것은 하나입니다.

객체지향, 테스트, 아키텍처 - 이것들은 무기입니다. 좋은 무기입니다. 하지만 무기를 분해 조립하는 법을 외우는 것과, 전장에서 무기를 쓰는 것과, 전쟁의 방향을 결정하는 것은 전혀 다른 레벨의 역량입니다.

지금 대부분의 개발 교육은 무기를 분해 조립하는 법을 가르칩니다. 그것도 6개월에서 1년씩 걸려서. 그 시간에 AI와 함께 전장에 나가서 무기를 실전에서 익히고, 전쟁의 방향을 읽는 눈을 기르는 것이 이 시대의 전략입니다.

세부 지식의 가치는 바닥에 수렴하고 있습니다. 하지만 그 지식을 통과한 사람의 판단력은 오히려 희소해지고 있습니다. 핵심은 통과하는 속도입니다. AI와 함께라면, 과거의 3년이 지금의 2주가 됩니다.

빠르게 통과하십시오. 그리고 올라가십시오.

다음 장에서는 이 투자 전략을 더 구체화합니다. AI 시대에 정말 공부해야 할 것과 그만해도 될 것을 분명하게 가르겠습니다.

셀프 체크

- ☐ 엔지니어링 세부 지식의 '시장 가치'가 왜 하락하고 있는지 설명할 수 있다
- ☐ “키워드만 알면 된다”의 의미와 한계를 모두 이해했다
- ☐ 재즈 비유의 세 단계(악보 → 즉흥 → 자유)를 자신의 현재 위치에 대입할 수 있다
- ☐ “먼저 배우고, 전부 잊으십시오”가 왜 모순이 아닌지 설명할 수 있다

이 장의 미션

1. 자신이 현재 보유한 엔지니어링 지식을 나열해 보십시오. 그 중에서 “키워드만으로 AI에게 위임할 수 있는 것”과 “아직 키워드조차 모르는 것”을 분류하십시오. 후자는 AI에게 “이 분야의 베스트 프랙티스를 알려줘”라고 물어보고 큰 숲을 파악하십시오.
2. 재즈의 세 단계에서 자신이 지금 어디에 있는지 솔직하게 평가하십시오. 그리고 다음 단계로 가기 위해 이번 주에 할 수 있는 행동 하나를 정하십시오.

AI와 함께 활용하기

- **베스트 프랙티스 탐색:** “나는 [기술 스택]을 사용하는 [연차]년차 개발자야. 이 기술 스택에서 시니어 개발자들이 실무에서 반드시 적용하는 엔지니어링 베스트 프랙티스 10가지를 알려줘. 각각을 키워드 + 한 줄 설명으로.”
- **에이전트 사이클 구축:** “이 프로젝트를 [아키텍처 패턴]으로 구조를 잡고, 같은 원칙으로 코드 리뷰까지 해줘. 문제가 있으면 수정하고 다시 리뷰하는 것을 반복해.”
- **압축 학습:** “나는 [개념]을 처음 배우는 사람이야. 이 개념이 실무에서 왜 중요한지, 우리 프로젝트에 어떻게 적용되는지, 적용 전후의 차이가 뭔지를 한 번에 보여줘.” # 9장. 기초 체력 재정의 - AI 시대에 정말 공부해야 할 것

투자 지도를 펼치겠습니다

8장에서 “세부 지식의 가치는 바닥에 수렴한다”고 했습니다. 그리고 “빠르게 통과하고 올라가라”고 했습니다.

그렇다면 올라간 그곳에서, 구체적으로 무엇에 투자해야 합니까? 이 장에서는 그 투자 지도를 펼치겠습니다.

2장에서 AI를 ‘곰셈기’라고 했습니다. 당신의 역량이 10이면 AI가 100으로 만들어주지만, 역량이 0이면 곰해도 0이라고요. 8장에서는 그 기초를 2주 만에 압축적으로 통과하는 방법을 이야기했습니다.

핵심 질문은 이것입니다.

곰해질 그 ‘기초 역량’의 실체는 무엇인가?

이 장에서 그것을 정의하겠습니다.

8.1 가치가 하락하는 역량 vs 가치가 상승하는 역량

역량의 환율이 바뀌고 있습니다

외환 시장에서 환율이 변하면, 어제까지 비싸던 통화가 오늘 싸집니다. 역량 시장에서도 같은 일이 벌어지고 있습니다. 어제까지 높은 가치를 인정받던 역량이, AI의 등장으로 급격히 평가절하되고 있습니다.

멘토링에서 경력 2년차 프론트엔드 개발자를 만난 적이 있습니다. 자기소개서에 “React 컴포넌트를 빠르게 작성할 수 있다”고 적어왔습니다. 자신감이 넘쳤습니다. 실제로 타이핑 속도도 빨랐고, 보일러플레이트를 외워서 치는 데 익숙했습니다. 그래서 하나 물어봤습니다. “이 컴포넌트가 불필요하게 리렌더링 되는 상황을 어떻게 진단하시겠어요?” 대답을 못했습니다. React의 렌더링 메커니즘, Virtual DOM의 비교 알고리즘, 상태 관리의 원리를 이해하지 못한 채, 코드의 형태만 외우고 있었던 겁니다.

안타깝지만 솔직하게 말씀드렸습니다. “보일러플레이트를 빠르게 치는 능력은 이제 AI가 0초 만에 합니다. 하지만 리렌더링 문제를 진단하는 능력은 AI에게 물어보더라도 본인이 이해하지 못하면 적용할 수 없습니다.”

그날 그 분의 표정이 아직 기억납니다. 자신이 투자한 시간의 방향이 잘못되었다는 것을 깨달은 순간이었으니까요.

AI 대체 가능성 매트릭스

어떤 역량이 가치가 하락하고, 어떤 역량이 가치가 상승하는지 한눈에 보겠습니다.

역량	AI 대체 가능성	가치 변화	투자 방향
문법/API 암기	높음	하락	축소 - 외우지 말고 이해만 하라
보일러플레이트 작성	매우 높음	급락	축소 - AI에게 위임하라
기계적 알고리즘 풀이	높음	하락	전환 - 원리 이해에 집중하라
단순 CRUD 구현	매우 높음	급락	축소 - AI가 더 빠르고 정확하다
시스템 설계/아키텍처	낮음	상승	확대 - 핵심 투자 영역
문제 분해 능력	낮음	상승	확대 - AI를 부리는 핵심 역량

역량	AI 대체 가능성	가치 변화	투자 방향
비즈니스 도메인 이해	매우 낮음	급등	최우선 - AI가 대체 불가능
코드 검증/리뷰 능력	중간	상승	확대 - AI 시대의 핵심 기술
CS 기초 (원리 이해)	낮음	상승	유지 - 배우는 방식만 전환
커뮤니케이션/협업	매우 낮음	상승	확대 - 사람만 할 수 있다

이 표에서 패턴이 보이십니까? 왼쪽 열에서 “암기”, “작성”, “기계적”이라는 단어가 붙은 역량은 모두 가치가 하락합니다. “설계”, “이해”, “판단”이라는 단어가 붙은 역량은 모두 가치가 상승합니다.

한 마디로 정리하면 이렇습니다. **“손이 하는 일”의 가치는 떨어지고, “머리가 하는 일”의 가치는 올라갑니다.**

이것이 1장에서 예고한 “공부의 방향 전환”의 구체적인 지도입니다. 이 지도를 보면서 자신의 시간 투자 포트폴리오를 점검해 보십시오. 가치가 하락하는 역량에 시간의 80%를 쓰고 있다면, 지금 당장 비율을 바꿔야 합니다.

8.2 기초 CS가 여전히 필수인 이유 - 하지만 배우는 방식이 다르다

“외우는 것”에서 “이해하는 것”으로

“AI가 다 해주니까 자료구조 공부 안 해도 되나요?”

이 질문을 정말 자주 받습니다. 그리고 대답은 명확합니다. **안 됩니다.** 하지만 배우는 방식은 완전히 달라져야 합니다.

비유를 하나 들겠습니다. 자동차를 운전하기 위해 엔진을 직접 조립할 줄 알 필요는 없습니다. 하지만 엔진이 어떻게 작동하는지 전혀 모르면, 이상한 소리가 났을 때 “이건 그냥 무시해도 되는 건가, 아니면 지금 당장 차를 세워야 하는 건가”를 판단할 수 없습니다. 기초 CS 지식은 바로 이 판단력의 토대입니다.

각 영역별로 왜 여전히 필요한지, 그리고 어떻게 배워야 하는지를 짚어보겠습니다.

자료구조: HashMap과 ArrayList의 차이를 코드로 외울 필요는 없습니다. AI가 알아서 적절한 자료구조를 선택해줍니다. 하지만 “왜 이 상황에서 HashMap이 ArrayList보다 나은지”를 이해하지 못하면, AI가 잘못된 선택을 했을 때 그것을 잡아낼 수 없습니다. 6장에서 다뤘던 “AI 결과물 검증”의 기반이 바로 이 이해입니다. 시간 복잡도가 $O(1)$ 인지 $O(n)$ 인지의 차이가, 사용자 10명일 때는 아무 상관 없지만 10만 명일 때는 서비스 장애로 이어진다는 감각. 이것은 외울 수 없습니다. 이해해야 합니다.

네트워크: HTTP 상태 코드를 외울 필요는 없습니다. 하지만 “왜 이 API 호출이 느린지”를 진단하려면 TCP 핸드셰이크, DNS 조회, 캐싱의 원리를 이해해야 합니다. AI에게 “이 API가 느린 이유를 분석해줘”라고 물어볼 수 있습니다. 하지만 AI의 답변 중 “DNS TTL을 늘려보세요”와 “서버 스펙을 올려보세요” 중 어느 것이 이 상황에 맞는 처방인지는 본인이 판단해야 합니다.

데이터베이스: SQL 문법을 달달 외울 필요는 없습니다. AI가 쿼리를 대신 작성해줍니다. 하지만 인덱스가 왜 필요한지, 정규화와 비정규화의 트레이드오프가 무엇인지, 트랜잭션 격리 수준이 왜 중요한지를 이해하지 못하면, AI가 만들어준 쿼리가 프로덕션 환경에서 DB를 죽이는 것을 막을 수 없습니다.

운영체제: 프로세스와 스레드의 차이를 코드 수준에서 외울 필요는 없습니다. 하지만 “왜 이 애플리케이션이 메모리를 계속 잡아먹는지”, “왜 멀티스레드 환경에서 데이터가 꼬이는지”를 이해하려면 기본적인 OS 개념이 필요합니다.

패턴이 보이십니까? 모든 영역에서 핵심은 같습니다. “**어떻게 쓰는가(How)**”는 AI에게 맡기고, “**왜 그런가(Why)**”를 이해하는 데 집중하라. 이것이 AI 시대의 CS 학습법입니다.

AI를 활용한 CS 학습법

과거에 CS를 배우려면 두꺼운 교과서를 펼치고, 추상적인 개념을 머릿속에서 시뮬레이션해야 했습니다. 이해가 안 되면 교수님에게 질문하거나, 운이 좋으면 잘 설명된 블로그를 찾았습니다.

지금은 다릅니다. AI를 개인 튜터로 활용할 수 있습니다. 핵심은, 추상적인 개념을 자신의 프로젝트 맥락에서 이해하는 것입니다.

예를 들어, 해시 테이블을 배운다고 합시다. 과거 방식: “해시 함수란 키를 고정 크기 값으로 매핑하는 함수이며, 충돌 해결 방법으로는 체이닝과 개방 주소법이 있다...” 교과서의 정의를 외우는 것입니다.

AI 시대 방식: “내 프로젝트에서 사용자 세션을 관리하고 있어. 지금은 ArrayList에 세션을 저장하고 있는데, 사용자가 늘어날수록 조회가 느려지고 있어. HashMap으로 바꾸면 왜 빨라지는지, 내 코드의 맥락에서 설명해줘.”

같은 해시 테이블이지만, 후자는 자신의 문제와 연결되어 있습니다. 추상적 개념이 구체적 경험이 되는 순간, 이해의 깊이가 달라집니다. 그리고 이렇게 배운 지식은 잊히지 않습니다.

8.3 도메인 지식: AI가 절대 대체할 수 없는 차별화 포인트

왜 도메인 지식이 최후의 해자인가

AI 대체 가능성 매트릭스에서 가치가 “급등”으로 표시된 항목이 하나 있었습니다. 비즈니스 도메인 이해. 이것이 왜 AI가 절대 대체할 수 없는 영역인지 설명하겠습니다.

AI는 코드를 잘 짭니다. 하지만 AI는 당신의 회사가 어떤 고객에게 어떤 가치를 제공하는지 모릅니다. AI는 “장바구니에 상품을 추가하는 API를 만들어줘”라는 지시에는 완벽하게 응합니다. 하지만 “우리 서비스에서 장바구니는 왜 7일 후에 자동 삭제되어야 하는가”는 대답하지 못합니다. 그 답은 비즈니스 정책에 있고, 그 정책은 고객의 행동 패턴에서 나오며, 그 패턴은 수개월간의 데이터 분석과 유저 인터뷰에서 도출된 것입니다.

5장에서 “Why를 먼저 정의하는 습관”을 다뤘습니다. 그때 “이 기능이 왜 필요한지”를 먼저 정리하라고 했습니다. 그 “왜”의 답을 줄 수 있는 것이 바로 도메인 지식입니다. AI는 “어떻게(How)” 구현할지는 알려줄 수 있지만, “왜(Why)” 이 기능이 필요한지는 도메인을 이해하는 사람만이 정의할 수 있습니다.

이것이 도메인 지식이 ‘최후의 해자(moat)’인 이유입니다. 성을 둘러싼 해자처럼, 도메인 지식은 AI가 쉽게 건너올 수 없는 경계입니다.

도메인 지식을 쌓는 구체적 방법

도메인 지식은 교과서에서 배울 수 없습니다. 현장에서 쌓이는 것입니다. 구체적으로 어떻게 쌓을 수 있는지 정리하겠습니다.

첫째, 이해관계자와 대화하십시오. 기획자, PM, 영업팀, 고객 지원팀. 이들은 코드를 모르지만, 고객을 알고 비즈니스를 압니다. “이 기능을 왜 만들어야 하나요?”라는 질문을 습관적으로 던지십시오. 대부분의 주니어 개발자는 JIRA 티켓을 받으면 바로 코드를 치기 시작합니다. 한 발만 물려서서 “이 티켓이 해결하려는 고객의 문제가 뭐죠?”라고 물으면, 같은 기능을 만들더라도 더 나은 설계가 나옵니다.

둘째, 유저 여정을 직접 경험하십시오. 자신이 만드는 서비스를 유저 입장에서 처음부터 끝까지 사용해 보십시오. 회원가입부터 결제까지, 또는 가입부터 이탈까지. 코드 뒤에 숨겨진 유저의 감정과 맥락을 이해하면, “아, 여기서 로딩이 2초 걸리면 유저가 이탈하겠구나”라는 판단이 자연스럽게 나옵니다.

셋째, 비즈니스 메트릭을 이해하십시오. DAU(일간 활성 사용자), 전환율, 이탈률, LTV(고객 생애 가치). 이런 지표들이 무엇을 의미하는지, 내가 만든 기능이 이 지표에 어떤 영향을 미치는지 파악하십시오. “이

캐시 최적화가 페이지 로딩 시간을 0.5초 줄였다”로 끝나지 말고, “그래서 이탈률이 3% 감소했다”까지 연결할 수 있어야 합니다.

이 세 가지를 꾸준히 실천하면, 당신은 “코드만 잘 치는 개발자”가 아니라 “비즈니스를 이해하는 개발자”가 됩니다. 그리고 후자는 AI가 아무리 발전해도 대체할 수 없는 존재입니다.

8.4 AI 시대의 학습 전략: “무엇을 모르는지 아는 것”의 가치

T자형 역량 설계

공부해야 할 것이 너무 많다고 느껴지십니까? 걱정하지 마십시오. 전부 다 깊게 알 필요는 없습니다.

AI 시대에 가장 효과적인 역량 구조는 **T자형(T-shaped)**입니다. 가로축은 넓은 이해, 세로축은 깊은 전문성입니다.

가로축, 즉 넓은 이해란 이런 것입니다. 프론트엔드 개발자이지만 DB 인덱스가 뭔지는 안다. 백엔드 개발자이지만 CSS 레이아웃의 기본 원리는 이해한다. 인프라를 직접 구축하지는 않지만 Docker 컨테이너가 왜 필요한지는 설명할 수 있다. 모든 분야의 전문가일 필요는 없습니다. 하지만 “이런 게 있다”는 것을 아는 것, 즉 “**무엇을 모르는지 아는 것**”이 중요합니다.

왜 이것이 AI 시대에 특히 가치가 올라갈까요? 5장에서 다뤘던 “AI에게 맥락을 제공하는 능력”과 직결되기 때문입니다. 문제가 발생했을 때 “이건 네트워크 문제일 수도 있고, DB 문제일 수도 있어”라는 가설을 세울 수 있으면, AI에게 정확한 질문을 던질 수 있습니다. 하지만 네트워크라는 개념 자체를 모르면, 가설조차 세울 수 없습니다.

세로축, 즉 깊은 전문성은 당신이 시장에서 대체 불가능한 존재가 되기 위한 축입니다. “백엔드 개발 + 결제 도메인”이든, “프론트엔드 개발 + 접근성 전문”이든, 하나의 영역에서 깊이를 가져야 합니다. 이 깊이가 바로 앞서 이야기한 도메인 지식과 만나는 지점입니다.

넓은 이해는 AI에게 좋은 질문을 던지게 해주고, 깊은 전문성은 AI의 답변을 검증하고 판단하게 해줍니다. 이 두 축이 만나는 교차점이, AI 시대에 가장 강력한 개발자의 프로필입니다.

AI를 학습 파트너로 활용하는 방법

마지막으로, AI를 학습 도구로 활용하는 구체적 방법을 정리하겠습니다.

첫째, “설명해줘” 대신 “내 상황에서 설명해줘”를 쓰십시오. 앞서 해시 테이블 예시에서 보여드린 것처럼, 추상적 개념을 자신의 프로젝트에 연결하면 이해의 깊이가 달라집니다. “인덱스가 뭐야?”보다 “내 프로젝트의 users 테이블에서 email로 검색할 때 인덱스가 왜 필요한지 설명해줘”가 10배 더 효과적입니다.

둘째, AI에게 “왜?”를 반복적으로 물으십시오. AI가 답변을 주면 “왜 그렇지?”라고 다시 묻습니다. 그러면 AI가 한 단계 더 깊은 설명을 해줍니다. 거기서 또 “왜?”를 물으면 더 깊어집니다. 이 과정이 어린이

의 “왜요?”와 비슷하다고 느끼실 수 있습니다. 하지만 이것이 개념의 뿌리까지 내려가는 가장 효과적인 방법입니다. 교과서는 깊이에 한계가 있지만, AI는 당신이 원하는 깊이까지 계속 파고들어줍니다.

셋째, 배운 것을 AI에게 설명해 보십시오. “내가 이해한 바로는 인덱스가 이렇게 작동하는 거야. 내 이해가 맞아? 틀린 부분이 있으면 교정해줘.” 이것은 ‘가르치면서 배우기(Learning by Teaching)’의 AI 버전입니다. 자신의 이해를 언어로 정리하는 과정에서 구멍이 드러나고, AI가 그 구멍을 정확히 짚어줍니다.

이 세 가지 방법의 공통점이 있습니다. 모두 “암기”가 아니라 “이해”를 위한 방법이라는 것입니다. AI 시대의 학습은 정보를 머릿속에 저장하는 것이 아니라, 개념 간의 관계를 파악하고 자신의 맥락에 연결하는 것입니다.

이 장을 마치며

8장에서 “먼저 배우고, 전부 잊으십시오”라고 했습니다. 이 장에서는 ‘배워야 할 것’의 실체를 정의했습니다.

곰셈기에 곱해질 기초 역량이란, 문법을 외우는 능력이 아니라 시스템을 이해하는 능력입니다. 코드를 빠르게 치는 능력이 아니라 문제를 분해하는 능력입니다. 그리고 AI가 절대 대체할 수 없는 도메인 지식입니다.

공부의 양은 줄어도 됩니다. 하지만 공부의 방향은 정확해야 합니다. 가치가 하락하는 역량에 시간을 쏟는 것을 멈추고, 가치가 상승하는 역량에 집중하십시오. 그것이 AI 시대의 투자 전략입니다.

다음 장에서는 이 투자 전략을 연차별 성장 경로와 연결하겠습니다. 주니어에서 시니어로 가는 사다리가 어떻게 달라졌는지, 그리고 각 단계에서 구체적으로 무엇을 해야 하는지를 이야기합니다.

셀프 체크

Q. AI 시대에 가치가 하락하는 역량과 상승하는 역량을 각각 3개씩 말할 수 있나요?

가치 하락: (1) 문법/API 암기 (2) 보일러플레이트 작성 (3) 기계적 알고리즘 풀이가치 상승: (1) 시스템 설계/아키텍처 (2) 비즈니스 도메인 이해 (3) 문제 분해 능력

Q. 기초 CS 지식이 AI 시대에도 여전히 필요한 이유를 한 문장으로 설명할 수 있나요?

AI가 코드를 대신 작성해주지만, AI의 결과물이 올바른지 검증하고 시스템 문제를 진단하려면 “왜 그런가(Why)”에 대한 CS 원리 이해가 필수적이기 때문입니다.

Q. 도메인 지식이 “AI가 대체할 수 없는 최후의 해자”인 이유는 무엇인가요?

AI는 “어떻게(How)” 구현할지는 알려줄 수 있지만, “왜(Why)” 이 기능이 필요한지는 비즈니스 맥락, 고객의 행동 패턴, 조직의 정책을 이해하는 사람만이 정의할 수 있기 때문입니다.

Q. T자형 역량에서 “가로축(넓은 이해)”이 AI 시대에 특히 중요해지는 이유를 설명할 수 있나요?

문제가 발생했을 때 여러 가능성을 가설로 세울 수 있어야, AI에게 정확한 질문을 던질 수 있습니다. 특정 개념의 존재 자체를 모르면 가설조차 세울 수 없고, 결국 AI에게 좋은 질문을 할 수 없기 때문입니다.

이 장의 미션

1. 자신의 기술 역량을 “가치 하락” / “가치 유지” / “가치 상승” 세 카테고리 분류해 보십시오. 그리고 현재 학습 시간의 분배가 이 분류와 일치하는지 점검하십시오. 불일치한다면, 향후 3개월간의 학습 시간 비율을 조정하는 계획을 세우십시오.
2. 자신이 일하고 있는 서비스(또는 만들고 싶은 서비스)의 비즈니스 도메인에 대해 “왜 이 기능이 필요한가?”를 3가지 이상 적어 보십시오. 적기 어렵다면, 기획자나 PM에게 물어보십시오. 그 대화 자체가 도메인 지식을 쌓는 첫걸음입니다.

AI와 함께 활용하기

- “나는 [직무] 개발자야. 내가 알고 있는 기술은 [기술 목록]이야. 이 중 AI 시대에 가치가 올라갈 것과 내려갈 것을 분류하고, 부족한 역량을 채우기 위한 3개월 학습 로드맵을 짜줘.”
- “나는 [프로젝트/서비스]를 개발하고 있어. 이 서비스의 비즈니스 도메인을 깊이 이해하기 위해 알아야 할 핵심 개념 10가지를 정리해줘. 각 개념이 실제 코드 설계에 어떤 영향을 미치는지도 설명해줘.”
- “나는 [CS 개념]을 배우고 있어. 이 개념을 내 프로젝트의 맥락에서 설명해줘. 내 프로젝트는 [프로젝트 설명]이야. 추상적인 정의 말고, 내 코드에서 이 개념이 어떻게 적용되는지 구체적인 예시로 보여줘.” # 10장. 성장 경로의 재설계 - 주니어에서 시니어로, 달라진 사다리

달라진 레벨업의 기준

개발자의 성장을 측정하는 방법이 바뀌었습니다.

불과 3년 전까지, “경험”이라는 단어는 곧 “코드를 얼마나 많이 짜봤는가”와 동의어였습니다. 스프링 부트로 API를 100개 만들어봤다, React 프로젝트를 5개 했다, SQL 쿼리를 수천 개 작성했다. 이런 양적 경험이 이력서에 적히고, 면접에서 인정받고, 연봉 협상의 근거가 되었습니다.

그 시대는 끝나가고 있습니다.

AI가 코드를 작성하는 시대에, “코드를 많이 짜본 경험”의 가치는 급격히 떨어집니다. 3장에서 이야기했듯이, Level 4 개발자는 하루 업무 시간의 10%만 직접 코드를 작성합니다. 나머지 90%는 설계하고, 검증하고, 판단합니다. 레벨업의 기준이 “코드를 얼마나 짰는가”에서 **“얼마나 복잡한 문제를 판단하고 해결했는가”**로 이동한 것입니다.

이것을 다르게 표현하면 이렇습니다.

과거에는 경험의 **양**이 성장의 척도였습니다. 지금은 경험의 **질**이 성장의 척도입니다.

CRUD API를 100개 만드는 것과, 분산 시스템의 동시성 문제를 한 번 해결하는 것. 어느 쪽이 더 큰 성장 일까요? 과거에는 전자가 “경험이 많다”고 인정받았습니다. 지금은 후자가 “더 어려운 문제를 풀어봤다”고 인정받습니다.

여기서 좋은 소식이 있습니다. 2장에서 이야기한 복리 효과를 기억하십니까? AI가 코드 작성을 처리해 주면, 개발자는 같은 시간 안에 더 많은 프로젝트를 경험할 수 있습니다. 더 많은 프로젝트를 경험하면 더 다양한 문제를 접합니다. 더 다양한 문제를 접하면 판단력이 빠르게 성장합니다. 판단력이 성장하면 더 어려운 문제를 맡게 됩니다.

AI를 활용하는 개발자 A가 2년 만에 도달하는 성장을, 전통적 방식의 개발자 B는 10년이 걸려야 따라잡는다는 계산. 그 차이의 본질은 코딩 속도가 아니었습니다. **경험 축적 속도**의 차이였습니다. AI가 있으면 같은 시간에 3배, 5배의 문제를 접할 수 있고, 그 경험이 복리로 쌓이는 것입니다.

성장의 사다리 자체가 바뀌었습니다. 낡은 사다리에 매달려 있으면, 열심히 올라가도 엉뚱한 곳에 도착합니다.

주니어 1~3년차: AI와 함께 기초를 압축하는 법

이 섹션이 이 장에서 가장 중요합니다. 이 책의 주요 독자인 주니어 개발자 분들에게, 가장 구체적인 이야기를 하겠습니다.

“빨리 배우는” 것이지 “건너뛰는” 것이 아닙니다

먼저 가장 많이 듣는 질문에 최종 답변을 드리겠습니다.

“AI가 다 해주니까 공부 안 해도 되나요?”

아닙니다. 정반대입니다.

2장에서 AI를 곱셈기라고 했습니다. 곱셈기의 원리를 떠올리십시오. 기초 역량이 10인 사람이 AI를 쓰면 100이 됩니다. 기초 역량이 0인 사람이 AI를 쓰면 여전히 0입니다. AI가 해주는 것은 기초를 **대체하는** 것이 아니라, 기초를 **배우는 속도를 가속하는** 것입니다.

구체적으로 어떻게 가속되는지 보겠습니다.

예전에 HTTP 상태 코드를 배우려면 어떻게 했습니까? 문서를 읽고, 예제를 찾고, 직접 코드를 짜보고, 에러를 만나고, Stack Overflow를 검색하고, 다시 코드를 고치고. 이 과정에 며칠이 걸렸습니다.

지금은 다릅니다. AI에게 “HTTP 상태 코드 체계를 내가 만들고 있는 REST API 맥락에서 설명해줘. 각 코드를 언제 써야 하는지 실제 예시와 함께”라고 요청합니다. 5분 만에 체계적인 설명을 받습니다. 그리고 바로 자신의 프로젝트에 적용해봅니다. 적용하다 생기는 의문을 다시 AI에게 물어봅니다. 30분이면 며칠치 학습을 압축할 수 있습니다.

핵심은 이것입니다. **배움의 과정을 건너뛰는 것이 아니라, 배움의 속도를 높이는 것입니다.** HTTP 상태 코드를 이해하지 않아도 된다는 뜻이 아닙니다. HTTP 상태 코드를 이전보다 훨씬 빠르게, 훨씬 깊이 이해할 수 있게 되었다는 뜻입니다.

코드 작성 -> AI 리뷰 -> 학습의 피드백 루프

주니어 시절 가장 빠르게 성장하는 방법은 좋은 코드 리뷰를 많이 받는 것입니다. 문제는, 시니어 개발자의 시간은 한정되어 있고, 코드 리뷰를 충분히 받기 어려운 환경에 있는 분들이 많다는 것입니다.

AI가 이 병목을 해소합니다.

코드를 작성한 뒤, AI에게 리뷰를 요청하는 습관을 들이십시오. 단, “이 코드 리뷰해줘”라고만 하면 피상적인 답변을 받습니다. 이렇게 요청해 보십시오.

“이 코드를 시니어 개발자의 관점에서 리뷰해줘. 동작은 하지만 프로덕션에 배포하기에 부족한 점이 있다면 구체적으로 지적해줘. 보안, 성능, 가독성, 에러 처리 관점에서.”

AI가 지적한 내용을 읽고, **왜** 그것이 문제인지 이해하고, 직접 수정해 보십시오. 이해가 안 되면 다시 물어보십시오. “왜 try-catch를 여기에 넣어야 하는지 좀 더 설명해줘. 안 넣으면 어떤 일이 생기는지 구체적인 시나리오로.”

이 루프를 반복하면, 시니어 개발자에게 매일 1대1 멘토링을 받는 것과 비슷한 효과를 얻을 수 있습니다. 24시간 언제든지, 몇 번이든, 같은 질문을 해도 짜증내지 않는 멘토가 생긴 셈입니다.

멘토링에서 만난 1년차 프론트엔드 개발자가 있었습니다. 소규모 회사에서 혼자 프론트를 담당하고 있었는데, 코드 리뷰를 받을 사람이 아무도 없다고 했습니다. 이 피드백 루프를 알려드렸더니, 2주 만에 눈에 띄게 코드 품질이 올라갔습니다. 본인이 가장 놀라더군요. “코드 리뷰를 받아본 적이 거의 없었는데, AI가 지적하는 걸 보니까 제가 몰랐던 게 이렇게 많았구나 싶었어요.”

환경의 한계를 도구로 극복한 것입니다. 2장에서 이야기한 노출 갭을, AI라는 도구가 메워준 사례입니다.

빠르게 성장하는 일간/주간 루틴

구체적인 루틴을 제안합니다. 주니어 1~3년차에 AI를 활용해 기초를 압축하는 루틴입니다.

매일 (30분)

1. 오늘 작성한 코드 중 가장 자신 없는 부분을 AI에게 리뷰 요청한다.
2. AI의 피드백 중 모르는 개념이 있으면, 그 자리에서 AI에게 물어보고 이해한다.
3. 이해한 내용을 한 줄로 메모한다. (예: “트랜잭션 격리 수준 - READ COMMITTED로는 팬텀 리드 못 막음”)

매주 (1시간)

1. 한 주간의 메모를 훑어보고, 반복적으로 나오는 약점을 파악한다.
2. 그 약점에 대해 AI에게 “이 개념을 내 프로젝트 맥락에서 깊이 있게 설명해줘”라고 요청한다.
3. 3장에서 이야기한 도구 스펙트럼에서 자신의 현재 Level을 점검한다. 지난주보다 한 단계라도 올라갔는가?

이 루틴은 화려하지 않습니다. 하루 30분, 일주일에 1시간. 하지만 이것을 6개월 동안 꾸준히 실행한 주니어와 그렇지 않은 주니어의 차이는, 2장에서 계산한 복리 효과가 보장합니다. 월 5% 성장률의 복리가 12개월이면 1.8배, 24개월이면 3.2배였습니다.

중니어 3~5년차: 코더에서 아키텍트로의 전환점

3년차를 넘기면, 개발자에게 근본적인 역할 전환이 요구됩니다. 코드를 직접 작성하는 사람에서, 코드의 방향을 결정하는 사람으로.

AI 시대에 이 전환은 더 빠르게, 더 극적으로 일어납니다.

구현자에서 설계자로

3~5년차에 접어들면, 업무의 무게중심이 바뀌어야 합니다. “이 API를 어떻게 구현하지?”에서 “이 시스템을 어떻게 설계하지?”로. “이 버그를 어떻게 고치지?”에서 “이 아키텍처에서 왜 이런 버그가 반복되지?”로.

AI가 이 전환을 가속합니다. 구현은 AI에게 맡기고, 당신은 설계에 집중할 수 있기 때문입니다. 3장에서 이야기한 도구 스펙트럼 Level 3에서 Level 4로의 전환이 바로 이 시점에 일어납니다. Level 3(AI 페어 프로그래머)에서는 AI와 함께 코드를 작성합니다. Level 4(에이전트 코딩)에서는 AI에게 구현을 맡기고, 당신은 결과를 검증합니다.

이 전환이 체감되는 순간이 있습니다. 어느 날 문득, 코드를 직접 치는 시간보다 AI가 작성한 코드를 리뷰하는 시간이 더 길어진 자신을 발견합니다. 그때가 바로 전환점입니다. 두려워하지 마십시오. 그것이 정상입니다.

AI 에이전트를 “부하 직원”으로 관리하는 감각

3~5년차에서 가장 중요한 역량은 위임입니다. 그리고 AI 에이전트에게 일을 위임하는 것은, 실제 팀원에게 일을 위임하는 것과 놀라울 정도로 비슷합니다.

신입 팀원에게 업무를 맡길 때 어떻게 하십니까? “이거 해”라고만 하면 원하는 결과가 나오지 않습니다. 배경을 설명하고, 제약 조건을 알려주고, 기대하는 결과물의 형태를 명확히 해야 합니다. 중간중간 진행 상황을 확인하고, 방향이 어긋나면 교정해야 합니다.

AI 에이전트도 마찬가지입니다. “이 기능 구현해줘”만으로는 부족합니다. 왜 이 기능이 필요한지, 어떤 제약이 있는지, 기존 코드와 어떻게 통합되어야 하는지를 알려줘야 합니다. 5장에서 다룬 “컨텍스트의 3 요소 - 배경(Why), 제약(Constraints), 기대 결과(What)”가 여기서 다시 활용됩니다.

이 감각은 나중에 실제 팀을 이끌 때 그대로 전이됩니다. AI 에이전트에게 효과적으로 위임하는 연습이, 결국 사람에게 효과적으로 위임하는 역량으로 이어지는 것입니다. 3~5년차에 이 감각을 키워두면, 시니어가 되었을 때 팀 리딩이 훨씬 자연스러워집니다.

기술 의사결정 역량

중니어 시기에 반드시 키워야 할 또 다른 역량은 “왜 이 기술을 선택했는가”를 설명할 수 있는 능력입니다.

Redis를 도입했다면, “빨라서”가 아니라 “읽기 빈도가 쓰기 빈도의 50배이고, 데이터 정합성보다 응답 속도가 비즈니스에 더 중요하기 때문”이라고 말할 수 있어야 합니다. Kafka를 선택했다면, “요즘 다들 쓰니까”가 아니라 “서비스 간 결합도를 낮추면서 이벤트 순서를 보장해야 하는 요구사항이 있기 때문”이라고 근거를 댈 수 있어야 합니다.

AI는 이 역량을 키우는 데도 강력한 도구입니다. “Redis vs Memcached, 우리 프로젝트 상황에서 어떤 것을 선택해야 할지 트레이드오프를 분석해줘. 읽기/쓰기 비율은 50:1이고, 데이터 만료 정책이 필요하고, 클러스터 확장 가능성을 고려해야 해”라고 AI에게 물으면, 혼자서는 며칠 걸릴 분석을 30분 만에 얻을 수 있습니다.

중요한 것은 AI의 분석 결과를 그대로 복사 붙여넣기하는 것이 아니라, 그 분석을 읽고 자신의 판단으로 소화하는 것입니다. “AI가 Redis를 추천했으니까 Redis 쓰겠습니다”는 주니어의 답변입니다. “AI의 분석에 따르면 Redis가 우리 상황에 더 적합한데, 제가 추가로 고려한 운영 복잡도까지 감안하면 이런 이유로 Redis가 맞다고 판단합니다”가 중니어의 답변입니다.

시니어 5년차 이상: AI 팀 리더로서의 새로운 역할

시니어 개발자의 역할이 근본적으로 확장되고 있습니다. 간략하게 짚고 넘어가겠습니다. 주니어 독자 분들께는 “앞으로 이런 방향으로 발전한다”는 비전으로 읽어주시면 됩니다.

멀티 에이전트 오케스트레이션

3장에서 도구 스펙트럼의 Level 5를 소개했습니다. 여러 AI 에이전트가 동시에 다른 작업을 수행하고, 개발자가 오케스트라의 지휘자처럼 전체를 조율하는 단계입니다.

2026년 2월 현재, 이것은 더 이상 먼 미래의 이야기가 아닙니다. 실제로 시니어 개발자들 중 일부는 하나의 에이전트가 백엔드를 구현하는 동안 다른 에이전트에게 프론트엔드를 맡기고, 또 다른 에이전트가 테스트를 작성하는 방식으로 일하고 있습니다. 혼자서 소규모 팀의 산출물을 내는 것이 가능해진 셈입니다.

AI 시대의 테크 리드

시니어 개발자, 특히 테크 리드의 역할은 세 가지 축으로 재편되고 있습니다.

첫째, **시스템 설계**. 이것은 변하지 않습니다. 오히려 더 중요해집니다. AI가 구현의 대부분을 처리하면, 설계의 품질이 최종 결과물의 품질을 결정하게 됩니다.

둘째, **팀 생산성 극대화**. AI 도구의 도입과 팀 내 확산을 이끄는 역할입니다. 어떤 도구를 도입할지, 어떤 워크플로우를 표준화할지, 팀원들의 AI 활용 수준을 어떻게 끌어올릴지.

셋째, **AI 워크플로우 설계**. AI 에이전트가 효과적으로 작동하도록 프로젝트 컨텍스트를 설계하고, 자동화 파이프라인을 구축하는 역할입니다. 4장에서 다른 프로젝트 컨텍스트 파일 작성, 시니어 레벨에서는 팀 전체의 AI 워크플로우 설계로 확장되는 것입니다.

2장에서 이야기한 “AI 증폭형 아키텍트”가 바로 이 역할입니다. 이것이 5년 후 시장에서 가장 귀한 존재가 되는 이유입니다.

채용 시장에서 살아남기: 포트폴리오와 면접의 변화

성장 경로를 재설계했다면, 그 성장을 시장에서 증명하는 방법도 알아야 합니다.

AI 활용 경험을 어떻게 증명하는가

포트폴리오의 기준이 바뀌고 있습니다. “이 프로젝트를 만들었습니다”에서 “이 프로젝트를 이렇게 만들었습니다”로.

2025년 이후의 포트폴리오에서 차별화 포인트는 이렇습니다.

과정을 보여주십시오. 최종 결과물만 GitHub에 올리는 것이 아니라, AI와 어떻게 협업했는지를 기록하십시오. 어떤 설계 판단을 내렸는지, AI에게 어떻게 지시했는지, AI의 결과물을 어떻게 검증하고 교정했는지. 이 과정 자체가 당신의 역량을 증명합니다. PR 하나에 “AI가 생성한 초안을 이런 관점에서 리뷰하고, 이런 부분을 수정했다”는 기록을 남기는 것이 효과적입니다.

판단의 근거를 남기십시오. 기술 의사결정 문서(ADR, Architecture Decision Record)를 작성하는 습관을 들이십시오. “왜 이 기술을 선택했는가”, “어떤 대안을 검토했는가”, “AI의 분석을 어떻게 활용했는가”를 기록합니다. 이 문서들이 쌓이면, 그 자체가 당신의 성장 기록이자 포트폴리오가 됩니다.

AI 시대 면접에서 달라지는 질문들

면접의 질문도 바뀌고 있습니다. 2023년까지만 해도 면접에서 코드를 직접 작성하는 라이브 코딩이 주된 평가 방식이었습니다. 지금도 여전히 중요하지만, 새로운 유형의 질문이 추가되고 있습니다.

“AI 코딩 도구를 실무에서 어떻게 활용하고 있습니까?” 이 질문에 “ChatGPT로 에러 메시지 검색합니다”라고 답하면 Level 1입니다. “Cursor로 멀티 파일 리팩토링을 하고, Claude Code로 에이전틱 코딩을 합니다. 구체적으로 최근 프로젝트에서 AI에게 OAuth 인증 구현을 맡기고, 보안 관점에서 결과를 검증한 경험이 있습니다”라고 답하면 Level 4입니다.

“AI가 생성한 코드에서 문제를 발견한 경험이 있습니까?” 이 질문은 검증 능력을 평가합니다. 6장에서 다룬 내용이 여기서 빛을 발합니다. AI가 생성한 코드의 구체적인 문제를 발견하고 수정한 경험을 이야기할 수 있으면 강력한 답변이 됩니다.

“이 시스템을 설계한다면 어떤 접근을 하겠습니까? AI를 어떻게 활용하겠습니까?” 이 질문은 설계 역량과 AI 활용 전략을 동시에 평가합니다. “먼저 AI와 아키텍처 토론을 하고, 트레이드오프를 분석한 뒤, 구현은 에이전트에게 맡기고 결과를 검증하겠습니다”라는 프레임을 보여줄 수 있으면 됩니다.

채용 패러독스의 기회

2장에서 이야기한 채용 패러독스를 기억하십니까? 기업들이 주니어 채용을 줄이고 있고, CS 전공 지원율이 떨어지고 있습니다. 모두가 “개발자 전망이 어둡다”며 도망치고 있습니다.

하지만 바로 그것이 기회입니다.

인재 파이프라인이 줄어든다는 것은, 파이프라인 안에 남아 있는 사람의 희소 가치가 올라간다는 뜻입니다. 특히 AI 역량까지 갖춘 채 버터낸 개발자는, 5년 후 시장에서 가장 귀한 존재가 됩니다.

지금 당장 채용 시장이 어렵다고 느끼실 수 있습니다. 하지만 이 혼란기를 버텨내면서 AI 역량을 쌓는 사람은, 시장이 안정되었을 때 압도적인 위치에 서게 됩니다. 모든 사람이 주식을 팔 때 사는 것이 가장 현명한 투자이듯, 모든 사람이 개발자를 포기할 때 실력을 쌓는 것이 가장 현명한 커리어 전략입니다.

2장에서 이야기한 “2028년 두 개의 계층” – AI 증폭형 아키텍트와 코드 상품화 노동자. 어디에 서느냐는 재능이 아니라 선택의 문제입니다. 지금 이 시점에서 방향을 정하고 움직이기 시작한 사람이, AI 증폭형 아키텍트 쪽에 설 수 있습니다.

이 장을 마치며

성장의 사다리가 바뀌었습니다. 코드의 양이 아니라 판단의 질로 올라가는 사다리입니다.

주니어라면, AI를 학습 가속기로 활용하십시오. 기초를 건너뛰는 것이 아니라 기초를 압축하는 것입니다. 매일 30분의 피드백 루프가 6개월 후 복리로 돌아옵니다.

중니어라면, 구현자에서 설계자로 전환하십시오. AI에게 위임하는 감각을 키우고, 기술 의사결정의 근거를 말할 수 있는 사람이 되십시오.

시니어라면, AI 팀 리더로서의 새로운 역할을 받아들이십시오. 시스템을 설계하고, 팀의 AI 워크플로우를 설계하는 사람.

어느 단계에 있든, 핵심은 같습니다. AI가 코드를 짜는 시대에, 개발자의 가치는 코드가 아니라 **판단**에 있습니다.

서문에서 “주니어에서 시니어로 성장하는 경로가 어떻게 달라지는가”를 다루겠다고 약속했습니다. 그 답이 이것입니다. 더 많은 코드를 짜는 것이 아니라, 더 나은 판단을 내리는 사람이 되는 것. 그리고 AI는 그 판단력을 키우는 가장 강력한 도구입니다.

셀프 체크

Q. AI 시대에 “레벨업”의 기준은 무엇으로 바뀌었습니까?

코드를 많이 짜본 양적 경험이 아니라, 더 복잡한 문제를 판단하고 해결한 질적 경험이 레벨업의 기준입니다. 코드 작성량이 아니라 판단의 질이 성장의 척도입니다.

Q. AI를 활용해 “기초를 압축한다”는 것은 구체적으로 어떤 의미입니까?

기초를 건너뛰는 것이 아니라, 학습 속도를 높이는 것입니다. 코드 작성 후 AI 리뷰를 받고, 모르는 개념을 즉시 AI에게 물어보고, 자신의 프로젝트 맥락에서 깊이 이해하는 피드백 루프를 통해, 이전에 며칠 걸리던 학습을 수십 분으로 압축할 수 있습니다.

Q. 3~5년차에 “구현자에서 설계자로” 전환해야 하는 이유는 무엇입니까?

AI가 구현의 대부분을 처리할 수 있는 시대에, 구현 능력만으로는 차별화가 되지 않습니다. 시스템을 어떻게 설계할지, 왜 이 기술을 선택했는지를 판단하고 설명할 수 있는 사람이 가치를 인정받습니다. 도구 스펙트럼 Level 3에서 Level 4로의 전환이 이 시점에 일어납니다.

Q. 채용 시장에서 AI 활용 경험을 증명하는 방법은 무엇입니까?

1. 과정을 기록합니다 - AI와의 협업 과정, 설계 판단, 코드 검증 과정을 PR이나 문서에 남깁니다.
2. 기술 의사결정 문서(ADR)를 작성합니다 - “왜 이 기술을 선택했는가”의 근거를 AI 분석과 자신의 판단을 결합하여 기록합니다.
3. 면접에서 구체적 경험을 이야기합니다 - AI에게 무엇을 맡겼고, 결과를 어떻게 검증했고, 어떤 문제를 발견했는지를 구체적으로 설명합니다.

이 장의 미션

1. 자신의 연차에 해당하는 섹션(9.2 주니어 / 9.3 중니어 / 9.4 시니어)을 다시 읽고, 오늘부터 실천할 수 있는 행동 1가지를 정하십시오. 주니어라면 “매일 30분 AI 코드 리뷰 루틴” 시작이 가장 효과적입니다.
2. 자신의 현재 위치에서 다음 레벨로 가기 위해 필요한 역량을 1가지만 정하고, 향후 6개월 성장 로드맵을 1페이지로 작성하십시오. AI에게 초안을 요청해도 좋습니다. 단, 최종 판단은 당신이 내리십시오.

AI와 함께 학습하기

- “나는 [연차]년차 [직무] 개발자입니다. AI 시대에 다음 레벨로 성장하려면 어떤 역량을 키워야 하는지, 구체적인 6개월 액션 플랜을 짜줘. 현재 내가 잘하는 것: [목록]. 부족한 것: [목록]. 목표: [설명].”
- “이 코드를 시니어 개발자의 관점에서 리뷰해줘. 동작은 하지만 프로덕션에 배포하기에 부족한 점이 있다면 구체적으로 지적해줘. 보안, 성능, 가독성, 예러 처리, 확장성 관점에서 분석하고, 각 지적에 대해 왜 그것이 문제인지 설명해줘.”
- “나는 현재 [기술 A]와 [기술 B] 중 하나를 선택해야 하는 상황이야. 프로젝트 맥락은 [설명]이야. 두 기술의 트레이드오프를 분석하고, 어떤 상황에서 어떤 것이 유리한지 정리해줘. 내가 기술 의사결정 문서를 작성할 수 있도록 구조화해줘.” # 11장. 에필로그 - 마차 시대가 끝나고, 당신은 어디로 갈 것인가

11.1 1부에서 3부까지, 하나의 메시지

긴 여정이었습니다.

1부에서 우리는 세상이 바뀌었다는 사실을 직시했습니다. 패러다임이 전환되고 있고, 양극화가 가속되고 있으며, 같은 도구를 쓰면서도 하늘과 땅 차이가 나는 격차가 이미 벌어지고 있다는 현실을요. “게임의 규칙이 바뀌었다”는 인식. 그것이 1부의 메시지였습니다.

2부에서는 그래서 어떻게 해야 하는지를 이야기했습니다. 환경을 세팅하고, AI와 설계하고, 코드를 검증하고, 품질을 끌어올리는 실전 워크플로우를 처음부터 끝까지 다뤘습니다. “새로운 방식으로 일하라.” 그것이 2부의 메시지였습니다.

3부에서는 시선을 더 넓혔습니다. 어떤 역량에 투자해야 하는지, 주니어에서 시니어로 가는 경로가 어떻게 달라지는지를 이야기했습니다. “새로운 기준으로 성장하라.” 그것이 3부의 메시지였습니다.

이 세 개의 메시지를 하나로 압축하면 이렇습니다.

패러다임이 바뀌었다. 새로운 방식으로 일하라. 새로운 기준으로 성장하라.

1장에서 이야기한 **From Syntax to Intent**. 문법에서 의도로. 이 전환은 단순히 코딩 방식의 변화가 아니었습니다. 일하는 방식의 변화이고, 성장하는 방식의 변화이며, 개발자로서 자기 자신을 정의하는 방식의 변화입니다.

11.2 예측이 아닌 준비

이 책을 읽는 동안, 어쩌면 이 질문이 머릿속 한편에 계속 있었을지 모릅니다.

“결국 AI가 개발자를 대체하는 거 아닌가?”

이 책 전체를 읽고 나셨다면, 이제 이것이 잘못된 질문이라는 걸 아실 겁니다. 마차 시대에 “자동차가 마부를 대체할까?”라고 묻는 것과 같은 질문입니다. 자동차는 마부를 대체하지 않았습니다. 마부라는 직업 자체를 의미 없게 만들었을 뿐입니다. 그리고 그 자리에 운전기사, 정비사, 교통 엔지니어라는 전혀 새로운 직업이 태어났습니다.

올바른 질문은 이것입니다.

“AI와 함께 일하는 개발자로서, 나는 얼마나 준비되어 있는가?”

예측은 틀립니다. 항상 틀려왔습니다. 6개월 뒤에 어떤 모델이 나올지, 1년 뒤에 어떤 도구가 지배적일지, 아무도 모릅니다. 하지만 준비는 배신하지 않습니다. AI와 협업하는 사고방식을 장착한 사람은, 도구가 바뀌어도 금방 적응합니다. 이 책에서 반복적으로 이야기한 원리입니다.

서문에서 이렇게 말씀드렸습니다. “이 책의 유효기간은 1년이다.”

지금, 그 말을 회수하겠습니다.

이 책에서 언급한 특정 도구의 기능이나 버전 정보는 이미 낡았을 수 있습니다. 제가 “현재 가장 좋은 방법”이라고 적은 것 중 일부는, 당신이 이 문장을 읽는 시점에 이미 더 나은 방법으로 대체되었을 수 있습니다. 그 부분에 대해서는 서문의 약속대로, 유효기간 1년을 인정합니다.

하지만 도구는 낡아도 사고방식은 남습니다.

“의도를 먼저 정의하라.” “AI의 결과물을 검증하라.” “문법이 아니라 설계에 투자하라.” “Why를 먼저 묻고, How는 AI에게 맡겨라.” 이 원칙들은 1년 후에도, 3년 후에도 유효할 겁니다. 도구의 이름이 바뀌고, 인터페이스가 바뀌고, 모델의 세대가 교체되어도, 인간이 의도를 정의하고 AI가 구현을 처리한다는 구조 자체는 바뀌지 않을 겁니다. 오히려 그 구조는 더 선명해질 겁니다.

이 책이 낡더라도, 이 책을 읽고 바뀐 당신의 사고방식은 낡지 않습니다.

11.3 내일부터 할 수 있는 세 가지

9개의 장에 걸쳐 많은 이야기를 했습니다. 셀프 체크, 미션, 실전 시나리오. 정보가 많으면 오히려 첫 발을 떼기 어려워지기도 합니다. 그래서 모든 것을 세 가지로 압축하겠습니다.

첫째, AI를 첫 번째 동료로 삼으십시오.

내일 아침, 코드를 짜기 전에 AI에게 먼저 말을 거십시오. “이 기능을 어떻게 구현하면 좋을까?” 거창할 필요 없습니다. 한 문장이면 됩니다. 5장에서 다뤘던 설계 토론의 시작은 이렇게 단순합니다. 코드를 치기 전에 생각을 먼저 정리하는 습관. 그것이 From Syntax to Intent의 가장 작은 실천입니다.

둘째, AI의 결과물을 무조건 검증하십시오.

AI가 생성한 코드를 그대로 커밋하지 마십시오. 한 번만 더 읽으십시오. “왜 이렇게 짰지?” “빠진 건 없나?” 6장에서 다뤘던 검증의 습관입니다. 이 습관 하나가 여러분을 “AI를 쓰는 사람”에서 “AI를 부리는 사람”으로 바꿉니다. 도구를 쓰는 것과 도구를 부리는 것의 차이는, 결과물을 검증하느냐 마느냐에서 갈립니다.

셋째, 문법이 아니라 설계에 투자하십시오.

오늘 알고리즘 문제를 풀 시간 30분이 있다면, 그 시간에 자신의 프로젝트 구조를 AI와 함께 분석해 보십시오. “이 아키텍처의 약점은 뭘까?” “트래픽이 10배가 되면 어디가 먼저 터질까?” 9장에서 이야기한 “가치가 상승하는 역량”에 투자하는 겁니다. 문법은 AI가 알아서 합니다. 당신은 판단하는 사람이 되십시오.

이 세 가지를 동시에 시작할 필요는 없습니다. 하나만 고르십시오. 내일부터 딱 하나.

마지막 미션

이 책의 마지막 미션입니다.

이 책을 덮고, 30분 안에 할 수 있는 가장 작은 행동 1가지를 정하고, 지금 바로 실행하십시오.

AI 코딩 도구를 설치하는 것도 좋습니다. 자신의 프로젝트에 컨텍스트 파일을 만드는 것도 좋습니다. AI에게 “내 코드를 리뷰해줘”라고 한 줄 치는 것도 좋습니다. 10장에서 이야기한 성장 로드맵을 한 페이지로 쓰는 것도 좋습니다.

크기는 상관없습니다. 방향만 맞으면 됩니다. 그리고 속도는 AI가 만들어줍니다.

마지막 AI 활용 프롬프트

이 책의 마지막 AI 프롬프트입니다. 이것을 복사해서, 당신이 선택한 AI 코딩 도구에 붙여넣으십시오.

“이 책에서 배운 내용을 바탕으로, 내 상황에 맞는 AI 네이티브 개발자 전환 로드맵을 짜줘. 현재 상태: [당신의 연차, 기술 스택, 현재 환경을 적으세요]. 목표: AI와 함께 일하는 개발자로 전환. 기간: 3개월. 첫째 주에 당장 시작할 수 있는 구체적인 행동 3가지부터 알려줘.”

11.4 존재론적 변화의 한가운데에서

1장의 첫 장면을 기억하시는지요.

오늘도 IntelliJ를 열었습니다. JIRA 티켓을 확인하고, Spring Boot 프로젝트에서 컨트롤러를 작성하기 시작합니다. import문을 정리하고, DTO 클래스를 만들고, 서비스 레이어를 구성합니다. 옆자리 동료는 터미널에 자연어로 뭔가를 입력하고 있습니다. 5분 뒤, 동료의 화면에는 컨트롤러, 서비스, 리포지토리, 테스트 코드가 이미 생성되어 있습니다.

“당신은 아직 DTO를 만들고 있습니다.”

그 장면이 이 책의 시작이었습니다. 불안과 위기감으로 시작된 여정이었습니다.

이제 그 장면을 다시 그려보겠습니다.

오늘도 터미널을 열었습니다. JIRA 티켓을 확인합니다. 하지만 이번에는 코드를 치지 않습니다. 먼저 AI에게 이 기능의 설계를 토론했습니다. 트레이드오프를 분석하고, 스펙을 정리합니다. 그리고 AI에게 구현을 맡깁니다. 생성된 코드를 리뷰하고, 비즈니스 로직을 검증하고, 테스트를 확인합니다. 옆자리에 새로운 동료가 물어봅니다.

“그거... 어떻게 하는 건가요?”

당신은 웃으며 화면을 보여줍니다. 1시간 동안, 당신이 AI와 함께 일하는 방식을 보여줍니다. 설계부터 시작하는 것, 의도를 정확하게 전달하는 것, 결과물을 검증하는 것. 동료의 눈이 커집니다.

“이걸... 다들 이렇게 하고 있었어요?”

이번에는 당신이 대답할 차례입니다.

“네. 그리고 이제 당신도 할 수 있습니다.”

1장에서 옆자리 동료를 바라보며 불안했던 당신이, 이제 그 옆자리 동료가 되었습니다. 그리고 누군가에게 같은 감각을 전달해줄 수 있는 사람이 되었습니다. 이것이 이 책이 원했던 전부입니다.

1장에서 이렇게 말씀드렸습니다.

마차 시대가 끝났는데 말 엉덩이를 때리고 있지 마십시오. AI라는 엔진에 올라타서 어디로 갈지를 정하는 운전자가 되십시오.

이제 마지막 한 마디를 덧붙이겠습니다.

마차 시대는 끝났습니다. 하지만 당신에게는 엔진이 있습니다. 연료도 있습니다. 도로도 열려 있습니다. 이 책을 통해 지도를 얻었습니다. 남은 것은 단 하나, 시동을 거는 것뿐입니다.

변화의 속도가 아니라 변화의 방향이 중요합니다. 방향만 맞으면, 속도는 AI가 만들어줍니다. 그리고 지금 이 순간이, 당신의 출발점입니다.

당신은 어떤 개발자가 될 것입니까.

시동을 거십시오.